

Vel Tech Group of Institutions

Name: SUSHANT KUMAR
Email: vtu29971@veltech.edu.in
Roll no: vtu29971
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L10

VTU_DAA_Week 2_COD

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Given an integer n. The task is to find the first n Fibonacci Numbers.

Input Format

The input consists of a single integer n, representing how many Fibonacci terms to print.

Output Format

The output prints n space-separated Fibonacci numbers starting from 0.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

Output: 0 1 1

Answer

```
// You are using GCC
#include <stdio.h>
```

```
int main() {
    int n;
    scanf("%d", &n);

    int a = 0, b = 1, next;

    for (int i = 0; i < n; i++) {
        printf("%d ", a);
        next = a + b;
        a = b;
        b = next;
    }

    return 0;
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Given a string *s*, check if it is a palindrome using recursion. A palindrome is a word, phrase, or sequence that reads the same backward as forward.

Input Format

The input consists of a single string *s* containing no spaces.

Output Format

The output prints "The string is a palindrome" if the string is symmetric.

Otherwise, prints "The string is not a palindrome".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: abba

Output: The string is a palindrome

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>

// Recursive function to check palindrome
int isPalindrome(char str[], int start, int end) {
    if (start >= end)
        return 1;

    if (str[start] != str[end])
        return 0;

    return isPalindrome(str, start + 1, end - 1);
}

int main() {
    char s[51];
    scanf("%s", s);

    int len = strlen(s);

    if (isPalindrome(s, 0, len - 1))
        printf("The string is a palindrome");
    else
        printf("The string is not a palindrome");

    return 0;
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Shilpa is learning about the Fibonacci sequence for her computer science class. She wants to generate the first N terms of the Fibonacci series.

Can you help Shilpa by writing a program that takes a number N and prints the first N terms of the Fibonacci series using recursion?

Input Format

The input consists of an integer N, representing the number of terms in the Fibonacci sequence that need to be printed.

Output Format

The output prints the first N Fibonacci numbers, separated by spaces.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

Output: 0 1 1 2 3 5

Answer

```
import java.util.Scanner;

// You are using GCC
int fib(int n) {
    if (n == 0)
        return 0;
    else if (n == 1)
        return 1;
    else
        return fib(n - 1) + fib(n - 2);
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int N = sc.nextInt();
    for(int i = 0; i < N; i++) {
        System.out.print(fib(i));
```

```
        if(i != N - 1) System.out.print(" ");
    }
    System.out.println();
}
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Ashish is preparing a mathematics assignment where he needs to calculate the factorial of a number. Since his teacher asked him to use recursion for this task, he plans to complete the method `int factorial` to solve it.

Help him to implement the task.

Input Format

The input consists of an integer num representing the number for which the factorial is to be calculated.

Output Format

The output prints the factorial value of num.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

Output: 6

Answer

```
#include <stdio.h>
```

```
// You are using GCC
```

```
int factorial(int n) {
```

```
    if (n <= 1) {
```

```
        return 1;
```

```
    }  
    return n * factorial(n - 1);  
}  
  
int main() {  
    int num;  
    scanf("%d", &num);  
    printf("%d\n", factorial(num));  
    return 0;  
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: SUSHANT KUMAR
Email: vtu29971@veltech.edu.in
Roll no: vtu29971
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L10

VTU_DAA_Week 2_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Manoj loves puzzles and recently encountered an interesting mathematical challenge. He needs to find how many ways he can express a number x as the sum of distinct positive integers raised to the power of n . For example, for a given number x , he wants to find all possible ways to represent it as a sum of distinct powers of integers.

Help him implement this task.

Example 1

Input:

10
2

Output:

1

Explanation:

$x = 10$

$n = 2$

$10 = 12 + 32$, hence we have only 1 possibility.

Example 2

Input:

100

2

Output:

3

Explanation:

$x = 100$

$n = 2$

$100 = 102$ OR $62 + 82$ OR $12 + 32 + 42 + 52 + 72$, hence total 3 possibilities.

Function Specifications: `getAllWays(int, int, int)`

Input Format

The first line of input consists of an integer x , representing the target sum.

The second line of input consists of an integer n , representing the power to which the integers are raised.

Output Format

The first line of output prints: The total number of ways to express x as the sum of distinct powers.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 100

2

Output: 3

Answer

// You are using GCC

#include <stdio.h>

#include <math.h>

// Function to calculate x^n manually to avoid floating point issues with pow()

```
int power(int base, int exp) {  
    int res = 1;  
    for (int i = 0; i < exp; i++) {  
        res *= base;  
    }  
    return res;  
}
```

// Recursive function to find the number of ways

```
int getAllWays(int x, int n, int current_num) {
```

// Calculate the power of the current number

```
    int p = power(current_num, n);
```

// If remaining sum is 0, we found 1 valid way

```
    if (x == 0) {
```

```
        return 1;
```

```
    }
```

// If the power exceeds the remaining sum, this path is invalid

```
    if (p > x) {
```

```
        return 0;
```

```
    }
```

// Return the sum of ways:

// 1. Including the current number: subtract p from x and move to next number

// 2. Excluding the current number: keep x as is and move to next number

```
    return getAllWays(x - p, n, current_num + 1) + getAllWays(x, n, current_num + 1);
```

```

}

int main() {
    int x, n;

    // Read target sum and power
    if (scanf("%d %d", &x, &n) == 2) {
        // Start recursion with current_num = 1
        printf("%d\n", getAllWays(x, n, 1));
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Given an integer array of coins of size N representing different types of currency and an integer sum M. The task is to find the number of ways to make the sum by using different combinations from the coins array, using recursion.

Function Specifications: int countWays(int [], int , int)

Input Format

The first line of input consists of the number of coins N.

The second line consists of N integers denoting the coin denominations.

The third line consists of the integer sum M to be produced with those coins.

Output Format

The output displays the number of ways to make the sum by using different combinations from the coins array.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

1 2 3

4

Output: 4

Answer

// You are using GCC

#include <stdio.h>

// Recursive function to count ways to reach sum M

int countWays(int coins[], int n, int m) {

// Base Case 1: If sum is 0, we found 1 valid combination (empty set)

if (m == 0) {

return 1;

}

// Base Case 2: If sum becomes negative, no solution exists

if (m < 0) {

return 0;

}

// Base Case 3: If no coins are left and sum is still > 0, no solution

if (n <= 0 && m >= 1) {

return 0;

}

// Return the sum of:

// 1. Including the current coin (coins[n-1]): m - value, same n

// 2. Excluding the current coin: same m, n - 1

return countWays(coins, n, m - coins[n - 1]) + countWays(coins, n - 1, m);

}

int main() {

int n, m;

// Read number of coins

if (scanf("%d", &n) != 1) return 0;

int coins[n];

// Read coin denominations

```
for (int i = 0; i < n; i++) {  
    scanf("%d", &coins[i]);  
}  
  
// Read target sum  
scanf("%d", &m);  
  
// Calculate and print result  
printf("%d\n", countWays(coins, n, m));  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Meredith is a cashier at a local convenience store. To improve the efficiency of making change for customers, Meredith wants to develop a small program that calculates the number of different ways she can give change using a given set of coin denominations. This will help her quickly figure out the number of ways to make change for a particular amount of money using the available coins.

Meredith has a list of coin denominations and a specific amount of money for which she needs to provide change. She wants to write a program to determine the number of possible ways to achieve the given amount using the available denominations. This will help her understand all possible combinations without having to manually calculate them each time.

Input Format

The first line contains an integer n , the number of different coin denominations.

The second line contains n space-separated integers representing the coin denominations.

The third line contains an integer m , the target amount of money for which Meredith needs to make change.

Output Format

The output displays the number of ways to make the sum by using different combinations from the coins array.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

1 2 3

4

Output: 4

Answer

```
#include <stdio.h>
```

```
int countWays(int coins[], int n, int m) {
    // Create an array to store results of sub-problems
    // table[i] will store the number of ways to make sum 'i'
    int table[m + 1];

    // Initialize all table values as 0
    for (int i = 0; i <= m; i++) {
        table[i] = 0;
    }

    // Base case: There is 1 way to make sum 0 (by using no coins)
    table[0] = 1;

    // Pick all coins one by one and update the table[] values
    // after the index greater than or equal to the value of the picked coin
    for (int i = 0; i < n; i++) {
        for (int j = coins[i]; j <= m; j++) {
            table[j] += table[j - coins[i]];
        }
    }

    return table[m];
}

int main() {
```

```

int n, m;

// Read the number of denominations
if (scanf("%d", &n) != 1) return 0;

int coins[n];
// Read the denominations
for (int i = 0; i < n; i++) {
    scanf("%d", &coins[i]);
}

// Read the target amount
if (scanf("%d", &m) != 1) return 0;

// Output the total number of ways
printf("%d\n", countWays(coins, n, m));

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

In a small research lab, a programmer named Arin is analyzing binary data to detect system flags stored as set bits in an integer. One day Arin receives an integer and must determine how many bits are set to 1 in its binary representation.

Write a program to ensure that this task is done so Arin can continue validating the system's binary logs. The objective is to count all set bits in the given number using any valid approach.

Function specification: countSetBits(int n)

Input Format

The input consists of a single positive integer n.

Output Format

The output prints a single integer representing the count of set bits (1s) in the binary representation of n.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 21

Output: 3

Answer

```
// You are using GCC
#include <stdio.h>

/**
 * Function to count the number of set bits (1s)
 * in the binary representation of n.
 */
int countSetBits(int n) {
    int count = 0;
    while (n > 0) {
        // This operation clears the least significant set bit
        n = n & (n - 1);
        count++;
    }
    return count;
}

int main() {
    int n;

    // Read the input integer
    if (scanf("%d", &n) == 1) {
        // Output the count of set bits
        printf("%d\n", countSetBits(n));
    }

    return 0;
}
```

Status : Correct

Marks : 10/10

5. Problem Statement

Raju is a computer graphics designer. He works around matrices with ease. His friend challenged him to rotate an $M \times M$ matrix clockwise with an angle of 90 degrees.

However, Raju was given a constraint that he must not use any additional 2D matrix for doing the above-mentioned task. The rotation must be done in place, which means Raju has to modify the given input matrix directly.

Example: 1

Input:

[[1,2,3],[4,5,6],[7,8,9]]

Output:

[[7,4,1],[8,5,2],[9,6,3]]

Example: 2

Input:

matrix = [[5,1,9,11],[2,4,8,10],[13,3,6,7],[15,14,12,16]]

Output:

[[15,13,2,5],[14,3,4,1],[12,6,8,9],[16,7,10,11]]

Input Format

The first line of the input is a single integer value for the size of the row and column, M .

The next M lines of the input are the matrix elements, each containing M integers representing the elements of the matrix.

Output Format

The output displays the rotated matrix, with each row printed on a new line.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 3

1 2 3

4 5 6

7 8 9

Output: 7 4 1

8 5 2

9 6 3

Answer

// You are using GCC

#include <stdio.h>

```
void rotateMatrix(int m, int matrix[m][m]) {
```

```
    // Step 1: Transpose the matrix
```

```
    for (int i = 0; i < m; i++) {
```

```
        for (int j = i + 1; j < m; j++) {
```

```
            int temp = matrix[i][j];
```

```
            matrix[i][j] = matrix[j][i];
```

```
            matrix[j][i] = temp;
```

```
        }
```

```
    }
```

```
    // Step 2: Reverse each row
```

```
    for (int i = 0; i < m; i++) {
```

```
        for (int j = 0; j < m / 2; j++) {
```

```
            int temp = matrix[i][j];
```

```
            matrix[i][j] = matrix[i][m - 1 - j];
```

```
            matrix[i][m - 1 - j] = temp;
```

```
        }
```

```
    }
```

```
}
```

```
int main() {
    int m;
    if (scanf("%d", &m) != 1) return 0;

    int matrix[m][m];

    // Input the matrix
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &matrix[i][j]);
        }
    }

    rotateMatrix(m, matrix);

    // Output the rotated matrix
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < m; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: SUSHANT KUMAR

Email: vtu29971@veltech.edu.in

Roll no: vtu29971

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L10

VTU_DAA_Week 2_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

In a kingdom far away, two types of soup are prepared daily — Soup A and Soup B. The royal chef always starts with n milliliters of each soup. Each time a servant is served, one of the following four serving operations is chosen at random (each with equal probability of 0.25):

Serve 100 ml of Soup A and 0 ml of Soup B

Serve 75 ml of Soup A and 25 ml of Soup B

Serve 50 ml of Soup A and 50 ml of Soup B

Serve 25 ml of Soup A and 75 ml of Soup B

The serving continues until either or both soups become empty. If the required soup amount for an operation exceeds the available quantity,

serve as much as possible (up to what remains).

You are to compute the probability that Soup A becomes empty first, plus half the probability that both soups become empty at the same time.

Special Note:

Since all operations serve soup in multiples of 25 ml, you must first scale n down to units of 25 ml before recursive processing:

Example

Sample Input 1

$n = 50$

Sample Output 2

0.625

Explanation

If we choose the first two operations, A will become empty first.

For the third operation, A and B will become empty at the same time.

For the fourth operation, B will become empty first.

So the total probability of A becoming empty first plus half the probability that A and B become empty at the same time, is $0.25 * (1 + 1 + 0.5 + 0) = 0.625$.

Input Format

The input consists of an integer n , representing the total volume of the soup.

Output Format

The output prints the probability that soup A will be empty first, plus half the probability that A and B become empty at the same time.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 50

Output: 0.625

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <math.h>
```

```
// Memoization table to store results of subproblems
```

```
double memo[201][201];
```

```
double getProbability(int a, int b) {
```

```
    // Base Cases
```

```
    if (a <= 0 && b <= 0) return 0.5;
```

```
    if (a <= 0) return 1.0;
```

```
    if (b <= 0) return 0.0;
```

```
    // Check if already calculated
```

```
    if (memo[a][b] > 0) return memo[a][b];
```

```
    // Recursive calculation based on 4 serving operations
```

```
    // Operations scaled by 25ml: (4,0), (3,1), (2,2), (1,3)
```

```
    memo[a][b] = 0.25 * (getProbability(a - 4, b) +
```

```
        getProbability(a - 3, b - 1) +
```

```
        getProbability(a - 2, b - 2) +
```

```
        getProbability(a - 1, b - 3));
```

```
    return memo[a][b];
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    if (scanf("%d", &n) != 1) return 0;
```

```
    // Special Case: As n grows very large, the probability approaches 1.0
```

```
    // For this specific problem/constraints (n <= 100), we proceed normally.
```

```
    if (n > 5000) {
```

```
        printf("1.000000\n");
```

```
        return 0;
```

```
    }
```

```

// Scale n down to units of 25ml (rounding up)
int scaled_n = (n + 24) / 25;

// Initialize memo table with 0
for (int i = 0; i < 201; i++) {
    for (int j = 0; j < 201; j++) {
        memo[i][j] = 0.0;
    }
}

printf("%.3f\n", getProbability(scaled_n, scaled_n));

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

In a unique team-building exercise, a company has decided to distribute a certain number of resources among its departments based on mathematical powers. The company has x resources that need to be distributed such that each department receives a different power of an integer. Specifically, each department i receives n resources, where n is a given power.

George, the resource manager, needs to find all possible ways to distribute exactly x resources using the powers of integers starting from 1. Each integer's power can only be used once.

Write a program to help George determine the number of ways to distribute x resources in this manner using a recursive manner.

Example 1

Input:

10

2

Output:

1

Explanation:

$x = 10$

$n = 2$

$10 = 12 + 32$, hence we have only 1 possibility.

Example 2

Input:

100

2

Output:

3

Explanation:

$x = 100$

$n = 2$

$100 = 102$ OR $62 + 82$ OR $12 + 32 + 42 + 52 + 72$, hence total 3 possibilities.

Input Format

The first line of input consists of the integer x , representing the total number of resources.

The second line consists of the integer n , representing the power to which the integers are raised.

Output Format

The output should be a single integer representing the number of ways to distribute exactly x resources.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 100

2

Output: 3

Answer

// You are using GCC

#include <stdio.h>

#include <math.h>

// Recursive function to find the number of ways

int countWays(int x, int n, int num) {

 // Calculate num^n

 long long val = 1;

 for (int i = 0; i < n; i++) {

 val *= num;

 }

 // If remaining sum is 0, we found 1 valid combination

 if (x == 0) {

 return 1;

 }

 // If the power exceeds the remaining sum, no more ways are possible

 if (val > x) {

 return 0;

 }

 // 1. Include the current power: subtract val from x and move to num + 1

 // 2. Exclude the current power: keep x as is and move to num + 1

 return countWays(x - (int)val, n, num + 1) + countWays(x, n, num + 1);

}

int main() {

 int x, n;

 // Read target sum and power

 if (scanf("%d %d", &x, &n) == 2) {


```
// Start recursion with integer 1
printf("%d\n", countWays(x, n, 1));
}

return 0;
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

In a small research lab, a programmer named Arin is analyzing binary data to detect system flags stored as set bits in an integer. One day Arin receives an integer and must determine how many bits are set to 1 in its binary representation.

Write a program to ensure that this task is done so Arin can continue validating the system's binary logs. The objective is to count all set bits in the given number using any valid approach.

Function specification: countSetBits(int n)

Input Format

The input consists of a single positive integer n.

Output Format

The output prints a single integer representing the count of set bits (1s) in the binary representation of n.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 21

Output: 3

Answer

```

// You are using GCC
#include <stdio.h>

// Function to count the number of set bits (1s) in binary representation
int countSetBits(int n) {
    int count = 0;
    while (n > 0) {
        // This clears the rightmost set bit
        n = n & (n - 1);
        count++;
    }
    return count;
}

int main() {
    int n;
    // Read the input integer
    if (scanf("%d", &n) == 1) {
        // Print the result of the countSetBits function
        printf("%d\n", countSetBits(n));
    }
    return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Given a square matrix of size $n \times n$, rotate the elements along each layer (or boundary) of the matrix by one position in a clockwise direction. The rotation should be performed layer by layer, starting from the outermost layer and proceeding inward.

Examples:

Input 2:

3 3

1 2 3

4 5 6

7 8 9

Output 2:

4 1 2

7 5 3

8 9 6

Explanation:

For a clockwise boundary rotation by one step:

We start with the outermost layer (there's only one layer in a 3×3 matrix)

We save the top-left element (1)

Move left edge upward: 4 1, 7 4

Move bottom edge leftward: 8 7, 9 8

Move right edge downward: 6 9, 3 6

Move top edge rightward: 2 3

Place the saved element (1) in the second position of the top row, so 1 2

Input Format

The first line contains two integers R and C — the number of rows and columns in the matrix.

The next R lines each contain C integers, representing the elements of the matrix.

Output Format

The output displays the rotated matrix after performing the clockwise boundary rotation.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 4 4

1 2 3 4

5 6 7 8

9 10 11 12

13 14 15 16

Output: 5 1 2 3

9 10 6 4

13 11 7 8

14 15 16 12

Answer

// You are using GCC

#include <stdio.h>

```
void rotateLayerWise(int r, int c, int matrix[r][c]) {
```

```
    int rowStart = 0, colStart = 0;
```

```
    int rowEnd = r - 1, colEnd = c - 1;
```

```
    while (rowStart < rowEnd && colStart < colEnd) {
```

```
        // Store the first element of the next row,
```

```
        // this will be the first element to be replaced in the current layer
```

```
        int prev = matrix[rowStart + 1][colStart];
```

```
        // Move elements of the top row to the right
```

```
        for (int i = colStart; i <= colEnd; i++) {
```

```
            int curr = matrix[rowStart][i];
```

```
            matrix[rowStart][i] = prev;
```

```
            prev = curr;
```

```
        }
```

```
        rowStart++;
```

```
        // Move elements of the rightmost column down
```

```
        for (int i = rowStart; i <= rowEnd; i++) {
```

```
            int curr = matrix[i][colEnd];
```

```
            matrix[i][colEnd] = prev;
```

```
            prev = curr;
```

```
        }
```

```
        colEnd--;
```

```
        // Move elements of the bottom row to the left
```

```
        for (int i = colEnd; i >= colStart; i--) {
```

```
            int curr = matrix[rowEnd][i];
```

```

        matrix[rowEnd][i] = prev;
        prev = curr;
    }
    rowEnd--;

    // Move elements of the leftmost column up
    for (int i = rowEnd; i >= rowStart; i--) {
        int curr = matrix[i][colStart];
        matrix[i][colStart] = prev;
        prev = curr;
    }
    colStart++;
}
}

int main() {
    int r, c;
    if (scanf("%d %d", &r, &c) != 2) return 0;

    int matrix[r][c];
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) {
            scanf("%d", &matrix[i][j]);
        }
    }

    rotateLayerWise(r, c, matrix);

    // Print the rotated matrix
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < r; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

Given a 2D matrix of size ROW $\times$ COL, print all elements of the matrix in diagonal order.

Diagonal Order Definition:

Start from the first element at position (0,0). Traverse all diagonals starting from the leftmost column going downward, then continue with diagonals starting from the bottom row going rightward. Within each diagonal, print elements from the top-right corner to bottom-left corner.

Example:

Input matrix

```
5 4
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16
17 18 19 20
```

Diagonal printing of the above matrix is

```
1
5 2
9 6 3
13 10 7 4
17 14 11 8
18 15 12
19 16
20
```

Input Format

The first line of input contains two integers, ROW and COL.

The next ROW lines each contain COL space-separated integers

Output Format

The output consists of multiple lines, one for each diagonal. Each line contains the matrix elements of that diagonal in the order from top-right to bottom-left, separated by spaces.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 5 4

1 2 3 4

5 6 7 8

9 10 11 12

13 14 15 16

17 18 19 20

Output: 1

5 2

9 6 3

13 10 7 4

17 14 11 8

18 15 12

19 16

20

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
void printDiagonalOrder(int rows, int cols, int matrix[rows][cols]) {
```

```
    // 1. Traverse diagonals starting from the leftmost column
```

```
    for (int k = 0; k < rows; k++) {
```

```
        int r = k;
```

```
        int c = 0;
```

```
        while (r >= 0 && c < cols) {
```

```
            printf("%d ", matrix[r][c]);
```

```
            r--;
```

```
            c++;
```

```

    }
    printf("\n");
}

// 2. Traverse diagonals starting from the bottom row (skipping first col)
for (int k = 1; k < cols; k++) {
    int r = rows - 1;
    int c = k;
    while (r >= 0 && c < cols) {
        printf("%d ", matrix[r][c]);
        r--;
        c++;
    }
    printf("\n");
}
}

int main() {
    int rows, cols;
    if (scanf("%d %d", &rows, &cols) != 2) return 0;

    int matrix[rows][cols];
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            scanf("%d", &matrix[i][j]);
        }
    }

    printDiagonalOrder(rows, cols, matrix);

    return 0;
}

```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: SUSHANT KUMAR

Email: vtu29971@veltech.edu.in

Roll no: vtu29971

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L10

VTU_DAA_Week 3_COD

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

You are given a set of N points on a 2D plane. Your task is to find the distance between the closest pair of points.

Write a program that takes as input the number of points N, followed by the x and y coordinates of each point. The program should output the distance between the closest pair of points, rounded to two decimal places.

Input Format

The first line of input consists of an integer N, representing the number of points.

The following N lines represent the coordinate points x and y, separated by space.

Output Format

The output prints a single floating-point number representing the distance between the closest pair of points, rounded to two decimal places.

Sample Test Case

Input: 3

2 3

12 30

40 50

Output: 28.79

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <math.h>
```

```
#include <float.h>
```

```
typedef struct {  
    double x, y;  
} Point;
```

```
// Function to calculate Euclidean distance between two points  
double calculateDistance(Point p1, Point p2) {  
    return sqrt(pow(p2.x - p1.x, 2) + pow(p2.y - p1.y, 2));  
}
```

```
int main() {  
    int n;  
  
    // Read the number of points  
    if (scanf("%d", &n) != 1) return 0;
```

```
    Point points[n];
```

```
    // Read coordinates for each point  
    for (int i = 0; i < n; i++) {  
        scanf("%le %le", &points[i].x, &points[i].y);  
    }
```

```
    // Initialize minDistance with a very large value  
    double minDistance = DBL_MAX;
```

```
    // Compare every pair of points (Brute Force)
```

```

for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
        double dist = calculateDistance(points[i], points[j]);
        if (dist < minDistance) {
            minDistance = dist;
        }
    }
}

// Print the result rounded to two decimal places
printf("%.2f\n", minDistance);

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Ishita is working on a mapping application that helps locate the nearest landmarks on a 2D map. Each landmark is represented by its (x, y) coordinate. Her task is to find out the shortest distance between any two landmarks from the given list.

Help Ishita write a program that calculates the distance between the closest pair of points (landmarks) on the map.

Input Format

The first line of input contains an integer N representing the number of landmarks.

The next N lines contain two space-separated integers, representing the x and y coordinates of each landmark.

Output Format

The output prints a single floating-point number representing the distance between any two landmarks rounded to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

5 7

12 32

21 23

43 65

Output: 12.73

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <math.h>
```

```
#include <float.h>
```

```
typedef struct {
```

```
    int x, y;
```

```
} Point;
```

```
// Function to calculate Euclidean distance between two points
```

```
double getDistance(Point p1, Point p2) {
```

```
    return sqrt(pow(p2.x - p1.x, 2) + pow(p2.y - p1.y, 2));
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    // Read the number of landmarks
```

```
    if (scanf("%d", &n) != 1) return 0;
```

```
    Point landmarks[n];
```

```
    // Read coordinates for each landmark
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d %d", &landmarks[i].x, &landmarks[i].y);
```

```
    }
```

```
    // Initialize minDistance with the largest possible double value
```

```
    double minDistance = DBL_MAX;
```

```

// Compare every unique pair (i, j)
for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
        double currentDist = getDistance(landmarks[i], landmarks[j]);
        if (currentDist < minDistance) {
            minDistance = currentDist;
        }
    }
}

// Print the result rounded to two decimal places
printf("%.2f\n", minDistance);

return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Ragu, an aspiring computer scientist, is currently learning about the concept of finding the closest pair of points in a plane. He wants to test his skills by solving a programming challenge related to this concept.

Given a set of points in a 2D plane, Ragu needs to find the closest pair of points and calculate their Euclidean distance.

Input Format

The first line of input consists of an integer N , representing the number of points in the plane.

The following N lines, each consisting of two space-separated real numbers x and y , represent the coordinates of a point.

Output Format

The first line of output prints the coordinates of the two points forming the closest pair in the below format:

"Closest pair of points: (x_1 , y_1) and (x_2 , y_2)"

The second line prints "Distance: <distance>" representing the Euclidean distance between the closest pair of points, rounded off to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

0 3

1 1

10 18

4 9

Output: Closest pair of points: (0, 3) and (1, 1)

Distance: 2.24

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <math.h>
```

```
#include <float.h>
```

```
typedef struct {
```

```
    double x, y;
```

```
} Point;
```

```
// Function to calculate Euclidean distance
```

```
double getDistance(Point p1, Point p2) {
```

```
    return sqrt(pow(p2.x - p1.x, 2) + pow(p2.y - p1.y, 2));
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    // Read the number of points
```

```
    if (scanf("%d", &n) != 1) return 0;
```

```
    Point pts[n];
```

```
    // Read point coordinates
```

```
    for (int i = 0; i < n; i++) {
```

```

scanf("%lf %lf", &pts[i].x, &pts[i].y);
}

double min_dist = DBL_MAX;
Point p1_best, p2_best;

// Compare every unique pair (i, j)
for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
        double d = getDistance(pts[i], pts[j]);
        if (d < min_dist) {
            min_dist = d;
            p1_best = pts[i];
            p2_best = pts[j];
        }
    }
}

// Print results as per formatting specifications
// Note: Use %.0f if the sample output implies integers,
// but %g or %f is safer for real numbers.
printf("Closest pair of points: (%.0f, %.0f) and (%.0f, %.0f)\n",
       p1_best.x, p1_best.y, p2_best.x, p2_best.y);
printf("Distance: %.2f\n", min_dist);

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Ragu, a passionate computer science student, is exploring geometry problems to sharpen his algorithmic thinking. Today, he challenges himself to determine the two closest points among a set of 2D coordinates.

Ragu must find the closest pair of points from the given set and compute the Euclidean distance between them with high precision. He also wants the output to display which two points are the closest, formatted clearly.

Input Format

The first line of input contains an integer N representing the number of points in the 2D plane.

The next N lines each contain two space-separated real numbers x and y representing the coordinates of each point.

Output Format

The first line of output prints the coordinates of the two points forming the closest pair in the below format:

"Closest pair of points: (x1, y1) and (x2, y2)"

The second line prints "Distance: <distance>" representing the Euclidean distance between the closest pair of points, rounded off to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

1 2

3 4

1 0

6 8

2 3

Output: Closest pair of points: (1, 2) and (2, 3)

Distance: 1.41

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <math.h>
```

```
#include <float.h>
```

```
typedef struct {
```

```
    double x, y;
```

```
} Point;
```



```
// Function to calculate Euclidean distance between two points
double getDistance(Point p1, Point p2) {
    return sqrt(pow(p2.x - p1.x, 2) + pow(p2.y - p1.y, 2));
}
```

```
int main() {
    int n;
```

```
    // Read the number of points
    if (scanf("%d", &n) != 1) return 0;
```

```
    Point pts[n];
```

```
    // Read coordinates for each point
    for (int i = 0; i < n; i++) {
        scanf("%lf %lf", &pts[i].x, &pts[i].y);
    }
```

```
    double min_dist = DBL_MAX;
    Point best_p1, best_p2;
```

```
    // Nested loops to compare every unique pair (i, j)
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            double d = getDistance(pts[i], pts[j]);
```

```
            // Update min_dist and the best pair if a shorter distance is found
            if (d < min_dist) {
                min_dist = d;
                best_p1 = pts[i];
                best_p2 = pts[j];
            }
        }
    }
```

```
    // Output formatting: coordinates without decimals (%.0f) and distance with
    two (%.2f)
```

```
    printf("Closest pair of points: (%.0f, %.0f) and (%.0f, %.0f)\n",
        best_p1.x, best_p1.y, best_p2.x, best_p2.y);
    printf("Distance: %.2f\n", min_dist);
```

```
    return 0;
```

}

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: SUSHANT KUMAR

Email: vtu29971@veltech.edu.in

Roll no: vtu29971

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L10

VTU_DAA_Week 3_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 33

Section 1 : Coding

1. Problem Statement

Adhi is learning about computational geometry and has come across the concept of a Convex Hull.

A Convex Hull is a convex polygon that encompasses a set of points in a two-dimensional plane, such that no point lies within the polygon's interior.

Adhi is particularly interested in determining whether a given point lies inside or outside the convex hull formed by a set of points.

Help Adhi by writing a program that checks whether a given point lies inside or outside the convex hull formed by a set of points.

Example

$N = 7$,

Points: $\{(1, 1), (2, 1), (3, 1), (4, 1), (4, 2), (4, 3), (4, 4)\}$,

Query: $X = 3, Y = 2$

Below is the image of the plotting of the given points:

The query $(3, 2)$ lies inside the Convex Hull.

Input Format

The first line of input consists of an integer N , representing the number of points in the set.

The following N lines, each containing two integers x and y , represent the coordinates of the points in the set.

The last line consists of two integers p_x and p_y , representing the coordinates of the test point.

Output Format

The output prints whether the given test points lie inside or outside the Convex Hull.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 7

1 1

2 1

3 1

4 1

4 2

4 3

4 4

3 2

Output: The point (3, 2) lies inside the Convex Hull.

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int x, y;
} Point;

// Helper to find the cross product (b-a) x (c-a)
int crossProduct(Point a, Point b, Point c) {
    return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
}

// Comparison function for sorting points
int compare(const void* a, const void* b) {
    Point* p1 = (Point*)a;
    Point* p2 = (Point*)b;
    if (p1->x != p2->x) return p1->x - p2->x;
    return p1->y - p2->y;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    Point pts[n];
    for (int i = 0; i < n; i++) scanf("%d %d", &pts[i].x, &pts[i].y);

    Point test;
    scanf("%d %d", &test.x, &test.y);

    // 1. Sort points to build Convex Hull
    qsort(pts, n, sizeof(Point), compare);

    // 2. Build Convex Hull (Monotone Chain)
    Point hull[2 * n];
    int h = 0;

    // Lower hull
```

```

for (int i = 0; i < n; i++) {
    while (h >= 2 && crossProduct(hull[h - 2], hull[h - 1], pts[i]) <= 0) h--;
    hull[h++] = pts[i];
}
// Upper hull
for (int i = n - 2, t = h + 1; i >= 0; i--) {
    while (h >= t && crossProduct(hull[h - 2], hull[h - 1], pts[i]) <= 0) h--;
    hull[h++] = pts[i];
}
h--; // Remove duplicate endpoint

// 3. Check if test point is inside the Hull
int inside = 1;
for (int i = 0; i < h; i++) {
    // If the point is to the right of any edge, it's outside
    if (crossProduct(hull[i], hull[(i + 1) % h], test) < 0) {
        inside = 0;
        break;
    }
}

if (inside)
    printf("The point (%d, %d) lies inside the Convex Hull.\n", test.x, test.y);
else
    printf("The point (%d, %d) lies outside the Convex Hull.\n", test.x, test.y);

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement:

Olivia, a city planner, is designing a recreational area surrounded by specific landmarks. To ensure balance in the design, Olivia needs to calculate the weighted centroid of the boundary formed by these landmarks using the convex hull.

Your task is to compute the convex hull and determine the weighted centroid of its vertices.

Note:

MAXN (100000): The maximum capacity for both the points and hull arrays, allowing storage of up to 100,000 points each. $1e-9$ is the threshold for floating-point comparisons.

Example:

Input:

6

0 0

4 0

4 3

0 3

2 1

3 2

Output:

Weighted Centroid: 2.00 1.50

Explanation:

Input Points: (0, 0), (2, 0), (2, 2), (0, 2), (1, 1), (3, 1).

Pivot Selection: The point with the lowest y-coordinate (and leftmost in case of a tie) is selected as the pivot: (0, 0).

Sorting by Polar Angle: Points are sorted by the polar angle relative to the pivot: (0, 0), (2, 0), (3, 1), (2, 2), (0, 2).

Convex Hull Construction: Using counterclockwise turns, the points forming the convex hull are: (0, 0), (2, 0), (3, 1), (2, 2), (0, 2).

Input Format

The first line contains an integer N, representing the number of points.

The next N lines each contain two floating-point numbers x and y, representing the coordinates of the points.

Output Format

The output prints the coordinates of the weighted centroid of the convex hull, formatted to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

0 0

2 0

2 2

0 2

1 1

Output: Weighted Centroid: 1.00 1.00

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <math.h>
```

```
typedef struct {  
    double x, y;  
} Point;
```

```
Point pivot;
```

```
// Euclidean distance squared
```

```
double distSq(Point p1, Point p2) {  
    return (p1.x - p2.x) * (p1.x - p2.x) + (p1.y - p2.y) * (p1.y - p2.y);  
}
```

```
// Orientation: 0 -> Collinear, 1 -> Clockwise, 2 -> Counterclockwise
```

```
int orientation(Point p, Point q, Point r) {  
    double val = (q.y - p.y) * (r.x - q.x) - (q.x - p.x) * (r.y - q.y);  
    if (fabs(val) < 1e-9) return 0;  
    return (val > 0) ? 1 : 2;  
}
```



```

// Compare function for qsort based on polar angle
int compare(const void *vp1, const void *vp2) {
    Point *p1 = (Point *)vp1;
    Point *p2 = (Point *)vp2;

    int o = orientation(pivot, *p1, *p2);
    if (o == 0)
        return (distSq(pivot, *p2) >= distSq(pivot, *p1)) ? -1 : 1;

    return (o == 2) ? -1 : 1;
}

```

```

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    Point *pts = malloc(n * sizeof(Point));
    int ymin = 0;

    for (int i = 0; i < n; i++) {
        scanf("%lf %lf", &pts[i].x, &pts[i].y);
        // Find the bottom-most point (pivot)
        if (i > 0) {
            if (pts[i].y < pts[ymin].y || (pts[i].y == pts[ymin].y && pts[i].x < pts[ymin].x))
                ymin = i;
        }
    }

    // Place pivot at first position and sort remaining points
    Point temp = pts[0];
    pts[0] = pts[ymin];
    pts[ymin] = temp;
    pivot = pts[0];

```

```

    qsort(&pts[1], n - 1, sizeof(Point), compare);

```

```

// Build the Hull

```

```

Point *hull = malloc(n * sizeof(Point));
int m = 0;
for (int i = 0; i < n; i++) {
    while (m >= 2 && orientation(hull[m - 2], hull[m - 1], pts[i]) != 2)

```

```

        m--;
        hull[m++] = pts[i];
    }

    // Calculate Weighted Centroid (Mean of unique Hull vertices)
    double sumX = 0, sumY = 0;
    for (int i = 0; i < m; i++) {
        sumX += hull[i].x;
        sumY += hull[i].y;
    }

    printf("Weighted Centroid: %.2f %.2f\n", sumX / m, sumY / m);

    free(pts);
    free(hull);
    return 0;
}

```

Status : Partially correct

Marks : 6/10

3. Problem Statement:

Gwen, a city planner, is designing a new park and needs to calculate the perimeter of the park's boundary. She has the coordinates of key boundary points and wants to find the minimum perimeter that encloses all these points.

Using the convex hull, Gwen needs help in computing both the boundary and its perimeter.

Note:

points[100000]: An array that can store up to 100,000 input points.
 hull[100000]: An array that can store up to 100,000 points on the convex hull.
 1e-9: A threshold for floating-point comparisons to handle precision issues.

Example

Input:

6

0.0 0.0

3.0 0.0

3.0 2.0

1.0 2.0

0.0 2.0

2.0 1.0

Output:

Convex Hull:

0 0

3 0

3 2

0 2

Perimeter: 10.00

Explanation:

Input Points: (0.0, 0.0), (3.0, 0.0), (3.0, 2.0), (1.0, 2.0), (0.0, 2.0), (2.0, 1.0).

Convex Hull Construction:

Pivot: Start with the lowest point, (0, 0). Sorted by Polar Angle: The points are sorted, and counterclockwise turns are included in the hull. Convex Hull

Points: (0, 0), (3, 0), (3, 2), (0, 2). Perimeter Calculation:

Distances between consecutive hull points: (0, 0)-(3, 0): 3.00, (3, 0)-(3, 2): 2.00, (3, 2)-(0, 2): 3.00, (0, 2)-(0, 0): 2.00. Total Perimeter: 10.00.

Input Format

The first line contains an integer N, representing the number of points.

The next N lines each contain two space separated decimal values x and y, representing the coordinates of the points.

Output Format

The first line prints "Convex Hull:" followed by the coordinates of the points on the convex hull, each on a new line, in counterclockwise order.

The last line prints the "Perimeter: ", followed by the total perimeter of the convex hull, formatted to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

0.0 0.0

3.0 0.0

3.0 2.0

1.0 2.0

0.0 2.0

2.0 1.0

Output: Convex Hull:

0 0

3 0

3 2

0 2

Perimeter: 10.00

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <math.h>
```

```
typedef struct {  
    double x, y;  
} Point;
```

```
Point pivot;
```

```
// Function to find the distance squared between two points
```

```
double distSq(Point p1, Point p2) {
```

```
    return (p1.x - p2.x) * (p1.x - p2.x) + (p1.y - p2.y) * (p1.y - p2.y);
```

```
}
```

// Orientation: 0 -> Collinear, 1 -> Clockwise, 2 -> Counterclockwise

```
int orientation(Point p, Point q, Point r) {  
    double val = (q.y - p.y) * (r.x - q.x) - (q.x - p.x) * (r.y - q.y);  
    if (fabs(val) < 1e-9) return 0;  
    return (val > 0) ? 1 : 2;  
}
```

// Comparison function for sorting points by polar angle

```
int compare(const void *vp1, const void *vp2) {  
    Point *p1 = (Point *)vp1;  
    Point *p2 = (Point *)vp2;  
  
    int o = orientation(pivot, *p1, *p2);  
    if (o == 0)  
        return (distSq(pivot, *p2) >= distSq(pivot, *p1)) ? -1 : 1;  
    return (o == 2) ? -1 : 1;  
}
```

```
int main() {  
    int n;  
    if (scanf("%d", &n) != 1) return 0;
```

```
    Point pts[n];  
    int ymin = 0;
```

```
    for (int i = 0; i < n; i++) {  
        scanf("%lf %lf", &pts[i].x, &pts[i].y);  
        // Find pivot: lowest y, then leftmost x  
        if (i > 0) {  
            if (pts[i].y < pts[ymin].y || (fabs(pts[i].y - pts[ymin].y) < 1e-9 && pts[i].x <  
pts[ymin].x))  
                ymin = i;  
        }  
    }  
}
```

// Swap pivot to first position

```
Point temp = pts[0];  
pts[0] = pts[ymin];  
pts[ymin] = temp;  
pivot = pts[0];
```

```

// Sort points 1 to n-1 by polar angle relative to pivot
qsort(&pts[1], n - 1, sizeof(Point), compare);

// Build the Convex Hull using a stack
Point hull[n];
int m = 0;
for (int i = 0; i < n; i++) {
    while (m >= 2 && orientation(hull[m - 2], hull[m - 1], pts[i]) != 2) {
        m--;
    }
    hull[m++] = pts[i];
}

// Print Hull Points
printf("Convex Hull:\n");
for (int i = 0; i < m; i++) {
    printf("%.0f %.0f\n", hull[i].x, hull[i].y);
}

// Calculate Perimeter
double perimeter = 0;
for (int i = 0; i < m; i++) {
    perimeter += sqrt(distSq(hull[i], hull[(i + 1) % m]));
}
printf("Perimeter: %.2f\n", perimeter);

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement:

Maria, a wildlife researcher, is studying the distribution of endangered species in a forest. She has mapped several points that represent observation locations inside the forest.

Maria wants to identify the K points that are the farthest from the forest's outer boundary, represented by a convex hull formed by a set of points. These points represent key observations, and she needs to rank them by

their distance to the outer boundary of the forest.

Help Maria compute the convex hull and find the K farthest points from the forest boundary.

Note:

MAXN (100000): Maximum number of points for points and hull arrays.
1e-9: Threshold for floating-point equality checks to handle precision errors.
1e-12: Threshold to detect degenerate (zero-length) segments.
1e30: Initial large value for minimum distance calculations.

Example:

Input:

7 3

0 0

4 0

4 4

0 4

2 2

1 2

3 2

Output:

2 2

3 2

1 2

Explanation:

Convex Hull Points: Using Graham's Scan, the convex hull is: (0, 0), (4, 0), (4, 4), (0, 4).

Distance to Hull: Distances of all points to the convex hull boundary are computed.

Top 3 Farthest Points: Sorted by descending distance, the top 3 points are (2, 2), (3, 2), (1, 2).

Input Format

The first line contains two integers N and K, representing the number of points and the number of farthest points to find, respectively.

The next N lines each contain two integers x and y, representing the coordinates of the points.

Output Format

The output prints K lines, each containing two integers x and y, representing the coordinates of the K farthest points from the convex hull boundary. Print the points in descending order of their distances to the boundary.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7 3

0 0

4 0

4 4

0 4

2 2

1 2

3 2

Output: 2 2

3 2

1 2

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <math.h>
```

```
typedef struct {  
    double x, y;  
    double distToHull;
```



```
int id;  
} Point;
```

```
Point pivot;
```

```
double distSq(Point p1, Point p2) {  
    return (p1.x - p2.x) * (p1.x - p2.x) + (p1.y - p2.y) * (p1.y - p2.y);  
}
```

```
int orientation(Point p, Point q, Point r) {  
    double val = (q.y - p.y) * (r.x - q.x) - (q.x - p.x) * (r.y - q.y);  
    if (fabs(val) < 1e-9) return 0;  
    return (val > 0) ? 1 : 2;  
}
```

```
int comparePoints(const void *vp1, const void *vp2) {  
    Point *p1 = (Point *)vp1;  
    Point *p2 = (Point *)vp2;  
    int o = orientation(pivot, *p1, *p2);  
    if (o == 0) return (distSq(pivot, *p2) >= distSq(pivot, *p1)) ? -1 : 1;  
    return (o == 2) ? -1 : 1;  
}
```

```
// Final ranking: Distance DESC, then Original Index DESC for tie-breaking
```

```
int compareFinal(const void *a, const void *b) {  
    Point *p1 = (Point *)a;  
    Point *p2 = (Point *)b;  
    if (fabs(p1->distToHull - p2->distToHull) > 1e-7) {  
        return (p2->distToHull > p1->distToHull) ? 1 : -1;  
    }  
    return p2->id - p1->id;  
}
```

```
double distToSegment(Point p, Point a, Point b) {  
    double l2 = distSq(a, b);  
    if (l2 < 1e-12) return sqrt(distSq(p, a));  
    double t = ((p.x - a.x) * (b.x - a.x) + (p.y - a.y) * (b.y - a.y)) / l2;  
    if (t < 0) return sqrt(distSq(p, a));  
    if (t > 1) return sqrt(distSq(p, b));  
    Point projection = {a.x + t * (b.x - a.x), a.y + t * (b.y - a.y)};  
    return sqrt(distSq(p, projection));  
}
```

```

int main() {
    int n, k;
    if (scanf("%d %d", &n, &k) != 2) return 0;

    Point pts[n], original[n];
    int ymin = 0;
    for (int i = 0; i < n; i++) {
        scanf("%lf %lf", &pts[i].x, &pts[i].y);
        pts[i].id = i;
        original[i] = pts[i];
        if (i > 0 && (pts[i].y < pts[ymin].y || (pts[i].y == pts[ymin].y && pts[i].x <
pts[ymin].x)))
            ymin = i;
    }

    Point temp = pts[0]; pts[0] = pts[ymin]; pts[ymin] = temp;
    pivot = pts[0];
    qsort(&pts[1], n - 1, sizeof(Point), comparePoints);

    Point hull[n];
    int hSize = 0;
    for (int i = 0; i < n; i++) {
        while (hSize >= 2 && orientation(hull[hSize - 2], hull[hSize - 1], pts[i]) != 2)
            hSize--;
        hull[hSize++] = pts[i];
    }

    for (int i = 0; i < n; i++) {
        double minDist = 1e30;
        for (int j = 0; j < hSize; j++) {
            double d = distToSegment(original[i], hull[j], hull[(j + 1) % hSize]);
            if (d < minDist) minDist = d;
        }
        original[i].distToHull = minDist;
    }

    qsort(original, n, sizeof(Point), compareFinal);

    for (int i = 0; i < k; i++) {
        printf("%.0f %.0f\n", original[i].x, original[i].y);
    }
}

```

```
return 0;
```

Status : Partially correct

Marks : 3.5/10

5. Problem Statement:

Henry, a wildlife researcher, is studying the distribution of endangered species in a forest. He has mapped several points that represent observation locations inside the forest.

Henry wants to identify the K points that are the farthest from the forest's outer boundary, represented by a convex hull formed by a set of points. These points represent key observations, and he needs to rank them by their distance to the outer boundary of the forest.

Help Henry compute the convex hull and find the K farthest points from the forest boundary.

Note:

MAXN (100000): Maximum number of points for points and hull arrays. $1e-9$: Threshold for floating-point equality checks to handle precision errors. $1e-12$: Threshold to detect degenerate (zero-length) segments. $1e30$: Initial large value for minimum distance calculations.

Example:

Input:

7 3

0 0

4 0

4 4

0 4

2 2

1 2
3 2

Output:

2 2
3 2
1 2

Explanation:

Convex Hull Points: Using Graham's Scan, the convex hull is: (0, 0), (4, 0), (4, 4), (0, 4).

Distance to Hull: Distances of all points to the convex hull boundary are computed.

Top 3 Farthest Points: Sorted by descending distance, the top 3 points are (2, 2), (3, 2), (1, 2).

Input Format

The first line contains two integers N and K, representing the number of points and the number of farthest points to find, respectively.

The next N lines each contain two integers x and y, representing the coordinates of the points.

Output Format

The output prints K lines, each containing two integers x and y, representing the coordinates of the K farthest points from the convex hull boundary. Print the points in descending order of their distances to the boundary.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7 3
0 0

4 0
4 4
0 4
2 2
1 2
3 2

Output: 2 2
3 2
1 2

Answer

// You are using GCC

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <math.h>
```

```
typedef struct {  
    double x, y;  
    double dist;  
    int id;  
} Point;
```

```
Point pivot;
```

```
double distSq(Point p1, Point p2) {  
    return (p1.x - p2.x) * (p1.x - p2.x) + (p1.y - p2.y) * (p1.y - p2.y);  
}
```

```
int orientation(Point p, Point q, Point r) {  
    double val = (q.y - p.y) * (r.x - q.x) - (q.x - p.x) * (r.y - q.y);  
    if (fabs(val) < 1e-9) return 0;  
    return (val > 0) ? 1 : 2;  
}
```

```
int comparePoints(const void *vp1, const void *vp2) {  
    Point *p1 = (Point *)vp1;  
    Point *p2 = (Point *)vp2;  
    int o = orientation(pivot, *p1, *p2);  
    if (o == 0) return (distSq(pivot, *p2) >= distSq(pivot, *p1)) ? -1 : 1;  
    return (o == 2) ? -1 : 1;  
}
```

```

int compareFinal(const void *a, const void *b) {
    Point *p1 = (Point *)a;
    Point *p2 = (Point *)b;
    if (fabs(p1->dist - p2->dist) > 1e-8) {
        return (p2->dist > p1->dist) ? 1 : -1;
    }
    return p2->id - p1->id; // Tie-breaker: Higher index first
}

```

```

double distToSegment(Point p, Point a, Point b) {
    double l2 = distSq(a, b);
    if (l2 < 1e-12) return sqrt(distSq(p, a));
    double t = ((p.x - a.x) * (b.x - a.x) + (p.y - a.y) * (b.y - a.y)) / l2;
    if (t < 0) return sqrt(distSq(p, a));
    if (t > 1) return sqrt(distSq(p, b));
    Point proj = {a.x + t * (b.x - a.x), a.y + t * (b.y - a.y)};
    return sqrt(distSq(p, proj));
}

```

```

int main() {
    int n, k;
    if (scanf("%d %d", &n, &k) != 2) return 0;

    Point pts[n], original[n];
    int ymin = 0;
    for (int i = 0; i < n; i++) {
        scanf("%lf %lf", &pts[i].x, &pts[i].y);
        pts[i].id = i;
        original[i] = pts[i];
        if (i > 0 && (pts[i].y < pts[ymin].y || (pts[i].y == pts[ymin].y && pts[i].x <
pts[ymin].x)))
            ymin = i;
    }

```

```

    Point temp = pts[0]; pts[0] = pts[ymin]; pts[ymin] = temp;
    pivot = pts[0];
    qsort(&pts[1], n - 1, sizeof(Point), comparePoints);

```

```

    Point hull[n];
    int hSize = 0;
    for (int i = 0; i < n; i++) {
        while (hSize >= 2 && orientation(hull[hSize - 2], hull[hSize - 1], pts[i]) != 2)
            hSize--;
    }

```

```

    hull[hSize++] = pts[i];
}

for (int i = 0; i < n; i++) {
    double minDist = 1e30;
    for (int j = 0; j < hSize; j++) {
        double d = distToSegment(original[i], hull[j], hull[(j + 1) % hSize]);
        if (d < minDist) minDist = d;
    }
    original[i].dist = minDist;
}

qsort(original, n, sizeof(Point), compareFinal);

for (int i = 0; i < k; i++) {
    printf("%.0f %.0f\n", original[i].x, original[i].y);
}

return 0;
}

```

Status : Partially correct

Marks : 3.5/10

Vel Tech Group of Institutions

Name: SUSHANT KUMAR
Email: vtu29971@veltech.edu.in
Roll no: vtu29971
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L10

VTU_DAA_Week 3_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 48.5

Section 1 : Coding

1. Problem Statement:

Javier, a landscape architect, is designing a new botanical garden and needs to determine the outer boundary that will enclose all the selected plant species.

He has gathered coordinates of key points where the plants are located and needs to calculate the convex hull to create a protective garden boundary. To determine this boundary, Javier decides to use Graham's Scan algorithm.

Help Javier compute the convex hull by implementing the algorithm to design the garden's boundary.

Input Format

The first line of input consists of an integer n , representing the number of plant species locations.

The next n lines contain two integers x and y , representing the coordinates of each location.

Output Format

The output prints "Convex Hull:" followed by the coordinates of the points forming the convex hull in counterclockwise order, with each point on a new line.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

0 0

0 1

1 0

1 1

Output: Convex Hull:

(0, 0)

(1, 0)

(1, 1)

(0, 1)

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <math.h>
```

```
typedef struct {
```

```
    int x, y;
```

```
} Point;
```

```
Point pivot;
```

```
// Distance squared between two points
```

```
int distSq(Point p1, Point p2) {
```

```
    return (p1.x - p2.x) * (p1.x - p2.x) + (p1.y - p2.y) * (p1.y - p2.y);
```

```
}
```

```
// Orientation: 0 -> Collinear, 1 -> Clockwise, 2 -> Counterclockwise
```

```
int orientation(Point p, Point q, Point r) {  
    int val = (q.y - p.y) * (r.x - q.x) - (q.x - p.x) * (r.y - q.y);  
    if (val == 0) return 0;  
    return (val > 0) ? 1 : 2;  
}
```

```
// Comparison function for sorting points by polar angle relative to pivot
```

```
int compare(const void *vp1, const void *vp2) {  
    Point *p1 = (Point *)vp1;  
    Point *p2 = (Point *)vp2;  
  
    int o = orientation(pivot, *p1, *p2);  
    if (o == 0)  
        return (distSq(pivot, *p2) >= distSq(pivot, *p1)) ? -1 : 1;  
  
    return (o == 2) ? -1 : 1;  
}
```

```
int main() {  
    int n;  
    if (scanf("%d", &n) != 1) return 0;  
    if (n < 1) return 0;
```

```
    Point pts[n];  
    int ymin = 0;
```

```
    for (int i = 0; i < n; i++) {  
        scanf("%d %d", &pts[i].x, &pts[i].y);  
        // Find the pivot: lowest y, then leftmost x  
        if (i > 0) {  
            if (pts[i].y < pts[ymin].y || (pts[i].y == pts[ymin].y && pts[i].x < pts[ymin].x))  
                ymin = i;  
        }  
    }  
}
```

```
// Place pivot at first position
```

```
Point temp = pts[0];  
pts[0] = pts[ymin];  
pts[ymin] = temp;
```

```

pivot = pts[0];

// Sort remaining points by polar angle
if (n > 1) {
    qsort(&pts[1], n - 1, sizeof(Point), compare);
}

// Handle small n or collinear points
if (n < 3) {
    printf("Convex Hull:\n");
    for (int i = 0; i < n; i++) {
        printf("(%d, %d)\n", pts[i].x, pts[i].y);
    }
    return 0;
}

// Build the Hull using a stack
Point hull[n];
int m = 0;
for (int i = 0; i < n; i++) {
    while (m >= 2 && orientation(hull[m - 2], hull[m - 1], pts[i]) != 2) {
        m--;
    }
    hull[m++] = pts[i];
}

// Output the results
printf("Convex Hull:\n");
for (int i = 0; i < m; i++) {
    printf("(%d, %d)\n", hull[i].x, hull[i].y);
}

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement:

Eva, a botanist, is surveying a nature reserve to protect rare plant species. During her survey, she marked a special tree, known as the "guardian tree,"

and now needs to map out the perimeter of the reserve, ensuring the guardian tree is included in the boundary. To do this, Eva needs to compute the convex hull of all the points, including the coordinates of the guardian tree.

Your task is to compute the convex hull, ensuring that the special guardian tree is included in the boundary.

Example

Input:

5

0.0 0.0

2.0 0.0

2.0 2.0

0.0 2.0

1.0 1.0

3.0 1.0

Output:

0 0

2 0

3 1

2 2

0 2

Explanation:

Input Points: (0.0, 0.0), (2.0, 0.0), (2.0, 2.0), (0.0, 2.0), (1.0, 1.0), (3.0, 1.0).

Pivot Selection: The point with the lowest y-coordinate (and leftmost in case of a tie) is selected as the pivot: (0, 0).

Sorting by Polar Angle: Points are sorted by the polar angle relative to the pivot: (0, 0), (2, 0), (3, 1), (2, 2), (0, 2).

Convex Hull Construction: Using counterclockwise turns, the points forming the convex hull are: (0, 0), (2, 0), (3, 1), (2, 2), (0, 2).

Input Format

The first line contains an integer N, representing the number of trees originally surveyed.

The next N lines each contain two double values x and y, representing the coordinates of the surveyed trees.

The last line contains two double values x and y, representing the coordinates of the guardian tree.

Output Format

The output prints the coordinates of the points forming the convex hull in counterclockwise order, ensuring the special guardian tree is included. Each coordinate should be printed on a new line, with the values of x and y separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

0.0 0.0

2.0 0.0

2.0 2.0

0.0 2.0

1.0 1.0

3.0 1.0

Output: 0 0

2 0

3 1

2 2

0 2

Answer

```
// You are using GCC
#include <stdio.h>
```

```
#include <stdlib.h>
#include <math.h>
```

```
typedef struct {
    double x, y;
} Point;
```

```
Point pivot;
```

```
// Function to calculate square of Euclidean distance
double distSq(Point p1, Point p2) {
    return (p1.x - p2.x) * (p1.x - p2.x) + (p1.y - p2.y) * (p1.y - p2.y);
}
```

```
// Orientation: 0 -> Collinear, 1 -> Clockwise, 2 -> Counterclockwise
int orientation(Point p, Point q, Point r) {
    double val = (q.y - p.y) * (r.x - q.x) - (q.x - p.x) * (r.y - q.y);
    if (fabs(val) < 1e-9) return 0;
    return (val > 0) ? 1 : 2;
}
```

```
// Comparison function for qsort based on polar angle relative to pivot
int compare(const void *vp1, const void *vp2) {
    Point *p1 = (Point *)vp1;
    Point *p2 = (Point *)vp2;

    int o = orientation(pivot, *p1, *p2);
    if (o == 0)
        return (distSq(pivot, *p2) >= distSq(pivot, *p1)) ? -1 : 1;

    return (o == 2) ? -1 : 1;
}
```

```
int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
```

```
    // Total points = N surveyed trees + 1 guardian tree
    int totalPoints = n + 1;
    Point pts[totalPoints];
    int ymin = 0;
```

```

for (int i = 0; i < totalPoints; i++) {
    scanf("%lf %lf", &pts[i].x, &pts[i].y);
    // Find pivot: lowest y, then leftmost x
    if (i > 0) {
        if (pts[i].y < pts[ymin].y || (fabs(pts[i].y - pts[ymin].y) < 1e-9 && pts[i].x <
pts[ymin].x))
            ymin = i;
    }
}

// Move pivot to first position
Point temp = pts[0];
pts[0] = pts[ymin];
pts[ymin] = temp;
pivot = pts[0];

// Sort remaining points by polar angle relative to pivot
qsort(&pts[1], totalPoints - 1, sizeof(Point), compare);

// Build the Convex Hull using a stack
Point hull[totalPoints];
int hSize = 0;
for (int i = 0; i < totalPoints; i++) {
    // While the turn is not counter-clockwise, pop from stack
    while (hSize >= 2 && orientation(hull[hSize - 2], hull[hSize - 1], pts[i]) != 2) {
        hSize--;
    }
    hull[hSize++] = pts[i];
}

// Output formatting: printing coordinates as integers to match sample
for (int i = 0; i < hSize; i++) {
    printf("%.0f %.0f\n", hull[i].x, hull[i].y);
}

return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Nathan is learning about computational geometry and has come across the concept of a Convex Hull.

A Convex Hull is a convex polygon that encompasses a set of points in a two-dimensional plane, such that no point lies within the polygon's interior.

Nathan is particularly interested in determining whether a given point lies inside or outside the convex hull formed by a set of points.

Help Nathan by writing a program that checks whether a given point lies inside or outside the convex hull formed by a set of points.

Example

$N = 7$,

Points: $\{(1, 1), (2, 1), (3, 1), (4, 1), (4, 2), (4, 3), (4, 4)\}$,

Query: $X = 3, Y = 2$

Below is the image of the plotting of the given points:

The query $(3, 2)$ lies inside the Convex Hull.

Input Format

The first line of input consists of an integer N , representing the number of points in the set.

The following N lines, each containing two integers x and y , represent the coordinates of the points in the set.

The last line consists of two integers p_x and p_y , representing the coordinates of the test point.

Output Format

The output prints whether the given test points lie inside or outside the Convex Hull.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 7

1 1

2 1

3 1

4 1

4 2

4 3

4 4

3 2

Output: The point (3, 2) lies inside the Convex Hull.

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct {  
    int x, y;  
} Point;
```

```
// Helper to find the cross product (b-a) x (c-a)
```

```
int crossProduct(Point a, Point b, Point c) {  
    return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);  
}
```

```
// Function to compare points for sorting
```

```
int compare(const void* a, const void* b) {  
    Point* p1 = (Point*)a;  
    Point* p2 = (Point*)b;  
    if (p1->x != p2->x) return p1->x - p2->x;  
    return p1->y - p2->y;  
}
```

```
int main() {  
    int n;  
    if (scanf("%d", &n) != 1) return 0;
```

```
Point pts[n];
for (int i = 0; i < n; i++) {
    scanf("%d %d", &pts[i].x, &pts[i].y);
}
```

```
Point query;
scanf("%d %d", &query.x, &query.y);
```

```
// 1. Sort points to build Convex Hull
qsort(pts, n, sizeof(Point), compare);
```

```
// 2. Build Convex Hull (Monotone Chain)
```

```
Point hull[2 * n];
```

```
int h = 0;
```

```
// Build lower hull
```

```
for (int i = 0; i < n; i++) {
    while (h >= 2 && crossProduct(hull[h - 2], hull[h - 1], pts[i]) <= 0) h--;
    hull[h++] = pts[i];
}
```

```
// Build upper hull
```

```
for (int i = n - 2, t = h + 1; i >= 0; i--) {
    while (h >= t && crossProduct(hull[h - 2], hull[h - 1], pts[i]) <= 0) h--;
    hull[h++] = pts[i];
}
```

```
h--; // Remove the duplicate endpoint
```

```
// 3. Check if query point is inside the hull
```

```
// A point is inside if it is to the left or on all CCW edges
```

```
int inside = 1;
```

```
for (int i = 0; i < h; i++) {
    if (crossProduct(hull[i], hull[(i + 1) % h], query) < 0) {
        inside = 0;
        break;
    }
}
```

```
if (inside) {
```

```
    printf("The point (%d, %d) lies inside the Convex Hull.\n", query.x, query.y);
```

```
} else {
```

```
    printf("The point (%d, %d) lies outside the Convex Hull.\n", query.x, query.y);
```

```
}
```

```
} return 0;
```

Status : Correct

Marks : 10/10

4. Problem Statement

Ragu, an aspiring computer scientist, is currently learning about the concept of finding the closest pair of points in a plane. He wants to test his skills by solving a programming challenge related to this concept.

Given a set of points in a 2D plane, Ragu needs to find the closest pair of points and calculate their Euclidean distance.

Input Format

The first line of input consists of an integer N , representing the number of points in the plane.

The following N lines, each consisting of two space-separated real numbers x and y , represent the coordinates of a point.

Output Format

The first line of output prints the coordinates of the two points forming the closest pair in the below format:

"Closest pair of points: (x_1, y_1) and (x_2, y_2) "

The second line prints "Distance: <distance>" representing the Euclidean distance between the closest pair of points, rounded off to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

0 3

1 1

10.18

4.9

Output: Closest pair of points: (0, 3) and (1, 1)

Distance: 2.24

Answer

// You are using GCC

#include <stdio.h>

#include <math.h>

#include <float.h>

```
typedef struct {  
    double x, y;  
} Point;
```

// Function to calculate Euclidean distance

```
double getDistance(Point p1, Point p2) {  
    return sqrt(pow(p1.x - p2.x, 2) + pow(p1.y - p2.y, 2));  
}
```

```
int main() {  
    int n;  
    if (scanf("%d", &n) != 1) return 0;
```

```
    Point pts[n];  
    for (int i = 0; i < n; i++) {  
        scanf("%lf %lf", &pts[i].x, &pts[i].y);  
    }
```

```
    double min_dist = DBL_MAX;  
    Point best_p1, best_p2;
```

```
    // Compare every unique pair (i, j)  
    for (int i = 0; i < n; i++) {  
        for (int j = i + 1; j < n; j++) {  
            double d = getDistance(pts[i], pts[j]);  
            if (d < min_dist) {  
                min_dist = d;  
                best_p1 = pts[i];  
                best_p2 = pts[j];  
            }  
        }  
    }
```

```

    }
    // Output formatting matches sample requirements
    printf("Closest pair of points: (%.0f, %.0f) and (%.0f, %.0f)\n",
           best_p1.x, best_p1.y, best_p2.x, best_p2.y);
    printf("Distance: %.2f\n", min_dist);

    return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement:

Lena, a geologist, is studying the boundaries of a newly discovered forest reserve. To define the outer edges of the reserve, Lena needs to identify the convex hull from a set of points marking different trees within the area.

Using Graham's Scan algorithm, help Lena compute the convex hull to outline the forest reserve's perimeter.

Example1

Input:

```

6
0 0
1 1
2 2
4 4
0 4
4 0

```

Output:

Convex Hull:

```

(0, 0)

```

(4, 0)

(4, 4)

(0, 4)

Explanation:

Pivot Selection: The bottom-most point is (0, 0).

Sorting by Polar Angle: Points sorted as (0, 0), (4, 0), (4, 4), (0, 4), (1, 1), (2, 2).

Convex Hull Construction: Start with (0, 0), and include points forming counterclockwise turns.

The final hull is: (0, 0), (4, 0), (4, 4), (0, 4).

Example2

Input:

3

0 0

0 3

3 0

Output:

Convex Hull:

(0, 0)

(3, 0)

(0, 3)

Explanation:

Pivot Selection: The bottom-most point is (0, 0).

Sorting by Polar Angle: Points sorted as (0, 0), (3, 0), (0, 3).

Convex Hull Construction: Start with (0, 0), and include points forming counterclockwise turns.

The final hull is: (0, 0), (3, 0), (0, 3).

Input Format

The first line of input contains an integer n , representing the number of trees in the forest reserve.

The next n lines contain two integers x and y , representing the coordinates of each tree.

Output Format

The output prints "Convex Hull:" followed by the coordinates of the trees forming the convex hull in counterclockwise order in new line.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

0 0

1 1

2 2

4 4

0 4

4 0

Output: Convex Hull:

(0, 0)

(4, 0)

(4, 4)

(0, 4)

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <math.h>
```

```
typedef struct {
```

```
    int x, y;
```

```
} Point;
```

Point pivot;

// Helper to find square of distance between two points

```
int distSq(Point p1, Point p2) {  
    return (p1.x - p2.x) * (p1.x - p2.x) + (p1.y - p2.y) * (p1.y - p2.y);  
}
```

// Orientation: 0 -> Collinear, 1 -> Clockwise, 2 -> Counterclockwise

```
int orientation(Point p, Point q, Point r) {  
    int val = (q.y - p.y) * (r.x - q.x) - (q.x - p.x) * (r.y - q.y);  
    if (val == 0) return 0;  
    return (val > 0) ? 1 : 2;  
}
```

// Comparison function for qsort based on polar angle relative to pivot

```
int compare(const void *vp1, const void *vp2) {  
    Point *p1 = (Point *)vp1;  
    Point *p2 = (Point *)vp2;  
  
    int o = orientation(pivot, *p1, *p2);  
    if (o == 0)  
        return (distSq(pivot, *p2) >= distSq(pivot, *p1)) ? -1 : 1;  
  
    return (o == 2) ? -1 : 1;  
}
```

```
int main() {  
    int n;  
    if (scanf("%d", &n) != 1 || n < 1) return 0;
```

```
    Point pts[n];  
    int ymin = 0;
```

```
    for (int i = 0; i < n; i++) {  
        scanf("%d %d", &pts[i].x, &pts[i].y);  
        // Pivot selection: lowest y, then leftmost x  
        if (i > 0) {  
            if (pts[i].y < pts[ymin].y || (pts[i].y == pts[ymin].y && pts[i].x < pts[ymin].x))  
                ymin = i;  
        }  
    }  
}
```



```

// Move pivot to first position
Point temp = pts[0];
pts[0] = pts[ymin];
pts[ymin] = temp;
pivot = pts[0];

// Sort points based on polar angle
if (n > 1) {
    qsort(&pts[1], n - 1, sizeof(Point), compare);
}

// Build the Hull using a stack-like array
Point hull[n];
int m = 0;
for (int i = 0; i < n; i++) {
    // While the last two points and current point don't make a CCW turn, pop
    while (m >= 2 && orientation(hull[m - 2], hull[m - 1], pts[i]) != 2) {
        m--;
    }
    hull[m++] = pts[i];
}

// Output formatting
printf("Convex Hull:\n");
for (int i = 0; i < m; i++) {
    printf("(%d, %d)\n", hull[i].x, hull[i].y);
}

return 0;
}

```

Status : Partially correct

Marks : 8.5/10

Vel Tech Group of Institutions

Name: SUSHANT KUMAR
Email: vtu29971@veltech.edu.in
Roll no: vtu29971
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L10

VTU_DAA_Week 4_COD

Attempt : 1
Total Mark : 40
Marks Obtained : 24

Section 1 : Coding

1. Problem Statement

You are a delivery manager who needs to plan the delivery route for a fleet of trucks to a set of distribution centers in a city. Each distribution center is at a different location, and the cost of traveling between centers is known.

Your goal is to find the shortest route that allows a truck to visit each distribution center exactly once and return to the starting point, using the Traveling Salesman Problem (TSP). Write a program to calculate the minimum cost of this route and output the optimal path.

Note: Solve using Branch and Bound approach.

Input Format

The first line contains an integer n, the number of distribution centers.

The next n lines contain n space-separated integers each, representing the adjacency matrix.

The ith integer on the jth line represents the cost of traveling from distribution center i to distribution center j.

Output Format

The first line prints:

Minimum Cost is: X

The second line prints:

Optimal Path: followed by the sequence of visited centers ending at the start node

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 3 9 5

3 0 6 4

9 6 0 7

5 4 7 0

Output: Minimum Cost is: 21

Optimal Path: 0 1 2 3 0

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <stdbool.h>
```

```
#include <limits.h>
```

```
#define MAX 10
```

```
int n;  
int adj[MAX][MAX];  
int final_path[MAX + 1];  
bool visited[MAX];  
int final_res = INT_MAX;
```

```
// Function to copy temporary solution to final solution
```

```
void copyToFinal(int curr_path[]) {  
    for (int i = 0; i < n; i++)  
        final_path[i] = curr_path[i];  
    final_path[n] = curr_path[0];  
}
```

```
// Function to find the minimum edge cost having an end at the vertex i
```

```
int firstMin(int i) {  
    int min = INT_MAX;  
    for (int k = 0; k < n; k++)  
        if (adj[i][k] < min && i != k)  
            min = adj[i][k];  
    return min;  
}
```

```
// Function to find the second minimum edge cost having an end at the vertex i
```

```
int secondMin(int i) {  
    int first = INT_MAX, second = INT_MAX;  
    for (int j = 0; j < n; j++) {  
        if (i == j) continue;  
        if (adj[i][j] <= first) {  
            second = first;  
            first = adj[i][j];  
        } else if (adj[i][j] <= second && adj[i][j] != first) {  
            second = adj[i][j];  
        }  
    }  
    return second;  
}
```

```
// Function that solves TSP using Branch and Bound
```

```

void TSPRec(int curr_bound, int curr_weight, int level, int curr_path[]) {
    if (level == n) {
        if (adj[curr_path[level - 1]][curr_path[0]] != 0) {
            int curr_res = curr_weight + adj[curr_path[level - 1]][curr_path[0]];
            if (curr_res < final_res) {
                copyToFinal(curr_path);
                final_res = curr_res;
            }
        }
    }
    return;
}

for (int i = 0; i < n; i++) {
    if (adj[curr_path[level - 1]][i] != 0 && visited[i] == false) {
        int temp = curr_bound;
        curr_weight += adj[curr_path[level - 1]][i];

        if (level == 1)
            curr_bound -= ((firstMin(curr_path[level - 1]) + firstMin(i)) / 2);
        else
            curr_bound -= ((secondMin(curr_path[level - 1]) + firstMin(i)) / 2);

        if (curr_bound + curr_weight < final_res) {
            curr_path[level] = i;
            visited[i] = true;
            TSPRec(curr_bound, curr_weight, level + 1, curr_path);
        }

        curr_weight -= adj[curr_path[level - 1]][i];
        curr_bound = temp;
        for (int k = 0; k < n; k++) visited[k] = false;
        for (int k = 0; k < level; k++) visited[curr_path[k]] = true;
    }
}
}

```

```

void solveTSP() {
    int curr_path[MAX + 1];
    int curr_bound = 0;
    for (int i = 0; i < n; i++) visited[i] = false;

    for (int i = 0; i < n; i++)

```

```

curr_bound += (firstMin(i) + secondMin(i));

curr_bound = (curr_bound & 1) ? curr_bound / 2 + 1 : curr_bound / 2;

visited[0] = true;
curr_path[0] = 0;
TSPRec(curr_bound, 0, 1, curr_path);
}

int main() {
    if (scanf("%d", &n) != 1) return 0;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &adj[i][j]);

    solveTSP();

    printf("Minimum Cost is: %d\n", final_res);
    printf("Optimal Path: ");
    for (int i = 0; i <= n; i++)
        printf("%d%c", final_path[i], (i == n) ? '\n' : ' ');

    return 0;
}

```

Status : Partially correct

Marks : 2/10

2. Problem Statement

You are an art collector who wants to visit a set of art galleries in a city. Each gallery is at a different location, and the cost of traveling between galleries is known.

Your goal is to find the shortest route that allows you to visit each gallery exactly once and return to your starting point, using the traveling salesman problem (TSP). Write a program to calculate the minimum cost of this route.

Note: Solve it using Branch and Bound technique.

Input Format

The first line contains an integer n, the number of galleries.

The next n lines contain n integers each, representing the adjacency matrix.

The ith integer on the jth line represents the cost of traveling from gallery i to gallery j.

Output Format

The output displays "Minimum Cost is: " followed by the minimum cost of the route that visits each gallery exactly once and returns to the starting gallery.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Output: Minimum Cost is: 80

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
#include <stdbool.h>
```

```
#define MAX 10
```

```
int n;
```

```
int adj[MAX][MAX];
```

```
int final_res = INT_MAX;
```

```
bool visited[MAX];
```

```
// Function to find the minimum edge cost connected to vertex i
```

```
int firstMin(int i) {
```

```
    int min = INT_MAX;
```

```

    for (int k = 0; k < n; k++)
        if (adj[i][k] < min && i != k)
            min = adj[i][k];
    return min;
}

```

```

// Function to find the second minimum edge cost connected to vertex i
int secondMin(int i) {
    int first = INT_MAX, second = INT_MAX;
    for (int j = 0; j < n; j++) {
        if (i == j) continue;
        if (adj[i][j] <= first) {
            second = first;
            first = adj[i][j];
        } else if (adj[i][j] <= second && adj[i][j] != first) {
            second = adj[i][j];
        }
    }
    return second;
}

```

```

// Recursive function to solve TSP using Branch and Bound
void TSPRec(int curr_bound, int curr_weight, int level, int curr_path[]) {
    // Base case: all nodes visited
    if (level == n) {
        // Check if there is an edge back to the starting node
        if (adj[curr_path[level - 1]][curr_path[0]] != 0) {
            int curr_res = curr_weight + adj[curr_path[level - 1]][curr_path[0]];
            if (curr_res < final_res) {
                final_res = curr_res;
            }
        }
    }
    return;
}

```

```

for (int i = 0; i < n; i++) {
    // Consider next vertex if it is not visited
    if (adj[curr_path[level - 1]][i] != 0 && !visited[i]) {
        int temp_bound = curr_bound;
        curr_weight += adj[curr_path[level - 1]][i];

        // Computation of bound for the next node

```



```

    if (level == 1)
        curr_bound -= ((firstMin(curr_path[level - 1]) + firstMin(i)) / 2);
    else
        curr_bound -= ((secondMin(curr_path[level - 1]) + firstMin(i)) / 2);

    // If current lower bound < current minimum, explore this path
    if (curr_bound + curr_weight < final_res) {
        curr_path[level] = i;
        visited[i] = true;
        TSPRec(curr_bound, curr_weight, level + 1, curr_path);
    }

    // Backtrack: reset weight, bound and visited status
    curr_weight -= adj[curr_path[level - 1]][i];
    curr_bound = temp_bound;
    visited[i] = false;
}
}
}

```

```

int main() {
    if (scanf("%d", &n) != 1) return 0;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &adj[i][j]);
        }
    }

    int curr_path[MAX + 1];
    int curr_bound = 0;

    // Calculate initial lower bound
    for (int i = 0; i < n; i++)
        curr_bound += (firstMin(i) + secondMin(i));

    // Rounding off the bound
    curr_bound = (curr_bound % 2 == 1) ? curr_bound / 2 + 1 : curr_bound / 2;

    // Start from node 0
    visited[0] = true;
    curr_path[0] = 0;
}

```

```
TSPRec(curr_bound, 0, 1, curr_path);  
  
printf("Minimum Cost is: %d\n", final_res);  
  
return 0;  
}
```

Status : Partially correct

Marks : 2/10

3. Problem Statement

Sarah, a young entrepreneur starting her e-commerce business, has a limited budget for storing products in her warehouse. Each product has a specific weight and value, and Sarah aims to maximize the total value of the products in her warehouse without exceeding the weight capacity.

To achieve this, Sarah decided to apply the dynamic programming technique, which helps explore potential product combinations and avoid unnecessary calculations.

Input Format

The first line contains a single integer N , representing the number of products.

The next N lines each contain two integers: $\text{weight}[i]$ and $\text{value}[i]$, where $\text{weight}[i]$ is the weight of the product and $\text{value}[i]$ is the value (profit) of the product.

The next line contains a single integer W , representing the weight capacity of the warehouse.

Output Format

The output prints a single integer, representing the maximum total profit.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 5

2 40

3 50

1 100

5 95

3 30

10

Output: 245

Answer

// You are using GCC

#include <stdio.h>

// Function to find the maximum of two integers

```
int max(int a, int b) {  
    return (a > b) ? a : b;  
}
```

```
int main() {  
    int N, W;
```

// Read the number of products

if (scanf("%d", &N) != 1) return 0;

int weights[N];

int values[N];

// Read weight and value for each product

```
for (int i = 0; i < N; i++) {  
    scanf("%d %d", &weights[i], &values[i]);  
}
```

// Read the warehouse capacity

if (scanf("%d", &W) != 1) return 0;

// dp[w] will store the maximum profit for capacity w

// Using a 1D array to save space (Optimized Knapsack)

int dp[W + 1];

// Initialize DP array with 0

```
for (int i = 0; i <= W; i++) {  
    dp[i] = 0;
```

```

    }
    // Build the DP table
    for (int i = 0; i < N; i++) {
        // Iterate backwards to ensure each item is used only once
        for (int w = W; w >= weights[i]; w--) {
            dp[w] = max(dp[w], values[i] + dp[w - weights[i]]);
        }
    }

    // The answer is the maximum profit for capacity W
    printf("%d\n", dp[W]);

    return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

You are a savvy thief preparing to rob a jewelry store. You have a knapsack with a limited capacity and a list of items, each with a certain weight and a corresponding profit.

Your objective is to maximize the total profit you can obtain by selecting items to place in your knapsack while staying within its weight capacity. Write a program to help you calculate the maximum profit you can achieve.

Note: Use the 0/1 knapsack method to solve the program.

Example

Input:

3

60 10

100 20

120 30

50

Output:

220

Explanation:

By selecting the second and third items, you achieve a maximum profit of Rs. 220. The sum of their profits (100 + 120) gives you the highest total profit while not exceeding the knapsack's weight capacity of 50.

Input Format

The first line contains an integer N, representing the number of items available in the store.

Each of the next N lines contains two space-separated integers P and W, where P is the profit that can be obtained by stealing the item and W is the weight of the item.

The last line contains an integer C representing the weight capacity of your knapsack.

Output Format

The output displays an integer representing the maximum total profit that you can achieve by selecting items for your knapsack.

Refer to the sample output for the formatting specifications

Sample Test Case

Input: 3

60 10

100 20

120 30

50

Output: 220

Answer

```

// You are using GCC
#include <stdio.h>

// Helper function to find the maximum of two integers
int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int N, C;

    // Read number of items
    if (scanf("%d", &N) != 1) return 0;

    int profits[N];
    int weights[N];

    // Read Profit and Weight for each item
    for (int i = 0; i < N; i++) {
        scanf("%d %d", &profits[i], &weights[i]);
    }

    // Read total knapsack capacity
    scanf("%d", &C);

    // dp[i][w] will store the maximum profit for 'i' items and capacity 'w'
    int dp[N + 1][C + 1];

    // Build the table in a bottom-up manner
    for (int i = 0; i <= N; i++) {
        for (int w = 0; w <= C; w++) {
            // Base case: 0 items or 0 capacity results in 0 profit
            if (i == 0 || w == 0) {
                dp[i][w] = 0;
            }
            // If weight of current item is less than or equal to capacity w
            else if (weights[i - 1] <= w) {
                dp[i][w] = max(profits[i - 1] + dp[i - 1][w - weights[i - 1]],
                               dp[i - 1][w]);
            }
            // If current item is heavier than capacity w, we skip it
            else {

```

```
        dp[i][w] = dp[i - 1][w];  
    }  
}
```

```
// The result is in the bottom-right corner of the table  
printf("%d\n", dp[N][C]);
```

```
return 0;  
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: SUSHANT KUMAR

Email: vtu29971@veltech.edu.in

Roll no: vtu29971

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L10

VTU_DAA_Week 4_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Meera is preparing for a science exhibition and needs to select a group of items whose total weight is exactly equal to a required target value. Each item can be selected at most once.

Given a list of item weights and a target weight K, determine whether it is possible to select a subset of items such that their sum is exactly K.

If such a subset exists, print any one valid subset. If no such subset exists, print -1.

This problem is derived from the Subset Sum problem, which is NP-Complete, and the solution must be implemented using Dynamic Programming.

Input Format

The first line contains two integers n and K

The second line contains n space-separated integers

Output Format

Print elements of one valid subset whose sum equals K

If no such subset exists, print -1

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5 9

3 3 4 1 2 5

Output: 5 4

Answer

// You are using GCC

#include <stdio.h>

#include <stdbool.h>

```
void findSubset(int n, int K, int arr[]) {
```

```
    // dp[i][j] will be true if a sum j can be formed by a subset of the first i elements
    bool dp[n + 1][K + 1];
```

```
    // Initialization
```

```
    for (int i = 0; i <= n; i++) {
```

```
        for (int j = 0; j <= K; j++) {
```

```
            if (j == 0)
```

```
                dp[i][j] = true; // Sum 0 is always possible (empty set)
```

```
            else
```

```
                dp[i][j] = false;
```

```
        }
```

```
    }
```

```
    // Fill the DP table
```

```
    for (int i = 1; i <= n; i++) {
```

```
        for (int j = 1; j <= K; j++) {
```

```
            if (arr[i - 1] <= j) {
```

```

        // Either exclude the item or include it
        dp[i][j] = dp[i - 1][j] || dp[i - 1][j - arr[i - 1]];
    } else {
        // Exclude the item
        dp[i][j] = dp[i - 1][j];
    }
}
}

// If no subset sums up to K
if (!dp[n][K]) {
    printf("-1\n");
    return;
}

// Backtrack to find the elements
int curr_sum = K;
bool first = true;
for (int i = n; i > 0 && curr_sum > 0; i--) {
    // If the sum was possible without the current item, move to the previous
    item
    // Otherwise, the current item is part of the subset
    if (!dp[i - 1][curr_sum]) {
        if (!first) printf(" ");
        printf("%d", arr[i - 1]);
        curr_sum -= arr[i - 1];
        first = false;
    }
}
printf("\n");
}

int main() {
    int n, K;
    if (scanf("%d %d", &n, &K) != 2) return 0;

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    findSubset(n, K, arr);
}

```

```
} return 0;
```

Status : Correct

Marks : 10/10

2. Problem Statement

Rohit is a travel planner who must organize a business trip covering multiple cities.

The travel cost between each pair of cities is given in a matrix.

He must:

Start from city 0
Visit every city exactly once
Return back to city 0
Your task is to compute:

The minimum possible travel cost
The sequence of cities visited
The average cost per step (excluding the final return step)

This problem is a classic Travelling Salesman Problem (TSP) solved using Dynamic Programming with Bitmasking (Held–Karp Algorithm).

Input Format

- The first line contains an integer n , the number of cities.
- The next n lines contain n space-separated integers, representing the cost matrix.

$\text{cost}[i][j]$ is the cost of traveling from city i to city j .

Output Format

Print the path:

The Path is: 0--->...--->0

Print the minimum cost:

Minimum cost is X

Print the average cost per step:

Average cost per step is Y

Rounded to 2 decimal places.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Output: The Path is: 0--->2--->3--->1--->0

Minimum cost is 80

Average cost per step is 26.67

Answer

```
#include <stdio.h>
```

```
#define INF 1e9
```

```
int n;
```

```
int cost[10][10];
```

```
int dp[1 << 10][10];
```

```
int parent[1 << 10][10];
```

```
void solve() {
```

```
    int full_mask = (1 << n) - 1;
```

```

for (int i = 0; i < (1 << n); i++) {
    for (int j = 0; j < n; j++) {
        dp[i][j] = INF;
    }
}

dp[1][0] = 0;

for (int mask = 1; mask <= full_mask; mask++) {
    for (int u = 0; u < n; u++) {
        if (!(mask & (1 << u))) continue;
        for (int v = 0; v < n; v++) {
            if (mask & (1 << v)) continue;
            int next_mask = mask | (1 << v);
            if (dp[next_mask][v] > dp[mask][u] + cost[u][v]) {
                dp[next_mask][v] = dp[mask][u] + cost[u][v];
                parent[next_mask][v] = u;
            }
        }
    }
}

```

```

int min_total_cost = INF;
int last_city = -1;

```

```

for (int i = 1; i < n; i++) {
    int current_res = dp[full_mask][i] + cost[i][0];
    if (current_res < min_total_cost) {
        min_total_cost = current_res;
        last_city = i;
    }
}

```

```

// Path Reconstruction
int path[11];
int curr_mask = full_mask;
int curr_city = last_city;
path[n] = 0;
for (int i = n - 1; i >= 1; i--) {
    path[i] = curr_city;
    int prev_city = parent[curr_mask][curr_city];
    curr_mask ^= (1 << curr_city);
}

```

```

    curr_city = prev_city;
}
path[0] = 0;

printf("The Path is: ");
for (int i = 0; i <= n; i++) {
    printf("%d", path[i]);
    if (i < n) printf("---->");
}
printf("\n");

printf("Minimum cost is %d\n", min_total_cost);

// Corrected Average Calculation: (Total Cost) / (n - 1)
double avg_cost = (double)min_total_cost / (n - 1);
printf("Average cost per step is %.2f\n", avg_cost);
}

int main() {
    if (scanf("%d", &n) != 1) return 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
        }
    }
    solve();
    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Captain Aidan, a daring pirate, has discovered an ancient treasure cave filled with valuable relics. However, the cave is protected by an ancient spell that limits the weight of the treasure he can carry. Aidan has a magic knapsack with a maximum weight capacity of W . Inside the cave, he finds n different treasures, each with a specific weight and value.

Since he can only take each treasure once, Aidan must carefully choose

which items to take to maximize the total value without exceeding the weight limit of his knapsack.

Help Captain Aidan pick the most valuable treasures while staying within the weight limit using branch and bound technique

Input Format

The first line contains two integers n (the number of items) and W (the capacity of the knapsack), separated by a space.

The second line contains n integers, where the i-th integer represents the weight of the i-th item.

The third line contains n integers, where the i-th integer represents the value of the i-th item

Output Format

The output prints the single integer representing the maximum value obtained by filling the knapsack with the given set of items.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5 60
10 20 30 40 50
60 100 120 140 180
Output: 280

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>
```

```
typedef struct {
    int weight;
    int value;
    double ratio;
} Item;
```

```
typedef struct {  
    int level;  
    int profit;  
    int weight;  
    double bound;  
} Node;
```

```
// Comparison function to sort items by value/weight ratio
```

```
int compare(const void* a, const void* b) {  
    Item* i1 = (Item*)a;  
    Item* i2 = (Item*)b;  
    if (i2->ratio > i1->ratio) return 1;  
    if (i2->ratio < i1->ratio) return -1;  
    return 0;  
}
```

```
// Function to calculate the upper bound of profit in a subtree
```

```
double calculateBound(Node u, int n, int W, Item items[]) {  
    if (u.weight >= W) return 0;
```

```
    double profit_bound = u.profit;  
    int j = u.level + 1;  
    int total_weight = u.weight;
```

```
    while (j < n && total_weight + items[j].weight <= W) {  
        total_weight += items[j].weight;  
        profit_bound += items[j].value;  
        j++;  
    }
```

```
// If there's still capacity, add fractional part of the next item
```

```
if (j < n) {  
    profit_bound += (W - total_weight) * items[j].ratio;  
}
```

```
    return profit_bound;  
}
```

```
int main() {
```

```
    int n, W;  
    if (scanf("%d %d", &n, &W) != 2) return 0;
```



```

Item items[n];
int weights[n], values[n];

for (int i = 0; i < n; i++) scanf("%d", &weights[i]);
for (int i = 0; i < n; i++) scanf("%d", &values[i]);

for (int i = 0; i < n; i++) {
    items[i].weight = weights[i];
    items[i].value = values[i];
    items[i].ratio = (double)values[i] / weights[i];
}

```

```

// Step 1: Sort items by ratio
qsort(items, n, sizeof(Item), compare);

```

```

// Step 2: Branch and Bound using a simple Queue (BFS approach)
Node queue[10000]; // Simple array-based queue for BFS
int head = 0, tail = 0;

```

```

Node u, v;
u.level = -1;
u.profit = 0;
u.weight = 0;
u.bound = calculateBound(u, n, W, items);

```

```

queue[tail++] = u;
int maxProfit = 0;

```

```

while (head < tail) {
    u = queue[head++];

```

```

    if (u.level == n - 1) continue;

```

```

    v.level = u.level + 1;

```

```

    // Case 1: Taking the current item
    v.weight = u.weight + items[v.level].weight;
    v.profit = u.profit + items[v.level].value;

```

```

    if (v.weight <= W && v.profit > maxProfit) {
        maxProfit = v.profit;
    }

```

```

    }

    v.bound = calculateBound(v, n, W, items);
    if (v.bound > maxProfit) {
        queue[tail++] = v;
    }

    // Case 2: Not taking the current item
    v.weight = u.weight;
    v.profit = u.profit;
    v.bound = calculateBound(v, n, W, items);

    if (v.bound > maxProfit) {
        queue[tail++] = v;
    }
}

printf("%d\n", maxProfit);

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Josh is learning about the Fractional Knapsack problem, where a knapsack has a fixed maximum capacity. Given a set of items, each with a value and a weight, Josh wants to maximize the total value placed into the knapsack.

Unlike the 0/1 Knapsack problem, fractions of items are allowed. Items must be selected using a Greedy strategy based on their value-to-weight ratio.

Write a program to determine the fraction of each item that should be included in the knapsack to achieve the maximum total value.

Formula

Value-to-Weight Ratio

$\text{ratio} = \text{value} / \text{weight}$

Fraction Calculation

$\text{fraction} = \text{remainingCapacity} / \text{weight}$

Notes

Items must be selected using the Greedy approach based on value-to-weight ratio. Sorting is required to determine the optimal order. Output fractions must be printed in the original item order. Use double datatype only for calculations.

Input Format

The first line contains an integer n , representing the number of items.

The second line contains n space-separated integers representing item values.

The third line contains n space-separated integers representing item weights.

The fourth line contains an integer capacity, representing knapsack capacity.

Output Format

Print n space-separated double values, representing the fraction of each item selected.

Output must be printed rounded to exactly two decimal places.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

3 5 1 2 4

40 50 20 10 30

75

Output: 0.00 0.70 0.00 1.00 1.00

Answer

// You are using GCC

```
#include <stdio.h>
#include <stdlib.h>
```

```
typedef struct {
    int id;
    int value;
    int weight;
    double ratio;
    double fraction;
} Item;
```

```
// Comparator to sort items by ratio in descending order
```

```
int compare(const void* a, const void* b) {
    Item* item1 = (Item*)a;
    Item* item2 = (Item*)b;
    if (item2->ratio > item1->ratio) return 1;
    if (item2->ratio < item1->ratio) return -1;
    return 0;
}
```

```
// Comparator to sort items back to their original input order
```

```
int compareOriginal(const void* a, const void* b) {
    Item* item1 = (Item*)a;
    Item* item2 = (Item*)b;
    return item1->id - item2->id;
}
```

```
int main() {
    int n, capacity;
    if (scanf("%d", &n) != 1) return 0;
```

```
    Item items[n];
    int values[n], weights[n];
```

```
    for (int i = 0; i < n; i++) scanf("%d", &values[i]);
    for (int i = 0; i < n; i++) scanf("%d", &weights[i]);
    scanf("%d", &capacity);
```

```
    for (int i = 0; i < n; i++) {
        items[i].id = i;
        items[i].value = values[i];
        items[i].weight = weights[i];
```

```

items[i].ratio = (double)values[i] / weights[i];
items[i].fraction = 0.0;
}

// Sort by ratio descending to apply Greedy logic
qsort(items, n, sizeof(Item), compare);

double remaining = (double)capacity;
for (int i = 0; i < n; i++) {
    if (remaining <= 0) {
        items[i].fraction = 0.0;
    } else if (items[i].weight <= remaining) {
        items[i].fraction = 1.0;
        remaining -= items[i].weight;
    } else {
        items[i].fraction = remaining / items[i].weight;
        remaining = 0;
    }
}

// Sort back to original order for output
qsort(items, n, sizeof(Item), compareOriginal);

for (int i = 0; i < n; i++) {
    printf("%.2f%s", items[i].fraction, (i == n - 1) ? "" : " ");
}
printf("\n");

return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

Ananya is preparing for a long trek and has a bag with limited capacity. She has several items, each with a weight and value, and she wants to maximize the total value she can carry. She may take whole items or fractions of them to fit in her bag.

The maximum value is calculated using the formula:

$$\text{Maximum Value} = \sum (\text{value}[i] * \text{fraction_taken}[i])$$

where $\text{fraction_taken}[i] = \min(1, \text{remaining_capacity} / \text{weight}[i])$

Write a program to help Ananya determine the maximum achievable value.

Input Format

The input consists of several lines, each representing an item.

Each line (until -1) contains two integers: weight and value of an item.

The input terminates with -1.

The final line contains one integer representing the bag's capacity.

Output Format

The output prints "The maximum value of the current list is:"

On the next line, print the maximum value achievable with exactly two decimal places

Refer to the sample outputs for the formatting specifications.

Sample Test Case

Input: 10 60

20 100

30 120

-1

50

Output: The maximum value of the current list is:

240.00

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct {  
    int weight;  
    int value;  
    double ratio;  
} Item;
```

```
// Comparison function to sort items by value/weight ratio in descending order
```

```
int compare(const void* a, const void* b) {  
    Item* item1 = (Item*)a;  
    Item* item2 = (Item*)b;  
    if (item2->ratio > item1->ratio) return 1;  
    if (item2->ratio < item1->ratio) return -1;  
    return 0;  
}
```

```
int main() {  
    Item items[1000];  
    int n = 0;  
    int w, v;
```

```
    // Read input until -1 is encountered  
    while (scanf("%d", &w) == 1 && w != -1) {  
        scanf("%d", &v);  
        items[n].weight = w;  
        items[n].value = v;  
        items[n].ratio = (double)v / w;  
        n++;  
    }
```

```
    int capacity;  
    scanf("%d", &capacity);
```

```
    // Sort items by ratio  
    qsort(items, n, sizeof(Item), compare);
```

```
    double totalValue = 0.0;  
    int remainingCapacity = capacity;
```

```
    for (int i = 0; i < n; i++) {  
        if (remainingCapacity == 0) break;  
        if (items[i].weight <= remainingCapacity) {
```

```
// Take the whole item
totalValue += items[i].value;
remainingCapacity -= items[i].weight;
} else {
    // Take a fraction of the item
    totalValue += items[i].value * ((double)remainingCapacity /
items[i].weight);
    remainingCapacity = 0;
}
}

printf("The maximum value of the current list is:\n");
printf("%.2f\n", totalValue);

return 0;
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: SUSHANT KUMAR
Email: vtu29971@veltech.edu.in
Roll no: vtu29971
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L10

VTU_DAA_Week 4_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Janu is tasked with creating a program that solves the Traveling Salesman Problem (TSP). The goal is to find the shortest possible route that visits a set of cities and returns to the starting city.

You are required to implement a program that takes input regarding the number of cities and the distance matrix between cities and outputs the optimal route and its associated minimum cost.

The diagonal values will be 0, as the distance from a city to itself is 0.

Note: Solve it using Branch and Bound approach.

Input Format

The first line of input consists of an integer n , representing the number of cities.

The next n lines contain n space-separated integers each, representing the distance matrix between the cities.

The cell (i, j) represents the distance from city i to city j .

Output Format

The first line of output displays "The Path is: " followed by the path of the route using ---> in between.

The second line displays "Minimum cost is " followed by the total minimum cost associated with the shortest route.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 4 1 3

4 0 2 1

1 2 0 5

3 1 5 0

Output: The Path is: 1 ---> 3 ---> 2 ---> 4 ---> 1

Minimum cost is 7

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
#include <limits.h>
```

```
#define MAX 15
```

```
int n;
```

```
int adj[MAX][MAX];
```

```
int final_path[MAX + 1];
```

```
bool visited[MAX];
```

```
int final_res = INT_MAX;
```

// Utility function to find the minimum edge cost for a city

```
int firstMin(int i) {  
    int min = INT_MAX;  
    for (int k = 0; k < n; k++)  
        if (adj[i][k] < min && i != k)  
            min = adj[i][k];  
    return min;  
}
```

// Utility function to find the second minimum edge cost for a city

```
int secondMin(int i) {  
    int first = INT_MAX, second = INT_MAX;  
    for (int j = 0; j < n; j++) {  
        if (i == j) continue;  
        if (adj[i][j] <= first) {  
            second = first;  
            first = adj[i][j];  
        } else if (adj[i][j] <= second && adj[i][j] != first) {  
            second = adj[i][j];  
        }  
    }  
    return second;  
}
```

// Recursive function to solve TSP using Branch and Bound

```
void TSPRec(int curr_bound, int curr_weight, int level, int curr_path[]) {  
    if (level == n) {  
        // Check if there is an edge back to the start city  
        if (adj[curr_path[level - 1]][curr_path[0]] != 0) {  
            int curr_res = curr_weight + adj[curr_path[level - 1]][curr_path[0]];  
            if (curr_res < final_res) {  
                for (int i = 0; i < n; i++)  
                    final_path[i] = curr_path[i];  
                final_path[n] = curr_path[0];  
                final_res = curr_res;  
            }  
        }  
    }  
    return;  
}
```

```
for (int i = 0; i < n; i++) {  
    if (adj[curr_path[level - 1]][i] != 0 && !visited[i]) {
```

```

    int temp = curr_bound;
    curr_weight += adj[curr_path[level - 1]][i];

    // Adjust lower bound for the next node
    if (level == 1)
        curr_bound -= ((firstMin(curr_path[level - 1]) + firstMin(i)) / 2);
    else
        curr_bound -= ((secondMin(curr_path[level - 1]) + firstMin(i)) / 2);

    // If current lower bound + weight < current best result, proceed
    if (curr_bound + curr_weight < final_res) {
        curr_path[level] = i;
        visited[i] = true;
        TSPRec(curr_bound, curr_weight, level + 1, curr_path);
    }

    // Backtrack
    curr_weight -= adj[curr_path[level - 1]][i];
    curr_bound = temp;
    visited[i] = false;
}
}
}

```

```

void solve() {
    int curr_path[MAX + 1];
    int curr_bound = 0;

    // Initial lower bound calculation
    for (int i = 0; i < n; i++)
        curr_bound += (firstMin(i) + secondMin(i));

    curr_bound = (curr_bound % 2 == 1) ? curr_bound / 2 + 1 : curr_bound / 2;

    visited[0] = true;
    curr_path[0] = 0;

    TSPRec(curr_bound, 0, 1, curr_path);
}

```

```

int main() {
    if (scanf("%d", &n) != 1) return 0;

```

```

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        scanf("%d", &adj[i][j]);
    }
}

solve();

// Print the Path (converting back to 1-based indexing)
printf("The Path is: ");
for (int i = 0; i <= n; i++) {
    printf("%d", final_path[i] + 1);
    if (i < n) printf("---->");
}
printf("\n");

// Print the Minimum Cost
printf("Minimum cost is %d\n", final_res);

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Karan is planning a delivery route that starts from city 0, visits every other city exactly once, and returns back to city 0.

The travel costs between cities are given as a cost matrix.

Your task is to find the minimum possible travel cost and print the optimal route.

Cost Formula

For a route:

0 a b ... 0

Total Cost = cost[0][a] + cost[a][b] + ... + cost[last][0]

Input Format

The first line contains an integer n, representing the number of cities.

The next n lines contain n space-separated integers representing the cost matrix.

Output Format

The output prints multiple lines showing path exploration in the format:

The first line prints the minimum travel cost.

The second line prints the optimal path starting from city 0 and ending at city 0.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Output: 80

0 1 3 2 0

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
int n;
```

```
int cost[5][5];
```

```
int min_cost = 1e9;
int best_path[6];
int current_path[6];
bool visited[5];
```

```
void findPath(int curr_city, int count, int current_cost) {
    // Base Case: All cities visited, now return to city 0
    if (count == n) {
        int total_cost = current_cost + cost[curr_city][0];
        if (total_cost < min_cost) {
            min_cost = total_cost;
            for (int i = 0; i < n; i++) {
                best_path[i] = current_path[i];
            }
            best_path[n] = 0; // Return to start
        }
        return;
    }
}
```

```
// Try visiting all other cities
for (int next_city = 0; next_city < n; next_city++) {
    if (!visited[next_city]) {
        visited[next_city] = true;
        current_path[count] = next_city;

        findPath(next_city, count + 1, current_cost + cost[curr_city][next_city]);

        // Backtrack
        visited[next_city] = false;
    }
}
}
```

```
int main() {
    if (scanf("%d", &n) != 1) return 0;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
        }
    }
}
```

```

// Initialize tracking
for (int i = 0; i < n; i++) visited[i] = false;

// Start from city 0
visited[0] = true;
current_path[0] = 0;

findPath(0, 1, 0);

// Output the minimum cost
printf("%d\n", min_cost);

// Output the optimal path
for (int i = 0; i <= n; i++) {
    printf("%d%s", best_path[i], (i == n) ? "" : " ");
}
printf("\n");

return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

You are a logistics manager responsible for planning delivery routes for a fleet of vehicles. Each vehicle needs to visit a set of locations to deliver goods, and the cost of traveling between locations is known.

Your goal is to find the shortest route for each vehicle, ensuring that each location is visited exactly once before returning to the starting point.

Note: Solve it using Branch and Bound approach.

Input Format

The first line contains an integer n , the number of locations.

The next n lines contain n integers each, representing the cost matrix of traveling between locations.

The i th integer on the j th line represents the cost of traveling from location i to location j .

Output Format

The first line of output displays "Shortest Path: " followed by the shortest path that visits all cities.

The second line of output displays "Minimum cost is " followed by the minimum cost.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
0 10 8 9 7
10 0 10 5 6
8 10 0 8 9
9 5 8 0 6
7 6 9 6 0

Output: Shortest Path: 1 3 4 2 5 1
Minimum cost is 34

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdbool.h>
#include <limits.h>
```

```
#define MAX 10
```

```
int n;
int adj[MAX][MAX];
int final_path[MAX + 1];
bool visited[MAX];
int final_res = INT_MAX;
```

```
// Utility function to find the minimum edge cost for a city
int firstMin(int i) {
    int min = INT_MAX;
```

```

    for (int k = 0; k < n; k++)
        if (adj[i][k] < min && i != k)
            min = adj[i][k];
    return min;
}

```

// Utility function to find the second minimum edge cost for a city

```

int secondMin(int i) {
    int first = INT_MAX, second = INT_MAX;
    for (int j = 0; j < n; j++) {
        if (i == j) continue;
        if (adj[i][j] <= first) {
            second = first;
            first = adj[i][j];
        } else if (adj[i][j] <= second && adj[i][j] != first) {
            second = adj[i][j];
        }
    }
    return second;
}

```

// Recursive function to solve TSP using Branch and Bound

```

void TSPRec(int curr_bound, int curr_weight, int level, int curr_path[]) {
    // Base case: all cities visited
    if (level == n) {
        // Check if there is an edge back to the starting city (City 0)
        if (adj[curr_path[level - 1]][curr_path[0]] != 0) {
            int curr_res = curr_weight + adj[curr_path[level - 1]][curr_path[0]];
            if (curr_res < final_res) {
                for (int i = 0; i < n; i++)
                    final_path[i] = curr_path[i];
                final_path[n] = curr_path[0];
                final_res = curr_res;
            }
        }
    }
    return;
}

```

```

for (int i = 0; i < n; i++) {
    if (adj[curr_path[level - 1]][i] != 0 && !visited[i]) {
        int temp_bound = curr_bound;
        curr_weight += adj[curr_path[level - 1]][i];
    }
}

```

```

// Adjust lower bound for the next node
if (level == 1)
    curr_bound -= ((firstMin(curr_path[level - 1]) + firstMin(i)) / 2);
else
    curr_bound -= ((secondMin(curr_path[level - 1]) + firstMin(i)) / 2);

// Pruning: if current cost + bound is better than global minimum
if (curr_bound + curr_weight < final_res) {
    curr_path[level] = i;
    visited[i] = true;
    TSPRec(curr_bound, curr_weight, level + 1, curr_path);
}

// Backtracking
curr_weight -= adj[curr_path[level - 1]][i];
curr_bound = temp_bound;
visited[i] = false;
}
}
}

```

```

void solve() {
    int curr_path[MAX + 1];
    int curr_bound = 0;

    // Calculate initial lower bound
    for (int i = 0; i < n; i++)
        curr_bound += (firstMin(i) + secondMin(i));

    // Divide by 2 (round up for odd sums)
    curr_bound = (curr_bound % 2 == 1) ? curr_bound / 2 + 1 : curr_bound / 2;

    visited[0] = true;
    curr_path[0] = 0;

    TSPRec(curr_bound, 0, 1, curr_path);
}

```

```

int main() {
    if (scanf("%d", &n) != 1) return 0;

```

```

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &adj[i][j]);
        }
    }

    solve();

    // Print Shortest Path (1-based indexing)
    printf("Shortest Path: ");
    for (int i = 0; i <= n; i++) {
        printf("%d%s", final_path[i] + 1, (i == n) ? "" : " ");
    }
    printf("\n");

    // Print Minimum Cost
    printf("Minimum cost is %d\n", final_res);

    return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Michael has been given an important task—developing a program to solve the classic 0/1 Knapsack Problem. His goal is to help users select the most valuable combination of items while staying within a specified weight limit.

In this problem, each item has a value and a weight, and the objective is to maximize the total value without exceeding the given weight capacity.

Michael's program should determine the maximum possible value that can be obtained by selecting the optimal set of items.

Input Format

The first line of input consists of an integer n , representing the number of items available for selection.

The second line of input consists of n space-separated integers, P, which represent the values of the items.

The third line of input consists of n space-separated integers, C, which represent the weights of the corresponding items.

The fourth line of input consists of an integer W, representing the maximum weight limit of the knapsack.

Output Format

The output displays the maximum value that can be obtained using the knapsack algorithm in the following format: "Maximum value: [maximum value]"

Refer to the sample outputs for the exact format.

Sample Test Case

Input: 6
300 150 120 100 90 80
6 3 3 2 2 2
10

Output: Maximum value: 490

Answer

```
// You are using GCC
#include <stdio.h>

// Function to find the maximum of two integers
int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n, W;

    // Read the number of items
    if (scanf("%d", &n) != 1) return 0;

    int values[n];
    int weights[n];
```

```

// Read the values of the items
for (int i = 0; i < n; i++) {
    scanf("%d", &values[i]);
}

// Read the weights of the items
for (int i = 0; i < n; i++) {
    scanf("%d", &weights[i]);
}

// Read the maximum weight capacity
if (scanf("%d", &W) != 1) return 0;

// dp[w] will store the maximum value for weight capacity w
int dp[W + 1];

// Initialize the DP array with 0
for (int i = 0; i <= W; i++) {
    dp[i] = 0;
}

// Process each item
for (int i = 0; i < n; i++) {
    // Iterate backwards through the DP array
    // This ensures we don't use the same item more than once for the same
    capacity
    for (int w = W; w >= weights[i]; w--) {
        dp[w] = max(dp[w], values[i] + dp[w - weights[i]]);
    }
}

// Output the result in the specified format
printf("Maximum value: %d\n", dp[W]);

return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

Ragul is interested in solving the knapsack problem. He has a list of n items, each with a weight and a value. Ragul wants to determine the maximum value he can obtain by selecting a combination of items to fit into his knapsack, which has a maximum weight capacity.

Help Ragul write a program that calculates the following:

Calculate and print the sum of all n values. Calculate and print the average value of all n values with exactly 2 decimal places. Calculate and print the maximum value that can be obtained by selecting a combination of items to fit into the knapsack without exceeding its capacity.

Note: Use the 0/1 knapsack method to solve the problem.

Input Format

The first line of input contains an integer n , representing the number of items.

The next line contains n space-separated integers, representing the weights of the items.

The next line contains n space-separated integers, representing the values of the items.

The last line contains an integer C , representing the maximum weight capacity of the knapsack.

Output Format

The output consists of the following in each line:

1. The sum of all n values as an integer.
2. The average value of all n values with exactly 2 decimal places is a double value.
3. The maximum value that can be obtained as an integer.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
10 15 20
45 65 70
30

Output: Sum of values: 180
Average of values: 60.00
Maximum amount: 115

Answer

```
// You are using GCC
#include <stdio.h>
```

```
// Function to find the maximum of two integers
```

```
int max(int a, int b) {
    return (a > b) ? a : b;
}
```

```
int main() {
    int n, C;
```

```
    // Read the number of items
    if (scanf("%d", &n) != 1) return 0;
```

```
    int weights[n];
    int values[n];
    int sum = 0;
```

```
    // Read weights
    for (int i = 0; i < n; i++) {
        scanf("%d", &weights[i]);
    }
```

```
    // Read values and calculate sum
    for (int i = 0; i < n; i++) {
        scanf("%d", &values[i]);
        sum += values[i];
    }
```

```
    // Read the knapsack capacity
    scanf("%d", &C);
```

```
    // 1. Print Sum
```



```

printf("Sum of values: %d\n", sum);

// 2. Calculate and print Average
double average = (double)sum / n;
printf("Average of values: %.2f\n", average);

// 3. Solve 0/1 Knapsack using DP
int dp[C + 1];

// Initialize DP array with 0
for (int i = 0; i <= C; i++) {
    dp[i] = 0;
}

for (int i = 0; i < n; i++) {
    // Traverse backwards to ensure 0/1 property
    for (int w = C; w >= weights[i]; w--) {
        dp[w] = max(dp[w], values[i] + dp[w - weights[i]]);
    }
}

// Print Maximum Value
printf("Maximum amount: %d\n", dp[C]);

return 0;
}

```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: SUSHANT KUMAR
Email: vtu29971@veltech.edu.in
Roll no: vtu29971
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L10

VTU_DAA_Week 1_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 11

Section 1 : MCQ

1. An algorithm must have a clear stopping point to ensure

Answer

Finiteness

Status : Correct

Marks : 1/1

2. Which of the following is true about algorithmic determinism?

Answer

Algorithms must produce the same output for the same input

Status : Correct

Marks : 1/1

3. An algorithm's effectiveness refers to

Answer

Each step contributes to solving the problem

Status : Correct

Marks : 1/1

4. Which notation is used to describe both the upper and lower bounds of an algorithm's efficiency?

Answer

None of the mentioned options

Status : Wrong

Marks : 0/1

5. What is the purpose of using asymptotic notations in the analysis of algorithm efficiency?

Answer

To take meaningful statements about an algorithm's efficiency based on its growth rate

Status : Correct

Marks : 1/1

6. What does the Big Theta notation (Θ) indicate about an algorithm's efficiency?

Answer

It describes the tightest possible bound on the algorithm's running time.

Status : Correct

Marks : 1/1

7. Which of the following statements is true regarding the relationship between Big Oh (O), Big Omega (Ω), and Big Theta (Θ) notations?

Answer

If $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$, then $f(n) = \Theta(g(n))$

Status : Correct

Marks : 1/1

8. What does "Big O" notation represent in relation to algorithms?

Answer

A method to analyze and express the efficiency of an algorithm

Status : Correct

Marks : 1/1

9. An algorithm is required to produce a definite and unambiguous outcome for every valid input. This characteristic is known as _____.

Answer

determinism

Status : Correct

Marks : 1/1

10. Which approach is used to prove the correctness of algorithms by analyzing a sequence of steps?

Answer

Mathematical induction

Status : Correct

Marks : 1/1

11. In the context of algorithm design, what does the term "in-place" refer to?

Answer

The algorithm works for both sequential and parallel execution

Status : Wrong

Marks : 0/1

12. What is the primary purpose of analyzing an algorithm's efficiency?

Answer

To optimize the algorithm's time and space efficiency

Status : Correct

Marks : 1/1

13. In the context of algorithmic problem-solving, what does the "generality of an algorithm" refer to?

Answer

The ability of an algorithm to handle a wide range of problem types

Status : Correct

Marks : 1/1

14. Which parameter is commonly used to measure an algorithm's input size?

Answer

Number of elements in the input array/list

Status : Wrong

Marks : 0/1

15. The concept of "Orders of Growth" in algorithm analysis refers to:

Answer

The way algorithms grow in efficiency over time

Status : Wrong

Marks : 0/1

Vel Tech Group of Institutions

Name: SUSHANT KUMAR
Email: vtu29971@veltech.edu.in
Roll no: vtu29971
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L10

VTU_DAA_Week 2_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 13

Section 1 : MCQ

1. What is a non-recursive algorithm?

Answer

An algorithm that does not use recursion

Status : Correct

Marks : 1/1

2. Which of the following is an example of a non-recursive sorting algorithm?

Answer

Bubble Sort

Status : Correct

Marks : 1/1

3. The time complexity of non-recursive algorithms is often expressed using:

Answer

All of the mentioned options

Status : Correct

Marks : 1/1

4. Which non-recursive algorithm is commonly used for searching in a sorted array by repeatedly dividing the search interval in half?

Answer

Binary Search

Status : Correct

Marks : 1/1

5. In non-recursive algorithms, iteration is commonly achieved through:

Answer

Loops

Status : Correct

Marks : 1/1

6. What is the output of the code?

```
#include <stdio.h>
```

```
int main() {  
    int num = 2;  
    do {  
        printf("%d ", num);  
        num *= 2;  
    } while (num <= 8);  
    return 0;  
}
```

Answer

2 4 8

Status : Correct

Marks : 1/1

7. What is the output for the code?

```
#include <stdio.h>
```

```
void nonRecursiveAlgorithm(int n) {  
    for (int i = 1; i <= n; i *= 2) {  
        printf("%d ", i);  
    }  
}
```

```
int main() {  
    nonRecursiveAlgorithm(8);  
    return 0;  
}
```

Answer

1 2 4 8

Status : Correct

Marks : 1/1

8. What is the output for the code?

```
#include <stdio.h>
```

```
void nonRecursiveFunction1(int n) {  
    while (n > 0) {  
        printf("%d ", n);  
        n /= 2;  
    }  
}
```

```
int main() {  
    int number = 80;  
    nonRecursiveFunction1(number);  
    return 0;  
}
```



```
}
```

Answer

80 40 20 10 5 2 1

Status : Correct

Marks : 1/1

9. What will be the output for the following recursive code?

```
#include <stdio.h>
```

```
int power(int base, int exponent) {  
    if (exponent == 0)  
        return 1;  
    else  
        return base * power(base, exponent - 1);  
}
```

```
int main() {  
    int result = power(3, 2);  
    printf("%d", result);  
    return 0;  
}
```

Answer

9

Status : Correct

Marks : 1/1

10. What will be the output for the following recursive code?

```
#include <stdio.h>
```

```
int num(int n, int rev) {  
    if (n == 0) {  
        return rev;  
    } else {  
        int digit = n % 10;  
        rev = rev * 10 + digit;  
    }
```

```
    return num(n / 10, rev);  
}  
}
```

```
int main() {  
    int d = 12345;  
    int rev = num(d, 0);  
    printf("%d", rev);  
    return 0;  
}
```

Answer

54321

Status : Correct

Marks : 1/1

11. What will be the output for the following recursive code?

```
#include <stdio.h>
```

```
int function(int number) {  
    if (number <= 0) {  
        return 1;  
    } else {  
        return number * function(number - 1);  
    }  
}
```

```
int main() {  
    int result;  
    result = function(4);  
    printf("%d", result);  
  
    return 0;  
}
```

Answer

24

Status : Correct

Marks : 1/1

12. What will be the output for the following recursive code?

```
#include <iostream>
using namespace std;
```

```
void print(int n) {
    if (n == 0)
        return;
    print(n / 2);
    cout << n % 2;
}
```

```
int main() {
    int num = 12;
    print(num);
    return 0;
}
```

Answer

0011

Status : Wrong

Marks : 0/1

13. What will be the output for the following recursive code?

```
#include <stdio.h>
```

```
int function(int a, int b) {
    if (b == 1)
        return a;
    else
        return a + function(a, b - 1);
}
```

```
int main() {
    int result = function(2, 3);
    printf("%d", result);
    return 0;
}
```

Answer

6

Status : Correct

Marks : 1/1

14. What will be the output for the following recursive code?

```
#include <stdio.h>
```

```
int function(int a, int b) {  
    if (b == 0)  
        return 0;  
    if (b == 1)  
        return a;  
    return a + function(a, b - 1);  
}
```

```
int main() {  
    int result = function(3, 8);  
    printf("%d", result);  
    return 0;  
}
```

Answer

24

Status : Correct

Marks : 1/1

15. What will be the output for the following recursive code?

```
#include <iostream>  
using namespace std;
```

```
int recursive(int n) {  
    if (n <= 0)  
        return 1;  
    else  
        return n + recursive(n - 1);  
}
```

```
}
```

```
int main() {  
    int result = recursive(5);  
    cout << result;  
    return 0;  
}
```

Answer

20

Status : Wrong

Marks : 0/1

Vel Tech Group of Institutions

Name: SUSHANT KUMAR
Email: vtu29971@veltech.edu.in
Roll no: vtu29971
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L10

VTU_DAA_Week 5_COD

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Sam needs to sort an array of integers using the divide-and-conquer method. He wants to implement the merge sort algorithm, displaying the array after each iteration to track the sorting process.

Help him write a program that divides the array, merges it back, and prints the array.

Input Format

The first line of input consists of an integer n , denoting the array size.

The second line consists of n space-separated integers, representing the array of elements.

Output Format

The first line of output prints "Given Array" followed by a (String), indicating the start of the initial array display.

The second line of output prints the array elements (integers) separated by spaces.

After each merge iteration, the output prints "After iteration X", where X is the iteration number (starting from 1). This is followed by the (String) "After iteration X" to indicate the iteration count.

On the next line, the current state of the array after that iteration is printed, showing the array elements (integers) separated by spaces.

Finally, after all iterations are completed, the output prints "Sorted Array" followed by a (String), indicating the sorted array.

The final line prints the sorted array (integers), with each element separated by spaces.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

4 1 5 3 2 6

Output: Given Array

4 1 5 3 2 6

After iteration 1

1 4 5 3 2 6

After iteration 2

1 4 5 3 2 6

After iteration 3

1 4 5 2 3 6

After iteration 4

1 4 5 2 3 6
After iteration 5
1 2 3 4 5 6
Sorted Array
1 2 3 4 5 6

Answer

You are using Python
import sys

iteration_count = 0

```
def merge(arr, low, mid, high):  
    global iteration_count  
    left = arr[low:mid + 1]  
    right = arr[mid + 1:high + 1]  
    i = j = 0  
    k = low  
    while i < len(left) and j < len(right):  
        if left[i] <= right[j]:  
            arr[k] = left[i]  
            i += 1  
        else:  
            arr[k] = right[j]  
            j += 1  
        k += 1  
    while i < len(left):  
        arr[k] = left[i]  
        i += 1  
        k += 1  
    while j < len(right):  
        arr[k] = right[j]  
        j += 1  
        k += 1  
    iteration_count += 1  
    print(f"After iteration {iteration_count}")  
    print(*(arr))
```

```
def merge_sort(arr, low, high):  
    if low < high:  
        mid = (low + high) // 2  
        merge_sort(arr, low, mid)
```



```

merge_sort(arr, mid + 1, high)
merge(arr, low, mid, high)

def main():
    input_data = sys.stdin.read().split()
    if not input_data:
        return
    n = int(input_data[0])
    arr = list(map(int, input_data[1:]))
    print("Given Array")
    print(*(arr))
    merge_sort(arr, 0, n - 1)
    print("Sorted Array")
    print(*(arr))

if __name__ == "__main__":
    main()

```

Status : Correct

Marks : 10/10

2. Problem Statement

Micheal is tasked with developing a system to manage and sort student records for a university. Each student record contains the student's name, GPA, age, and major. The goal is to implement a program that allows the user to input multiple student records and then sort these records based on a chosen criterion: GPA, age, or major.

Can you guide Micheal in developing the program using a quick sort algorithm?

Input Format

The first line of input consists of an integer n , representing the number of student records.

For each student record, the following lines consist of:

- Name (string)
- GPA (float)
- Age (integer)

- Major (string)

The last line of input consists of the sorting criterion, which can be one of the following strings:

- "gpa"
- "age"
- "major"

Output Format

The first line of output will print:

Sorted Student Records:

The output should display a sorted list of student records in a tabular format based on the chosen criterion. The GPA value should be rounded off to two decimal places.

Table Header

"Name\t\tGPA\t\tAge\t\tMajor"

Table Content

For each student, the format of the output will be:

"Name\tGPA\tAge\t\tMajor"

- The values should be aligned as follows:
- Name: Left-aligned, up to 10 characters.
- GPA: Displayed with two decimal places.
- Age: Left-aligned, up to 3 digits.
- Major: Left-aligned, up to 15 characters.

Note: "\t" - It creates a horizontal tab space

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

Harish

7.8

23

ece

Trell

8

21

cse

Kamalesh

7

25

it

Maaran

5.6

20

civil

age

Output: Sorted Student Records:

Name	GPA	Age	Major
------	-----	-----	-------

Maaran	5.60	20	civil
--------	------	----	-------

Trell	8.00	21	cse
-------	------	----	-----

Harish	7.80	23	ece
--------	------	----	-----

Kamalesh	7.00	25	it
----------	------	----	----

Answer

```
import sys
```

```
class Student:
```

```
    def __init__(self, name, gpa, age, major):
```

```
        self.name = name
```

```
        self.gpa = float(gpa)
```

```
        self.age = int(age)
```

```
        self.major = major
```

```
def get_attr(student, criterion):
```

```
    if criterion == "gpa":
```

```
        return student.gpa
```

```
    elif criterion == "age":
```

```
    return student.age
else:
    return student.major
```

```
def partition(students, low, high, criterion):
    pivot = get_attr(students[high], criterion)
    i = low - 1
    for j in range(low, high):
        val = get_attr(students[j], criterion)
        if val <= pivot:
            i += 1
            students[i], students[j] = students[j], students[i]
    students[i + 1], students[high] = students[high], students[i + 1]
    return i + 1
```

```
def quick_sort(students, low, high, criterion):
    if low < high:
        pi = partition(students, low, high, criterion)
        quick_sort(students, low, pi - 1, criterion)
        quick_sort(students, pi + 1, high, criterion)
```

```
def main():
    data = sys.stdin.read().split()
    if not data:
        return
```

```
    n = int(data[0])
    students = []
    idx = 1
    for _ in range(n):
        name = data[idx]
        gpa = data[idx+1]
        age = data[idx+2]
        major = data[idx+3]
        students.append(Student(name, gpa, age, major))
        idx += 4
```

```
    criterion = data[idx]
```

```
    quick_sort(students, 0, n - 1, criterion)
```

```
    print("Sorted Student Records:")
```

```
print("Name\t\tGPA\t\tAge\t\tMajor")
for s in students:
    print(f"{s.name}\t{s.gpa:.2f}\t{s.age}\t\t{s.major}")

if __name__ == "__main__":
    main()
```

Status : Correct

Marks : 10/10

3. Problem Statement

Ragavi, a computer scientist, is working on efficient text processing for search engines. She has a list of words in random order and needs to sort them in lexicographic order using Heap Sort to improve search performance.

Help Ragavi implement Heap Sort to arrange the words in lexicographic order efficiently.

Input Format

The first line consists of an integer N denoting the number of strings.

The next line consists of N space-separated strings.

Output Format

The output displays N space-separated sorted strings in lexicographic order.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 10

banana pineapple mango chikoo jack star guava lemon tomato orange

Output: banana chikoo guava jack lemon mango orange pineapple star tomato

Answer

You are using Python

```

import sys

def heapify(arr, n, i):
    largest = i
    l = 2 * i + 1
    r = 2 * i + 2

    if l < n and arr[l] > arr[largest]:
        largest = l

    if r < n and arr[r] > arr[largest]:
        largest = r

    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

def heap_sort(arr):
    n = len(arr)
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)
    for i in range(n - 1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]
        heapify(arr, i, 0)

def main():
    input_data = sys.stdin.read().split()
    if not input_data:
        return
    n = int(input_data[0])
    arr = input_data[1:n+1]
    heap_sort(arr)
    print(*(arr))

if __name__ == "__main__":
    main()

```

Status : Correct

Marks : 10/10

4. Problem Statement

Imagine you are working as a librarian in a large library. The library uses an automated system to keep track of book positions on the shelves. Each book is assigned a unique ID, but some IDs are more common than others due to the popularity of certain editions.

The system logs the positions of books in a sorted list based on their IDs. You need to find the first and last positions of a specific book ID in this sorted list using recursive binary search.

Example

Input:

10

1 2 2 2 2 3 4 8 8 8

8

Output:

7 9

Explanation:

The first and last positions of book ID 8 are [7, 9] since the index is 0-based.

Input Format

The first line of input consists of an integer n , representing the total number of book IDs.

The second line consists of a sorted list of n integers representing the book IDs.

The third line consists of an integer x , representing the specific book ID to find.

Output Format

The output displays indexes of the first and last occurrence of x , as space-separated integers.

If book ID x is not present in the shelves, print "NO OCCURRENCES".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10

1 2 2 2 2 3 4 8 8 8

8

Output: 7 9

Answer

You are using Python

```
import sys
```

```
def find_first(arr, low, high, x, result):
    if low > high:
        return result
    mid = (low + high) // 2
    if arr[mid] == x:
        return find_first(arr, low, mid - 1, x, mid)
    elif x < arr[mid]:
        return find_first(arr, low, mid - 1, x, result)
    else:
        return find_first(arr, mid + 1, high, x, result)
```

```
def find_last(arr, low, high, x, result):
    if low > high:
        return result
    mid = (low + high) // 2
    if arr[mid] == x:
        return find_last(arr, mid + 1, high, x, mid)
    elif x < arr[mid]:
        return find_last(arr, low, mid - 1, x, result)
    else:
        return find_last(arr, mid + 1, high, x, result)
```

```
def main():
    data = sys.stdin.read().split()
    if not data:
        return
    n = int(data[0])
```



```
arr = list(map(int, data[1:n+1]))
x = int(data[n+1])

first = find_first(arr, 0, n - 1, x, -1)
last = find_last(arr, 0, n - 1, x, -1)

if first == -1:
    print("NO OCCURRENCES")
else:
    print(f"{first} {last}")

if __name__ == "__main__":
    main()
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: SUSHANT KUMAR

Email: vtu29971@veltech.edu.in

Roll no: vtu29971

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L10

VTU_DAA_Week 5_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Given an array of integers, count the number of smaller elements that appear to the right of each element. Implement a solution using merge sort to count these elements efficiently. Output the result as an array of counts corresponding to each element in the original array.

Example

Input:

4

5 2 6 1

Output:

2 1 1 0

Explanation:

To the right of 5, there are 2 smaller elements (2 and 1).

To the right of 2, there is only 1 smaller element (1).

To the right of 6, there is 1 smaller element (1).

To the right of 1, there is 0 smaller element.

Input Format

The first line of input contains a single integer n, representing the length of the array.

The second line contains n space-separated integers, representing the elements of the array.

Output Format

The output prints a single line containing the counts of smaller elements for each value in the array, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

5 2 6 1

Output: 2 1 1 0

Answer

```
# You are using Python
import sys
```

```
def merge_sort(indices, low, high, arr, counts):
    if low < high:
        mid = (low + high) // 2
        merge_sort(indices, low, mid, arr, counts)
        merge_sort(indices, mid + 1, high, arr, counts)
        merge(indices, low, mid, high, arr, counts)
```

```
def merge(indices, low, mid, high, arr, counts):
```

```
    temp = []
```

```
    i = low
```

```
    j = mid + 1
```

```
    right_count = 0
```

```
    while i <= mid and j <= high:
```

```
        if arr[indices[j]] < arr[indices[i]]:
```

```
            right_count += 1
```

```
            temp.append(indices[j])
```

```
            j += 1
```

```
        else:
```

```
            counts[indices[i]] += right_count
```

```
            temp.append(indices[i])
```

```
            i += 1
```

```
    while i <= mid:
```

```
        counts[indices[i]] += right_count
```

```
        temp.append(indices[i])
```

```
        i += 1
```

```
    while j <= high:
```

```
        temp.append(indices[j])
```

```
        j += 1
```

```
    for k in range(len(temp)):
```

```
        indices[low + k] = temp[k]
```

```
def main():
```

```
    data = sys.stdin.read().split()
```

```
    if not data:
```

```
        return
```

```
    n = int(data[0])
```

```
    arr = list(map(int, data[1:n+1]))
```

```
    indices = list(range(n))
```

```
    counts = [0] * n
```

```
    merge_sort(indices, 0, n - 1, arr, counts)
```

```
    print(*(counts))
```

```
if __name__ == "__main__":  
    main()
```

Status : Correct

Marks : 10/10

2. Problem Statement

Ravi, a mathematics teacher, is helping his students understand quadratic equations. He wants to apply the quadratic formula $[A * x^2 + B * x + C]$ to each number in a sorted array. Once the formula is applied, the modified numbers need to be sorted again. Ravi plans to use QuickSort for efficient sorting. Help Ravi by implementing a program that performs these operations.

Input Format

The first line contains an integer n , representing the size of the array.

The second line contains n space-separated integers, representing the elements of the array in ascending order.

The third line contains three integers A , B , and C , the coefficients of the quadratic formula.

Output Format

The first line of output prints: Array after applying the equation and sorting

The subsequent lines print: <sorted_value>

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

-1 0 1 2 3 4

-1 2 -1

Output: Array after applying the equation and sorting:

-9 -4 -4 -1 -1 0

Answer

You are using Python

```
import sys
```

```
def partition(arr, low, high):
```

```
    pivot = arr[high]
```

```
    i = low - 1
```

```
    for j in range(low, high):
```

```
        if arr[j] <= pivot:
```

```
            i += 1
```

```
            arr[i], arr[j] = arr[j], arr[i]
```

```
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
```

```
    return i + 1
```

```
def quick_sort(arr, low, high):
```

```
    if low < high:
```

```
        pi = partition(arr, low, high)
```

```
        quick_sort(arr, low, pi - 1)
```

```
        quick_sort(arr, pi + 1, high)
```

```
def main():
```

```
    data = sys.stdin.read().split()
```

```
    if not data:
```

```
        return
```

```
    n = int(data[0])
```

```
    arr = list(map(int, data[1:n+1]))
```

```
    a = int(data[n+1])
```

```
    b = int(data[n+2])
```

```
    c = int(data[n+3])
```

```
    transformed = []
```

```
    for x in arr:
```

```
        val = (a * (x**2)) + (b * x) + c
```

```
        transformed.append(val)
```

```
    quick_sort(transformed, 0, n - 1)
```

```
    print("Array after applying the equation and sorting:")
```

```
    print(*(transformed))
```

```
if __name__ == "__main__":  
    main()
```

Status : Correct

Marks : 10/10

3. Problem Statement

Amit, an e-commerce manager, is working on an inventory management system to track the top-selling products. Given the sales data of various products, he wants to quickly determine the Kth best-selling product to make informed restocking decisions.

Help Amit implement Heap Sort to efficiently find the Kth best-selling product from the given sales data.

Input Format

The first line of input consists of an integer n, representing the number of products.

The second line consists of n space-separated integers, representing the sales data of each product.

The third line contains an integer K, representing the rank of the product to retrieve.

Output Format

The output prints the Kth best-selling product based on the given sales data.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
10 20 15 5 25
2
Output: 20

Answer

```
# You are using Python
import sys
```

```
def heapify(arr, n, i):
    largest = i
    l = 2 * i + 1
    r = 2 * i + 2

    if l < n and arr[l] > arr[largest]:
        largest = l

    if r < n and arr[r] > arr[largest]:
        largest = r

    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)
```

```
def get_kth_best_selling(arr, n, k):
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)
```

```
    count = 0
    for i in range(n - 1, n - k - 1, -1):
        result = arr[0]
        arr[0] = arr[i]
        heapify(arr, i, 0)
        count += 1
    if count == k:
        return result
```

```
def main():
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    n = int(input_data[0])
    arr = list(map(int, input_data[1:n+1]))
    k = int(input_data[n+1])

    print(get_kth_best_selling(arr, n, k))
```



```
if __name__ == "__main__":  
    main()
```

Status : Correct

Marks : 10/10

4. Problem Statement

Tom is working in a warehouse inventory system, where the item IDs are assigned sequentially in ascending order.

He wants to develop a program using recursive binary search to efficiently determine the closest item ID that is less than or equal to a given target ID.

The program takes the total number of items and their sorted IDs as input, assisting warehouse staff in quickly identifying the closest available item less than or equal to the target ID. Help Tom in this program.

Input Format

The first line of input consists of an integer N, representing the number of items in the warehouse.

The second line consists of N space-separated integers, representing the sorted list of item IDs.

The third line consists of an integer representing the target item ID.

Output Format

The output prints "The closest item ID less than or equal to X is Y", where X is the target ID and Y is the closest item ID less than or equal to X.

If no such element exists, print "No closest item with an ID less than or equal to X exists in the warehouse", where X is the target ID.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7

17 25 38 47 51 62 79

50

Output: The closest item ID less than or equal to 50 is 47

Answer

```
import sys
```

```
def find_floor(arr, low, high, target, res):
```

```
    if low > high:
```

```
        return res
```

```
    mid = (low + high) // 2
```

```
    if arr[mid] == target:
```

```
        return arr[mid]
```

```
    if arr[mid] < target:
```

```
        return find_floor(arr, mid + 1, high, target, arr[mid])
```

```
    else:
```

```
        return find_floor(arr, low, mid - 1, target, res)
```

```
def main():
```

```
    data = sys.stdin.read().split()
```

```
    if not data:
```

```
        return
```

```
    n = int(data[0])
```

```
    arr = list(map(int, data[1:n+1]))
```

```
    target = int(data[n+1])
```

```
    result = find_floor(arr, 0, n - 1, target, -1)
```

```
    if result == -1:
```

```
        print(f"No closest item with an ID less than or equal to {target} exists in the  
warehouse")
```

```
    else:
```

```
        print(f"The closest item ID less than or equal to {target} is {result}")
```

```
if __name__ == "__main__":
```

```
    main()
```

Status : Correct

Marks : 10/10

5. Problem Statement

Rahul owns a bookstore and maintains a sorted list of ISBN numbers of books in his inventory. When a customer asks for a specific book, Rahul wants to quickly check whether the book is available in his inventory.

Write a program that implements a non-recursive binary search to determine if a given ISBN exists in the sorted list of ISBN numbers.

Input Format

The first line contains an integer N, representing the number of ISBN numbers in the inventory.

The second line contains N space-separated integers representing the sorted list of ISBN numbers.

The third line contains a single integer X, the ISBN number to be searched.

Output Format

If the ISBN X is found in the inventory, print: "Book with ISBN X is available at index Y" where Y is the zero-based index of the ISBN in the list.

If the ISBN X is not found, print: "Book with ISBN X is not available in the inventory"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
2 3 40 50 20
3

Output: Book with ISBN 3 is available at index 1

Answer

```
# You are using Python
import sys
```

```
def binary_search(arr, n, x):
```

```
    low = 0
```

```
    high = n - 1
```

```
    while low <= high:
```

```
        mid = (low + high) // 2
```

```
        if arr[mid] == x:
```

```
            return mid
```

```
        elif arr[mid] < x:
```

```
            low = mid + 1
```

```
        else:
```

```
            high = mid - 1
```

```
    return -1
```

```
def main():
```

```
    data = sys.stdin.read().split()
```

```
    if not data:
```

```
        return
```

```
    n = int(data[0])
```

```
    arr = list(map(int, data[1:n+1]))
```

```
    x = int(data[n+1])
```

```
    result = binary_search(arr, n, x)
```

```
    if result != -1:
```

```
        print(f"Book with ISBN {x} is available at index {result}")
```

```
    else:
```

```
        print(f"Book with ISBN {x} is not available in the inventory")
```

```
if __name__ == "__main__":
```

```
    main()
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: SUSHANT KUMAR

Email: vtu29971@veltech.edu.in

Roll no: vtu29971

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L10

VTU_DAA_Week 5_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 48

Section 1 : Coding

1. Problem Statement

Suppose you are working on a project that involves sorting a large dataset of student records based on a single criterion, such as their CGPA, age, or major. You have decided to use a modified quicksort algorithm to efficiently sort the dataset while ensuring that the sorting is stable, i.e., that records with equal values maintain their original relative order.

To achieve this, you will implement a modified version of the quicksort algorithm that maintains stability during the sorting process. Specifically, you will use a three-way partitioning scheme that sorts the records into three groups: those with values less than the pivot, those with values equal to the pivot, and those with values greater than the pivot. When the algorithm processes subarrays with equal pivot values, it uses insertion sort to maintain stability.

Write a function that implements this modified version of the quicksort algorithm and uses it to sort the student records according to one specified criterion (CGPA, age, or major). Your function should take as input a list of student records and return the sorted list in ascending order based on the chosen criterion. If two records have the same values for the sorting criterion, their relative order should be maintained in the sorted list.

Input Format

The first line contains an integer n , representing the number of student records to be sorted.

The next n groups of 4 lines each contain the details of each student:

- Line 1: Name (string)
- Line 2: CGPA (float)
- Line 3: Age (integer)
- Line 4: Major (string)

The last line contains the sorting criteria (one of three options: "CGPA", "age", or "major").

Output Format

The output displays the sorted student records in a table format with the following specifications:

Header row: "Name CGPA Age Major"

Each student record is displayed in one row with the following format:

- Name: Left-aligned, 10 characters wide
- CGPA: Left-aligned, 8 characters wide, displayed with 2 decimal places
- Age: Left-aligned, 5 characters wide
- Major: Left-aligned, 20 characters wide

The table is sorted in ascending order according to the chosen sorting criteria.

Refer to the sample outputs for the formatting specifications.

Sample Test Case

Input: 4

harish
7.8
23
ece
trello
8
21
cse
kamalesh
7
25
it
maaran
5.6
20
civil
age

Output: Name CGPA Age Major
maaran 5.60 20 civil
trello 8.00 21 cse
harish 7.80 23 ece
kamalesh 7.00 25 it

Answer

You are using Python
import sys

```
class Student:
    def __init__(self, name, cgpa, age, major):
        self.name = name
        self.cgpa = float(cgpa)
        self.age = int(age)
        self.major = major
```

```
def get_criterion_value(student, criterion):
    c = criterion.lower()
    if c == "cgpa" or c == "gpa":
        return student.cgpa
    elif c == "age":
        return student.age
    elif c == "major":
        return student.major
```

```
return 0
```

```
def stable_quick_sort(records, criterion):  
    if len(records) <= 1:  
        return records
```

```
    pivot_index = len(records) // 2  
    pivot_record = records[pivot_index]  
    pivot_val = get_criterion_value(pivot_record, criterion)
```

```
    left = [x for x in records if get_criterion_value(x, criterion) < pivot_val]  
    middle = [x for x in records if get_criterion_value(x, criterion) == pivot_val]  
    right = [x for x in records if get_criterion_value(x, criterion) > pivot_val]
```

```
    return stable_quick_sort(left, criterion) + middle + stable_quick_sort(right,  
criterion)
```

```
def main():  
    input_data = sys.stdin.read().split()  
    if not input_data:  
        return
```

```
    n = int(input_data[0])  
    records = []  
    current = 1
```

```
    for _ in range(n):  
        name = input_data[current]  
        cgpa = input_data[current + 1]  
        age = input_data[current + 2]  
        major = input_data[current + 3]  
        records.append(Student(name, cgpa, age, major))  
        current += 4
```

```
    criterion = input_data[current]  
    sorted_records = stable_quick_sort(records, criterion)
```

```
    header = f"{'Name':<10}{'CGPA':<8}{'Age':<5}{'Major':<20}"  
    print(header.rstrip())
```

```
    for s in sorted_records:  
        line = f"{s.name:<10}{s.cgpa:<8.2f}{s.age:<5}{s.major:<20}"
```



```
print(line.rstrip())  
if __name__ == "__main__":  
    main()
```

Status : Partially correct

Marks : 8/10

2. Problem Statement

Vishnu, a math enthusiast, is given a task to explore the magic of numbers. He has an array of positive integers, and his goal is to find the integer with the highest digit sum in the sorted array using the merge sort algorithm.

You have to assist Vishnu in implementing the merge sort algorithm.

Example

Input:

6

789 123 456 876 234 567

Output:

The sorted array is: 123 234 456 567 789 876

The integer with the highest digit sum is: 789

Explanation

Sorted Array Output: 123 234 456 567 789 876

The digit sum of an integer is the sum of its individual digits.

In this case, the integer 789 has the highest digit sum ($7 + 8 + 9 = 24$).

Input Format

The first line of input consists of an integer N , representing the number of elements in the array.

The second line consists of N space-separated integers, representing the array elements.

Output Format

The first line prints "The sorted array is: " followed by the sorted array in ascending order.

The second line prints "The integer with the highest digit sum is: " followed by the integer with the highest-digit sum.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

123 456 789 321 654

Output: The sorted array is: 123 321 456 654 789

The integer with the highest digit sum is: 789

Answer

You are using Python
import sys

```
def get_digit_sum(n):  
    s = 0  
    temp = n  
    while temp > 0:  
        s += temp % 10  
        temp //= 10  
    return s
```

```
def merge(arr, l, m, r):  
    n1 = m - l + 1  
    n2 = r - m  
    L = arr[l:m+1]  
    R = arr[m+1:r+1]  
    i = j = 0  
    k = l  
    while i < n1 and j < n2:  
        if L[i] <= R[j]:  
            arr[k] = L[i]  
            i += 1
```

```

else:
    arr[k] = R[j]
    j += 1
    k += 1
while i < n1:
    arr[k] = L[i]
    i += 1
    k += 1
while j < n2:
    arr[k] = R[j]
    j += 1
    k += 1

def merge_sort(arr, l, r):
    if l < r:
        m = l + (r - l) // 2
        merge_sort(arr, l, m)
        merge_sort(arr, m + 1, r)
        merge(arr, l, m, r)

def main():
    input_data = sys.stdin.read().split()
    if not input_data:
        return
    n = int(input_data[0])
    arr = list(map(int, input_data[1:n+1]))

    merge_sort(arr, 0, n - 1)

    max_sum = -1
    ans = -1
    for x in arr:
        d_sum = get_digit_sum(x)
        if d_sum > max_sum:
            max_sum = d_sum
            ans = x

    print("The sorted array is:", *(arr))
    print("The integer with the highest digit sum is:", ans)

if __name__ == "__main__":
    main()

```

Status : Correct

Marks : 10/10

3. Problem Statement

Pavi is working on a project to help a library manage its collection of books efficiently. As part of this project, you need to develop a program to assist the library staff in finding specific books in their extensive catalog.

The library's book collection is organized in such a way that each book is assigned a unique numerical identifier. The library staff has provided you with a list of these identifiers, and you need to write a program to help them find a book's identifier efficiently.

Therefore, you decide to implement a recursive binary search algorithm to quickly locate books based on their unique identifiers.

Input Format

The first line of input contains an integer, n , representing the number of books in the library's collection.

The second line of input consists of n space-separated integers, where each integer represents the unique identifier of a book in the library's collection.

The third line of input contains a single integer t , representing the unique identifier of the book the library staff is searching for.

Output Format

If the book with the identifier target is found in the library's collection, the output prints the index (0-based) of the book in the collection.

If the book is not found, print " t not present", where the t is the unique identifier of a book.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
12 13 14 15 16
14

Output: 2

Answer

You are using Python
import sys

```
def recursive_binary_search(arr, low, high, target):  
    if low > high:  
        return -1  
  
    mid = (low + high) // 2  
    if arr[mid] == target:  
        return mid  
    elif arr[mid] > target:  
        return recursive_binary_search(arr, low, mid - 1, target)  
    else:  
        return recursive_binary_search(arr, mid + 1, high, target)  
  
def main():  
    data = sys.stdin.read().split()  
    if not data:  
        return  
    n = int(data[0])  
    arr = list(map(int, data[1:n+1]))  
    target = int(data[n+1])  
  
    result = recursive_binary_search(arr, 0, n - 1, target)  
  
    if result != -1:  
        print(result)  
    else:  
        print(f"{target} not present")  
  
if __name__ == "__main__":  
    main()
```

Status : Correct

Marks : 10/10

4. Problem Statement

Jack is building a number analyzer tool that identifies numbers based on the sum of their digits. He is given a list of integers and wants to find the smallest number in the list whose digit sum equals a target value d .

To do this efficiently, Jack first sorts the list based on each number's digit sum and then applies binary search to find a match. Write a program to ensure that Jack can determine the correct number using this approach. If no number in the list matches the digit sum d , print -1.

Input Format

The first line of input consists of an integer n , representing the number of elements in the list.

The second line contains n space-separated integers, representing the sorted list.

The third line contains an integer d , representing the target digit sum.

Output Format

The output prints a single integer representing the smallest number from the list whose digit sum equals d , based on sorted digit sum order.

If no such number exists, print -1.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
12 23 30 48 56
3

Output: 12

Answer

```
# You are using Python
import sys
```

```
def get_digit_sum(n):  
    return sum(int(d) for d in str(n))
```

```
def binary_search_first(arr, target):  
    low = 0  
    high = len(arr) - 1  
    res = -1  
    while low <= high:  
        mid = (low + high) // 2  
        if arr[mid][0] == target:  
            res = mid  
            high = mid - 1  
        elif arr[mid][0] < target:  
            low = mid + 1  
        else:  
            high = mid - 1  
    return res
```

```
def main():  
    input_data = sys.stdin.read().split()  
    if not input_data:  
        return  
  
    n = int(input_data[0])  
    raw_list = list(map(int, input_data[1:n+1]))  
    target_d = int(input_data[n+1])  
  
    pairs = []  
    for x in raw_list:  
        pairs.append((get_digit_sum(x), x))  
  
    pairs.sort()  
  
    idx = binary_search_first(pairs, target_d)  
  
    if idx != -1:  
        print(pairs[idx][1])  
    else:  
        print("-1")  
  
if __name__ == "__main__":
```

main()

Status : Correct

Marks : 10/10

5. Problem Statement

Riya, an HR manager, needs a quick way to identify the Kth highest salary among employees for bonus distribution. To efficiently retrieve this information, she wants to use the max-heap technique.

Help Riya implement a solution that finds the Kth highest salary from the given list of salaries.

Input Format

The first line of input consists of an integer N, representing the number of employees.

The second line consists of N space-separated integers, representing the salaries of N employees.

The third line consists of an integer K, indicating the rank of the desired Kth highest salary.

Output Format

The output prints: <k>th largest Salary: <result>

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7
30000 25000 40000 35000 20000 28000 29500
5

Output: 5th largest Salary: 28000

Answer

You are using Python


```
import sys
```

```
def heapify(arr, n, i):
```

```
    largest = i
```

```
    l = 2 * i + 1
```

```
    r = 2 * i + 2
```

```
    if l < n and arr[l] > arr[largest]:
```

```
        largest = l
```

```
    if r < n and arr[r] > arr[largest]:
```

```
        largest = r
```

```
    if largest != i:
```

```
        arr[i], arr[largest] = arr[largest], arr[i]
```

```
        heapify(arr, n, largest)
```

```
def get_kth_largest(arr, n, k):
```

```
    for i in range(n // 2 - 1, -1, -1):
```

```
        heapify(arr, n, i)
```

```
    for i in range(n - 1, n - k - 1, -1):
```

```
        result = arr[0]
```

```
        arr[0] = arr[i]
```

```
        heapify(arr, i, 0)
```

```
    return result
```

```
def main():
```

```
    data = sys.stdin.read().split()
```

```
    if not data:
```

```
        return
```

```
    n = int(data[0])
```

```
    salaries = list(map(int, data[1:n+1]))
```

```
    k = int(data[n+1])
```

```
    result = get_kth_largest(salaries, n, k)
```

```
    print(f"{k}th largest Salary: {result}")
```

```
if __name__ == "__main__":
```

```
    main()
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: SUSHANT KUMAR
Email: vtu29971@veltech.edu.in
Roll no: vtu29971
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L10

VTU_DAA_Week 6_CY

Attempt : 1
Total Mark : 50
Marks Obtained : 24.5

Section 1 : Coding

1. Problem Statement

Virat is given a group of individuals, and he wants to identify if there is an influential individual within the group. An influential individual is defined as someone who is followed by more people than they follow.

He is provided with a list of follow-up relationships among the individuals in the form of a matrix. Your task is to help Virat determine if there is an influential individual and, if so, find their index within the group.

Write a program to solve this problem using Warshall's Algorithm.

Input Format

The first line of input consists of an integer N, representing the number of individuals in the group.

The following N lines will contain N space-separated integers, representing the following relationship matrix for the individuals.

Output Format

If an influential individual is found, print "Influential Individual: X," where X is the index (0-based) of the influential individual.

Otherwise, print "There is no influential individual in the group".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 1 1 0

0 0 1 0

0 0 0 0

0 0 1 0

Output: Influential Individual: 2

Answer

// You are using GCC

#include <stdio.h>

```
int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
```

```
    int adj[10][10];
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &adj[i][j]);
        }
    }
```

```
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (adj[i][k] && adj[k][j]) {
```

```

        adj[i][j] = 1;
    }
}

int found_index = -1;
for (int i = 0; i < n; i++) {
    int following = 0;
    int followed_by = 0;

    for (int j = 0; j < n; j++) {
        if (i != j) {
            if (adj[i][j]) following++;
            if (adj[j][i]) followed_by++;
        }
    }

    if (followed_by > following) {
        found_index = i;
        break;
    }
}

if (found_index != -1) {
    printf("Influential Individual: %d\n", found_index);
} else {
    printf("There is no influential individual in the group\n");
}

return 0;
}

```

Status : Partially correct

Marks : 4.5/10

2. Problem statement

Sarah has been tasked with developing a program to determine the shortest paths among nodes in a directed network represented as a graph containing nodes and weighted edges.

Her goal is to implement the Floyd-Warshall algorithm to compute the shortest path matrix for this directed network.

Input Format

The first line of input consists of an integer n , representing the number of nodes in the network.

The next n lines contain a space-separated integer, representing the adjacency matrix 'graph' of the network. Each integer is the weight of the edge from node i to node j .

Output Format

The output prints the shortest path matrix as ' $n \times n$ ' integers, where each element $\text{graph}[i][j]$ represents the shortest distance from node ' i ' to node ' j '.

Print each row of the matrix on a new line, with elements separated by a space, and if no path exists, 999 will be displayed.

Refer to the sample output for the formatting specifications.

Sample Test Case

```
Input: 4
0 3 999 4
8 0 2 999
5 999 0 1
2 999 999 0
Output: 0 3 5 4
5 0 2 3
3 6 0 1
2 5 7 0
```

Answer

```
// You are using GCC
#include <stdio.h>
```

```
int main() {
    int n;
```

```

if (scanf("%d", &n) != 1) return 0;

int graph;

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        scanf("%d", &graph[i][j]);
    }
}

for (int k = 0; k < n; k++) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (graph[i][k] + graph[k][j] < graph[i][j]) {
                graph[i][j] = graph[i][k] + graph[k][j];
            }
        }
    }
}

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        printf("%d", graph[i][j]);
        if (j < n - 1) {
            printf(" ");
        }
    }
    printf("\n");
}

return 0;
}

```

Status : Wrong

Marks : 0/10

3. Problem Statement

You are working as a backend engineer for a drone delivery logistics company that operates across multiple cities. Each city is represented as a node, and the drone flight paths between cities are represented as directed edges with weights indicating the time (in minutes) it takes for a drone to

travel between them.

Due to varying air traffic, regulations, and geography, the direct travel time between cities may not always be the fastest. Sometimes, going through intermediate cities results in shorter delivery times.

Your task is to build a system that calculates the shortest possible delivery time between every pair of cities, considering all possible intermediate routes.

To efficiently compute this, you're required to implement the Floyd-Warshall Algorithm.

Input Format

The first line of input contains an integer N representing the total number of cities.

The next N lines contains N space-separated integers, representing the delivery time matrix:

$\text{time}[i][j]$ is the time (in minutes) taken to fly directly from City i to City j .

A value of 0 means the same city (i.e., $i == j$).

A value of 999 indicates no direct route between City i and City j .

Output Format

The output prints the shortest distance between every pair of cities.

If there is no shortest path between two cities, then print INF.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
0 5 999 10
999 0 3 999


```
999 999 0 1
999 999 999 0
Output: 0 5 8 9
INF 0 3 4
INF INF 0 1
INF INF INF 0
```

Answer

```
// You are using GCC
#include <stdio.h>
```

```
/**
 * Floyd-Warshall Algorithm for Shortest Path calculation between all city pairs.
 *
 * Logic:
 * 1. Initialize distance matrix from input.
 * 2. Treat 999 as the infinity marker as per problem statement.
 * 3. Use an internal large value for infinity to perform additions safely.
 * 4. Iteratively update paths by considering every city 'k' as an intermediate
    point.
 */
```

```
int main() {
    int n;
    if (scanf("%d", &n) != 1) {
        return 0;
    }

    int dist[100][100];
    const int INPUT_INF = 999;
    const int INTERNAL_INF = 1000000; // Sufficiently large value for N=100 and
    weights=10^4
```

```
    // Read the delivery time matrix
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            int val;
            scanf("%d", &val);
            if (val == INPUT_INF && i != j) {
                dist[i][j] = INTERNAL_INF;
            } else {
                dist[i][j] = val;
            }
        }
    }
```

```

    }
    }
}

// Floyd-Warshall core logic
for (int k = 0; k < n; k++) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            // Check if path through city 'k' is shorter than current direct/known
            path
            if (dist[i][k] != INTERNAL_INF && dist[k][j] != INTERNAL_INF) {
                if (dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }
}

// Print the resulting shortest path matrix
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        if (dist[i][j] >= INTERNAL_INF) {
            printf("INF");
        } else {
            printf("%d", dist[i][j]);
        }
    }

    // Formatting: space between integers, newline at end of row
    if (j < n - 1) {
        printf(" ");
    }
    printf("\n");
}

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Given the weights and values of n items, put these items in a knapsack of capacity W to get the maximum total value in the knapsack. In other words, given two integer arrays, $val[0..n-1]$ and $wt[0..n-1]$ which represent values and weights associated with n items, respectively. Also given an integer W which represents knapsack capacity, find out the maximum value subset of $val[]$ such that the sum of the weights of this subset is smaller than or equal to W . You cannot break an item, either pick the complete item or don't pick it.

Solve this problem using dynamic programming.

Input Format

The first line of input represents the number of items.

The second line of input consists of the values of each item separated by spaces.

The third line of input consists of the weights of each item separated by spaces.

The fourth line of input consists of an integer, which represents the maximum capacity of the knapsack.

Output Format

The output prints the maximum value of items that can be held by the knapsack.

Sample Test Case

Input: 3
60 100 120
10 20 30
50

Output: 220

Answer

```
import sys
```

```
def solve():  
    input_data = sys.stdin.read().split()  
    if not input_data:
```

```

return

n = int(input_data[0])
val = list(map(int, input_data[1:n+1]))
wt = list(map(int, input_data[n+1:2*n+1]))
w_capacity = int(input_data[2*n+1])

dp = [[0 for _ in range(w_capacity + 1)] for _ in range(n + 1)]

for i in range(1, n + 1):
    for w in range(1, w_capacity + 1):
        if wt[i-1] <= w:
            dp[i][w] = max(val[i-1] + dp[i-1][w-wt[i-1]], dp[i-1][w])
        else:
            dp[i][w] = dp[i-1][w]

print(dp[n][w_capacity])

if __name__ == "__main__":
    solve()

```

Status : Correct

Marks : 10/10

5. Problem Statement

Sarah loves challenges and has recently taken up a new programming problem. She is fascinated by the classic "0/1 Knapsack Problem" and wants to create a program that can efficiently solve it.

The 0/1 Knapsack Problem involves selecting a combination of items with given weights and values to maximize the total value, while not exceeding a given weight capacity.

Write a program that takes input for the 0/1 Knapsack Problem and outputs the maximum value that can be achieved, as well as the selected weights that contribute to this maximum value.

Note: The weights are printed in reverse order of the input, meaning the algorithm starts checking from the last item and prints items that are included first.

Input Format

The first line contains an integer n , representing the number of items available.

The second line contains n integers separated by space, representing the values of the items. The i th integer corresponds to the value of the i th item.

The third line contains n integers separated by space, representing the weights of the items. The i th integer corresponds to the weight of the i th item.

The fourth line contains an integer W , representing the maximum weight capacity of the knapsack.

Output Format

The first line output displays "Maximum value: " followed by an integer, representing the maximum value that can be achieved.

The second line output displays "Selected weights: " followed by space-separated integers, representing the selected weights contributing to the maximum value.

Refer to the sample outputs for the exact format.

Sample Test Case

Input: 3

60 100 120

10 20 30

50

Output: Maximum value: 220

Selected weights: 30 20

Answer

```
#include <stdio.h>
```

```
int max(int a, int b) {  
    return (a > b) ? a : b;  
}
```

```
int main() {
```

```

int n, W;
if (scanf("%d", &n) != 1) return 0;

int val;
int wt;

for (int i = 0; i < n; i++) scanf("%d", &val[i]);
for (int i = 0; i < n; i++) scanf("%d", &wt[i]);

if (scanf("%d", &W) != 1) return 0;

int dp[n + 1][W + 1];

for (int i = 0; i <= n; i++) {
    for (int j = 0; j <= W; j++) {
        if (i == 0 || j == 0) {
            dp[i][j] = 0;
        } else if (wt[i - 1] <= j) {
            dp[i][j] = max(val[i - 1] + dp[i - 1][j - wt[i - 1]], dp[i - 1][j]);
        } else {
            dp[i][j] = dp[i - 1][j];
        }
    }
}

printf("Maximum value: %d\n", dp[W]);
printf("Selected weights:");

int res = dp[W];
int current_w = W;
for (int i = n; i > 0 && res > 0; i--) {
    if (res != dp[i - 1][current_w]) {
        printf(" %d", wt[i - 1]);
        res -= val[i - 1];
        current_w -= wt[i - 1];
    }
}
printf("\n");

return 0;
}

```

Status : Wrong

Marks : 0/10

Vel Tech Group of Institutions

Name: SUSHANT KUMAR

Email: vtu29971@veltech.edu.in

Roll no: vtu29971

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L10

VTU_DAA_Week 6_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Anitha is working on packing expensive artifacts into a container that has a limited weight capacity. Each artifact has a weight and a value, and Anitha wants to maximize the total worth of artifacts she can carry, even if it means taking only a fraction of an artifact when the container is nearly full.

The maximum worth is calculated as:

Maximum Worth = $\sum (\text{value}[i] * \text{fraction_taken}[i])$,

where $\text{fraction_taken}[i] = \min(1, \text{remaining_capacity} / \text{weight}[i])$

Write a program to ensure that Anitha can calculate the maximum worth of objects she can carry in the container.

Input Format

The first line contains an integer N, representing the number of items.

The second line contains N space-separated integers representing the weights of the items.

The third line contains N space-separated integers representing the values of the items.

The fourth line contains an integer representing the capacity of the container.

Output Format

The output prints a single line showing the total maximum worth of objects carried as a double with exactly two decimal places, in the following format:

"Objects worth Rs.X.YY"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
10 20 30
60 100 120
50

Output: Objects worth Rs.240.00

Answer

```
// You are using Java
import java.util.*;
```

```
class Item {
    int weight;
    int value;
    double ratio;
```

```
Item(int weight, int value) {  
    this.weight = weight;  
    this.value = value;  
    this.ratio = (double) value / weight;  
}  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
  
        if (!sc.hasNextInt()) return;  
        int n = sc.nextInt();  
  
        int[] weights = new int[n];  
        for (int i = 0; i < n; i++) {  
            weights[i] = sc.nextInt();  
        }  
  
        int[] values = new int[n];  
        for (int i = 0; i < n; i++) {  
            values[i] = sc.nextInt();  
        }  
  
        int capacity = sc.nextInt();  
  
        Item[] items = new Item[n];  
        for (int i = 0; i < n; i++) {  
            items[i] = new Item(weights[i], values[i]);  
        }  
  
        // Sort items by value/weight ratio in descending order  
        Arrays.sort(items, (a, b) -> Double.compare(b.ratio, a.ratio));  
  
        double totalWorth = 0.0;  
        int remainingCapacity = capacity;  
  
        for (Item item : items) {  
            if (remainingCapacity >= item.weight) {  
                totalWorth += item.value;  
                remainingCapacity -= item.weight;  
            } else {  

```

```

        // Take fraction of the item
        totalWorth += item.ratio * remainingCapacity;
        break;
    }
}

System.out.printf("Objects worth Rs.%.2f\n", totalWorth);
sc.close();
}
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Raju is a professional hiker preparing for a long trekking expedition. He has a backpack that can hold a maximum weight of W kilograms. There are N essential items you want to carry, each with a specific weight and importance value (utility score). His goal is to maximize the total importance value of the items you take while ensuring that the total weight does not exceed the backpack's capacity.

He can either include or exclude each item—meaning he cannot take fractions of an item (e.g., you cannot take half of a tent). Given the weights and importance values of the items, determine the maximum possible importance value he can carry in his backpack. Help him to achieve his task.

Input Format

The first line contains two integers, N (number of items) and W (maximum weight the backpack can carry).

The second line contains N integers, representing the weights of the items.

The third line contains N integers, representing the importance values of the items.

Output Format

The output prints a single integer, the maximum importance value that can be

achieved without exceeding the weight limit.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 4 5

2 3 4 5

3 4 5 6

Output: 7

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
int max(int a, int b) {  
    return (a > b) ? a : b;  
}
```

```
int main() {  
    int n, w_limit;  
    if (scanf("%d %d", &n, &w_limit) != 2) return 0;
```

```
    int weights[n];  
    int values[n];
```

```
    for (int i = 0; i < n; i++) {  
        scanf("%d", &weights[i]);
```

```
    }  
    for (int i = 0; i < n; i++) {  
        scanf("%d", &values[i]);  
    }
```

```
    // Using a 1D DP array to optimize space  
    // size is w_limit + 1 to account for 0 capacity  
    static int dp[100001] = {0};
```

```
    for (int i = 0; i < n; i++) {  
        // Iterate backwards to avoid using the same item multiple times  
        for (int w = w_limit; w >= weights[i]; w--) {
```

```

        dp[w] = max(dp[w], values[i] + dp[w - weights[i]]);
    }
}

printf("%d\n", dp[w_limit]);

return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Rohit is responsible for scheduling flights for an airline company. He has a list of N flights numbered from 0 to N-1. Some flights have specific constraints that require them to depart after other flights.

Write a program to help Rohit determine if it's possible to create a flight schedule that adheres to these constraints using Warshall's algorithm.

Example 1

Input:

5

0 1

1 2

2 3

3 4

4 5

Output:

Yes

Explanation:

Flight 0 must depart after Flight 1.

Flight 1 must depart after Flight 2.

Flight 2 must depart after Flight 3.

Flight 3 must depart after Flight 4.

Flight 4 must depart after Flight 5.

There are no circular dependencies or conflicts in the schedule, so it is possible to create a flight schedule that adheres to the constraints.

Example 2

Input:

6

0 1

1 2

2 3

3 4

4 5

5 0

Output:

No

Explanation:

Flight 0 must depart after Flight 1.

Flight 1 must depart after Flight 2.

Flight 2 must depart after Flight 3.

Flight 3 must depart after Flight 4.

Flight 4 must depart after Flight 5.

Flight 5 must depart after Flight 0.

There is a circular dependency in the flight constraints. Therefore, it is not

possible to create a flight schedule that follows the constraints.

Input Format

The first line of input consists of an integer N, representing the number of flights.

The following N lines consist of two space-separated integers each: the flight number and the flight number that must depart before it.

Output Format

If it is possible to create a flight schedule that adheres to the constraints, print "Yes".

Otherwise, print "No".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

0 1

1 2

2 3

3 4

4 5

Output: Yes

Answer

```
// You are using Java
import java.util.*;
```

```
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```
        if (!sc.hasNextInt()) return;
        int n = sc.nextInt();
```

```
        // We use a matrix size based on the potential flight IDs.
```

```
        // Given N constraints and 0-based numbering, we use a fixed size or find
        max.
```

```

// For constraints up to 10, a 20x20 matrix is safe.
int[] adj = new int[20][20];
boolean[] exists = new boolean[20];

for (int i = 0; i < n; i++) {
    int u = sc.nextInt();
    int v = sc.nextInt();
    adj[u][v] = 1;
    exists[u] = true;
    exists[v] = true;
}

// Warshall's Algorithm to find transitive closure
for (int k = 0; k < 20; k++) {
    for (int i = 0; i < 20; i++) {
        for (int j = 0; j < 20; j++) {
            if (adj[i][k] == 1 && adj[k][j] == 1) {
                adj[i][j] = 1;
            }
        }
    }
}

boolean possible = true;
for (int i = 0; i < 20; i++) {
    if (exists[i] && adj[i][i] == 1) {
        possible = false;
        break;
    }
}

if (possible) {
    System.out.println("Yes");
} else {
    System.out.println("No");
}

sc.close();
}
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

A traveler needs to find the shortest path between every pair of cities in a road network. The cities are connected by roads, each with a specific travel cost. To do this, the traveler uses the Floyd-Warshall Algorithm to calculate the shortest distances between every pair of cities in a given edge-weighted undirected graph.

Example1

Input:

4

3

0 1 2

1 2 3

2 3 4

Output:

Original matrix

0 2 INF INF

2 0 3 INF

INF 3 0 4

INF INF 4 0

Shortest path matrix

0 2 5 9

2 0 3 7

5 3 0 4

9 7 4 0

Explanation:

Given: 4 nodes (0, 1, 2, 3) and 3 edges: 0 1 (weight 2), 1 2 (weight 3), 2 3

(weight 4)

Any node pair without a direct edge is considered INF (unreachable initially).

Step 1: Construct Initial Graph (Original Matrix)

0 2 INF INF

2 0 3 INF

INF 3 0 4

INF INF 4 0

Diagonal elements (self to self) are 0. If no direct path exists, it's INF. Step 2: Running Floyd-Warshall Algorithm

Using node 0 as an intermediate: No updates since it only connects to node 1.

Using node 1 as an intermediate: $0 \rightarrow 2$ via 1: $0 \rightarrow 1 (2) + 1 \rightarrow 2 (3) = 5$

$2 \rightarrow 0$ also updates to 5.

Using node 2 as an intermediate: $0 \rightarrow 3$ via 2: $0 \rightarrow 2 (5) + 2 \rightarrow 3 (4) = 9$

$1 \rightarrow 3$ via 2: $1 \rightarrow 2 (3) + 2 \rightarrow 3 (4) = 7$

Using node 3 as an intermediate: No further updates.

Step 3: Final Shortest Path Matrix

0 2 5 9

2 0 3 7

5 3 0 4

9 7 4 0

Input Format

The first line contains an integer V representing the number of cities (vertices) in the network.

The second line contains an integer E representing the number of roads (edges) in the network.

The next E lines each contain three space-separated integers u, v, w, where u and v represent the cities connected by a road, and w represents the travel cost (weight) of the road.

Output Format

The first line of output prints the original matrix, where each entry represents the direct travel cost between two cities. If there is no direct road between two cities, represent the cost as INF.

Then, print the shortest path matrix, where each entry represents the shortest travel cost between every pair of cities, computed using the Floyd-Warshall algorithm.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

3

0 1 2

1 2 3

2 3 4

Output: Original matrix

0 2 INF INF

2 0 3 INF

INF 3 0 4

INF INF 4 0

Shortest path matrix

0 2 5 9

2 0 3 7

5 3 0 4

9 7 4 0

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define INF 1000000
```

```

void printMatrix(int **matrix, int V) {
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            if (matrix[i][j] == INF) {
                printf("INF");
            } else {
                printf("%d", matrix[i][j]);
            }
            if (j < V - 1) printf(" ");
        }
        printf("\n");
    }
}

```

```

int main() {
    int V, E;
    if (scanf("%d %d", &V, &E) != 2) return 0;

    int **dist = (int **)malloc(V * sizeof(int *));
    int **original = (int **)malloc(V * sizeof(int *));
    for (int i = 0; i < V; i++) {
        dist[i] = (int *)malloc(V * sizeof(int));
        original[i] = (int *)malloc(V * sizeof(int));
        for (int j = 0; j < V; j++) {
            if (i == j) {
                dist[i][j] = 0;
                original[i][j] = 0;
            } else {
                dist[i][j] = INF;
                original[i][j] = INF;
            }
        }
    }
}

```

```

for (int i = 0; i < E; i++) {
    int u, v, w;
    scanf("%d %d %d", &u, &v, &w);
    if (u < V && v < V) {
        dist[u][v] = w;
        dist[v][u] = w;
        original[u][v] = w;
    }
}

```

```

        original[v][u] = w;
    }
}

printf("Original matrix\n");
printMatrix(original, V);

for (int k = 0; k < V; k++) {
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            if (dist[i][k] != INF && dist[k][j] != INF &&
                dist[i][k] + dist[k][j] < dist[i][j]) {
                dist[i][j] = dist[i][k] + dist[k][j];
            }
        }
    }
}

printf("Shortest path matrix\n");
printMatrix(dist, V);

for (int i = 0; i < V; i++) {
    free(dist[i]);
    free(original[i]);
}
free(dist);
free(original);

return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

A person starts walking from position $X = 0$, find the probability of reaching exactly on $X = N$ if she can only take either 2 steps or 3 steps. The probability for step length 2 is given i.e. P , and the probability for step length 3 is $1 - P$.

Write a program for the same.

Note: Use Dynamic Programming

Example

Input: N = 5, P = 0.20

Output: 0.32

Explanation:

There are two ways to reach 5.

2+3 with probability = $0.2 * 0.8 = 0.16$

3+2 with probability = $0.8 * 0.2 = 0.16$

So, total probability = 0.32.

Input Format

The input consists of an integer N representing the target position and a double-point number P representing the probability of taking a step of length 2, separated by a space.

Output Format

The output prints a double-point number representing the probability of reaching the exact position N, formatted to two decimal places.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5 0.20

Output: 0.32

Answer

```
# You are using Python
import sys
```

```
def solve():
```

```
input_data = sys.stdin.read().split()
if not input_data:
    return
```

```
n = int(input_data[0])
p = float(input_data[1])
q = 1 - p
```

```
# dp[i] represents the probability of reaching position i
dp = [0.0] * (n + 1)
```

```
# Base case: probability of being at position 0 is 1
dp[0] = 1.0
```

```
for i in range(1, n + 1):
    # To reach position i, she could have come from i-2 or i-3
    prob_from_2 = 0.0
    prob_from_3 = 0.0
```

```
    if i >= 2:
        prob_from_2 = dp[i-2] * p
    if i >= 3:
        prob_from_3 = dp[i-3] * q
```

```
    dp[i] = prob_from_2 + prob_from_3
```

```
print(f"{dp[n]:.2f}")
```

```
if __name__ == "__main__":
    solve()
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: SUSHANT KUMAR

Email: vtu29971@veltech.edu.in

Roll no: vtu29971

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L10

VTU_DAA_Week 6_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Anitha is working on packing expensive artifacts into a container that has a limited weight capacity. Each artifact has a weight and a value, and Anitha wants to maximize the total worth of artifacts she can carry, even if it means taking only a fraction of an artifact when the container is nearly full.

The maximum worth is calculated as:

Maximum Worth = $\sum (\text{value}[i] * \text{fraction_taken}[i])$,

where $\text{fraction_taken}[i] = \min(1, \text{remaining_capacity} / \text{weight}[i])$

Write a program to ensure that Anitha can calculate the maximum worth of objects she can carry in the container.

Input Format

The first line contains an integer N, representing the number of items.

The second line contains N space-separated integers representing the weights of the items.

The third line contains N space-separated integers representing the values of the items.

The fourth line contains an integer representing the capacity of the container.

Output Format

The output prints a single line showing the total maximum worth of objects carried as a double with exactly two decimal places, in the following format:

"Objects worth Rs.X.YY"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
10 20 30
60 100 120
50

Output: Objects worth Rs.240.00

Answer

```
// You are using Java
import java.util.*;
```

```
class Item {
    int weight;
    int value;
    double ratio;
```

```
Item(int weight, int value) {  
    this.weight = weight;  
    this.value = value;  
    this.ratio = (double) value / weight;  
}  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
  
        if (!sc.hasNextInt()) return;  
        int n = sc.nextInt();  
  
        int[] weights = new int[n];  
        for (int i = 0; i < n; i++) {  
            weights[i] = sc.nextInt();  
        }  
  
        int[] values = new int[n];  
        for (int i = 0; i < n; i++) {  
            values[i] = sc.nextInt();  
        }  
  
        int capacity = sc.nextInt();  
  
        Item[] items = new Item[n];  
        for (int i = 0; i < n; i++) {  
            items[i] = new Item(weights[i], values[i]);  
        }  
  
        // Sort items by value/weight ratio in descending order  
        Arrays.sort(items, (a, b) -> Double.compare(b.ratio, a.ratio));  
  
        double totalWorth = 0.0;  
        int remainingCapacity = capacity;  
  
        for (Item item : items) {  
            if (remainingCapacity >= item.weight) {  
                totalWorth += item.value;  
                remainingCapacity -= item.weight;  
            } else {  

```

```

        // Take fraction of the item
        totalWorth += item.ratio * remainingCapacity;
        break;
    }
}

System.out.printf("Objects worth Rs.%.2f\n", totalWorth);
sc.close();
}
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Raju is a professional hiker preparing for a long trekking expedition. He has a backpack that can hold a maximum weight of W kilograms. There are N essential items you want to carry, each with a specific weight and importance value (utility score). His goal is to maximize the total importance value of the items you take while ensuring that the total weight does not exceed the backpack's capacity.

He can either include or exclude each item—meaning he cannot take fractions of an item (e.g., you cannot take half of a tent). Given the weights and importance values of the items, determine the maximum possible importance value he can carry in his backpack. Help him to achieve his task.

Input Format

The first line contains two integers, N (number of items) and W (maximum weight the backpack can carry).

The second line contains N integers, representing the weights of the items.

The third line contains N integers, representing the importance values of the items.

Output Format

The output prints a single integer, the maximum importance value that can be

achieved without exceeding the weight limit.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 4 5

2 3 4 5

3 4 5 6

Output: 7

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
int max(int a, int b) {  
    return (a > b) ? a : b;  
}
```

```
int main() {  
    int n, w_limit;  
    if (scanf("%d %d", &n, &w_limit) != 2) return 0;
```

```
    int weights[n];  
    int values[n];
```

```
    for (int i = 0; i < n; i++) {  
        scanf("%d", &weights[i]);
```

```
    }  
    for (int i = 0; i < n; i++) {  
        scanf("%d", &values[i]);  
    }
```

```
    // Using a 1D DP array to optimize space  
    // size is w_limit + 1 to account for 0 capacity  
    static int dp[100001] = {0};
```

```
    for (int i = 0; i < n; i++) {  
        // Iterate backwards to avoid using the same item multiple times  
        for (int w = w_limit; w >= weights[i]; w--) {
```

```
        dp[w] = max(dp[w], values[i] + dp[w - weights[i]]);
    }
}

printf("%d\n", dp[w_limit]);

return 0;
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Rohit is responsible for scheduling flights for an airline company. He has a list of N flights numbered from 0 to N-1. Some flights have specific constraints that require them to depart after other flights.

Write a program to help Rohit determine if it's possible to create a flight schedule that adheres to these constraints using Warshall's algorithm.

Example 1

Input:

5

0 1

1 2

2 3

3 4

4 5

Output:

Yes

Explanation:

Flight 0 must depart after Flight 1.

Flight 1 must depart after Flight 2.

Flight 2 must depart after Flight 3.

Flight 3 must depart after Flight 4.

Flight 4 must depart after Flight 5.

There are no circular dependencies or conflicts in the schedule, so it is possible to create a flight schedule that adheres to the constraints.

Example 2

Input:

6

0 1

1 2

2 3

3 4

4 5

5 0

Output:

No

Explanation:

Flight 0 must depart after Flight 1.

Flight 1 must depart after Flight 2.

Flight 2 must depart after Flight 3.

Flight 3 must depart after Flight 4.

Flight 4 must depart after Flight 5.

Flight 5 must depart after Flight 0.

There is a circular dependency in the flight constraints. Therefore, it is not

possible to create a flight schedule that follows the constraints.

Input Format

The first line of input consists of an integer N, representing the number of flights.

The following N lines consist of two space-separated integers each: the flight number and the flight number that must depart before it.

Output Format

If it is possible to create a flight schedule that adheres to the constraints, print "Yes".

Otherwise, print "No".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

0 1

1 2

2 3

3 4

4 5

Output: Yes

Answer

```
// You are using Java
import java.util.*;
```

```
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```
        if (!sc.hasNextInt()) return;
        int n = sc.nextInt();
```

```
        // We use a matrix size based on the potential flight IDs.
```

```
        // Given N constraints and 0-based numbering, we use a fixed size or find
        max.
```

```

// For constraints up to 10, a 20x20 matrix is safe.
int[][] adj = new int[20][20];
boolean[] exists = new boolean[20];

for (int i = 0; i < n; i++) {
    int u = sc.nextInt();
    int v = sc.nextInt();
    adj[u][v] = 1;
    exists[u] = true;
    exists[v] = true;
}

// Warshall's Algorithm to find transitive closure
for (int k = 0; k < 20; k++) {
    for (int i = 0; i < 20; i++) {
        for (int j = 0; j < 20; j++) {
            if (adj[i][k] == 1 && adj[k][j] == 1) {
                adj[i][j] = 1;
            }
        }
    }
}

boolean possible = true;
for (int i = 0; i < 20; i++) {
    if (exists[i] && adj[i][i] == 1) {
        possible = false;
        break;
    }
}

if (possible) {
    System.out.println("Yes");
} else {
    System.out.println("No");
}

sc.close();
}
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

A traveler needs to find the shortest path between every pair of cities in a road network. The cities are connected by roads, each with a specific travel cost. To do this, the traveler uses the Floyd-Warshall Algorithm to calculate the shortest distances between every pair of cities in a given edge-weighted undirected graph.

Example1

Input:

4

3

0 1 2

1 2 3

2 3 4

Output:

Original matrix

0 2 INF INF

2 0 3 INF

INF 3 0 4

INF INF 4 0

Shortest path matrix

0 2 5 9

2 0 3 7

5 3 0 4

9 7 4 0

Explanation:

Given: 4 nodes (0, 1, 2, 3) and 3 edges: 0 1 (weight 2), 1 2 (weight 3), 2 3

(weight 4)

Any node pair without a direct edge is considered INF (unreachable initially).

Step 1: Construct Initial Graph (Original Matrix)

0 2 INF INF

2 0 3 INF

INF 3 0 4

INF INF 4 0

Diagonal elements (self to self) are 0. If no direct path exists, it's INF. Step 2: Running Floyd-Warshall Algorithm

Using node 0 as an intermediate: No updates since it only connects to node 1.

Using node 1 as an intermediate: $0 \rightarrow 2$ via 1: $0 \rightarrow 1 (2) + 1 \rightarrow 2 (3) = 5$

$2 \rightarrow 0$ also updates to 5.

Using node 2 as an intermediate: $0 \rightarrow 3$ via 2: $0 \rightarrow 2 (5) + 2 \rightarrow 3 (4) = 9$

$1 \rightarrow 3$ via 2: $1 \rightarrow 2 (3) + 2 \rightarrow 3 (4) = 7$

Using node 3 as an intermediate: No further updates.

Step 3: Final Shortest Path Matrix

0 2 5 9

2 0 3 7

5 3 0 4

9 7 4 0

Input Format

The first line contains an integer V representing the number of cities (vertices) in the network.

The second line contains an integer E representing the number of roads (edges) in the network.

The next E lines each contain three space-separated integers u, v, w, where u and v represent the cities connected by a road, and w represents the travel cost (weight) of the road.

Output Format

The first line of output prints the original matrix, where each entry represents the direct travel cost between two cities. If there is no direct road between two cities, represent the cost as INF.

Then, print the shortest path matrix, where each entry represents the shortest travel cost between every pair of cities, computed using the Floyd-Warshall algorithm.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

3

0 1 2

1 2 3

2 3 4

Output: Original matrix

0 2 INF INF

2 0 3 INF

INF 3 0 4

INF INF 4 0

Shortest path matrix

0 2 5 9

2 0 3 7

5 3 0 4

9 7 4 0

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define INF 1000000
```

```

void printMatrix(int **matrix, int V) {
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            if (matrix[i][j] == INF) {
                printf("INF");
            } else {
                printf("%d", matrix[i][j]);
            }
            if (j < V - 1) printf(" ");
        }
        printf("\n");
    }
}

```

```

int main() {
    int V, E;
    if (scanf("%d %d", &V, &E) != 2) return 0;

    int **dist = (int **)malloc(V * sizeof(int *));
    int **original = (int **)malloc(V * sizeof(int *));
    for (int i = 0; i < V; i++) {
        dist[i] = (int *)malloc(V * sizeof(int));
        original[i] = (int *)malloc(V * sizeof(int));
        for (int j = 0; j < V; j++) {
            if (i == j) {
                dist[i][j] = 0;
                original[i][j] = 0;
            } else {
                dist[i][j] = INF;
                original[i][j] = INF;
            }
        }
    }
}

```

```

for (int i = 0; i < E; i++) {
    int u, v, w;
    scanf("%d %d %d", &u, &v, &w);
    if (u < V && v < V) {
        dist[u][v] = w;
        dist[v][u] = w;
        original[u][v] = w;
    }
}

```

```

        original[v][u] = w;
    }
}

printf("Original matrix\n");
printMatrix(original, V);

for (int k = 0; k < V; k++) {
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            if (dist[i][k] != INF && dist[k][j] != INF &&
                dist[i][k] + dist[k][j] < dist[i][j]) {
                dist[i][j] = dist[i][k] + dist[k][j];
            }
        }
    }
}

printf("Shortest path matrix\n");
printMatrix(dist, V);

for (int i = 0; i < V; i++) {
    free(dist[i]);
    free(original[i]);
}
free(dist);
free(original);

return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

A person starts walking from position $X = 0$, find the probability of reaching exactly on $X = N$ if she can only take either 2 steps or 3 steps. The probability for step length 2 is given i.e. P , and the probability for step length 3 is $1 - P$.

Write a program for the same.

Note: Use Dynamic Programming

Example

Input: N = 5, P = 0.20

Output: 0.32

Explanation:

There are two ways to reach 5.

2+3 with probability = $0.2 * 0.8 = 0.16$

3+2 with probability = $0.8 * 0.2 = 0.16$

So, total probability = 0.32.

Input Format

The input consists of an integer N representing the target position and a double-point number P representing the probability of taking a step of length 2, separated by a space.

Output Format

The output prints a double-point number representing the probability of reaching the exact position N, formatted to two decimal places.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5 0.20

Output: 0.32

Answer

```
# You are using Python
import sys
```

```
def solve():
```

```
input_data = sys.stdin.read().split()
if not input_data:
    return
```

```
n = int(input_data[0])
p = float(input_data[1])
q = 1 - p
```

```
# dp[i] represents the probability of reaching position i
dp = [0.0] * (n + 1)
```

```
# Base case: probability of being at position 0 is 1
dp[0] = 1.0
```

```
for i in range(1, n + 1):
    # To reach position i, she could have come from i-2 or i-3
    prob_from_2 = 0.0
    prob_from_3 = 0.0
```

```
    if i >= 2:
        prob_from_2 = dp[i-2] * p
    if i >= 3:
        prob_from_3 = dp[i-3] * q
```

```
    dp[i] = prob_from_2 + prob_from_3
```

```
print(f"{dp[n]:.2f}")
```

```
if __name__ == "__main__":
    solve()
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: SUSHANT KUMAR
Email: vtu29971@veltech.edu.in
Roll no: vtu29971
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L10

VTU_DAA_Week 7_CY

Attempt : 1
Total Mark : 50
Marks Obtained : 39

Section 1 : Coding

1. Problem Statement

Mary is planning to build a railway network to connect cities efficiently. She needs a program that, given the number of cities and the segments between them, determines the minimum cost to connect all cities using an undirected weighted graph.

Implement a solution using Kruskal's Algorithm to find the Minimum Spanning Tree (MST) of the city network. If it is not possible to connect all cities, output an appropriate message

Example

Input

3

3

0 1 4

1 2 5

0 2 3

Output

7

Explanation

The input specifies 3 cities and 3 track segments with their corresponding weights.

The track segments are:

Track Segment 1: Connects city 0 and city 1, with a weight of 4.

Track Segment 2: Connects city 1 and city 2 with a weight of 5.

Track Segment 3: Connects city 0 and city 2 with a weight of 3.

The algorithm finds the minimum cost to build railway tracks connecting all cities. In this case, the minimum spanning tree includes track segment 3 (0-2 with weight 3) and track segment 1 (0-1 with weight 4). The total cost is $3 + 4 = 7$.

Input Format

The first line of input contains an integer `num_cities`, representing the number of cities (numbered 0 to `num_cities - 1`).

The second line contains an integer `num_segments`, representing the number of undirected segments (edges) between cities.

The next `num_segments` lines each contain three space-separated integers: source, destination, and weight, representing an undirected segment between city source and city destination with the given cost.

Output Format

The first line of output prints an integer representing the minimum cost to connect all cities.

If it is not possible to connect all the cities, the output prints 'It is not possible to connect all cities' (without a period).

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

3

0 1 4

1 2 5

0 2 3

Output: 7

Answer

You are using Python

```
import sys
```

```
class DisjointSet:
```

```
    def __init__(self, n):  
        self.parent = list(range(n))  
        self.components = n
```

```
    def find(self, i):  
        if self.parent[i] == i:  
            return i  
        self.parent[i] = self.find(self.parent[i])  
        return self.parent[i]
```

```
    def union(self, i, j):  
        root_i = self.find(i)  
        root_j = self.find(j)  
        if root_i != root_j:  
            self.parent[root_i] = root_j  
            self.components -= 1  
            return True  
        return False
```

```
def solve():  
    input_data = sys.stdin.read().split()
```

```

if not input_data:
    return

num_cities = int(input_data[0])
num_segments = int(input_data[1])

edges = []
idx = 2
for _ in range(num_segments):
    u = int(input_data[idx])
    v = int(input_data[idx+1])
    w = int(input_data[idx+2])
    edges.append((u, v, w))
    idx += 3

# Sort edges based on weight for Kruskal's
edges.sort(key=lambda x: x[2])

ds = DisjointSet(num_cities)
min_cost = 0
edges_used = 0

for u, v, w in edges:
    if ds.union(u, v):
        min_cost += w
        edges_used += 1

# A graph is connected if we used exactly n-1 edges for n vertices
if edges_used == num_cities - 1:
    print(min_cost)
else:
    print("It is not possible to connect all cities")

if __name__ == "__main__":
    solve()

```

Status : Partially correct

Marks : 9/10

2. Problem Statement

Maya, a project planner, must connect multiple office branches using

network cables while keeping the total cost within a given budget. Each possible connection between two branches has an associated cost.

Write a program to find the Minimum Spanning Tree (MST) of the branch network using Kruskal's Algorithm on an undirected weighted graph. If the total cost of the MST is within the given budget, display the cost. Otherwise, indicate that it is not possible.

It is guaranteed that the graph is connected in all test cases

Input Format

The first line contains three space-separated integers V, E, and budget, representing the number of branches (vertices, numbered 1 to V), the number of possible connections (edges), and the total budget respectively.

Each of the next E lines contains three space-separated integers src, dest, and weight, representing an undirected connection between branch src and branch dest with cost weight.

Output Format

If the MST cost is within the budget, the output prints:

'Minimum Cost Spanning Tree within the budget: X'

where X is an integer representing the total minimum cost.

Otherwise, the output prints:

'A valid MST within the budget is not possible.'

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3 3 5

1 2 2

2 3 3

3 1 4

Output: Minimum Cost Spanning Tree within the budget: 5

Answer

```
# You are using Python
import sys
```

```
class DisjointSet:
    def __init__(self, n):
        self.parent = list(range(n + 1))

    def find(self, i):
        if self.parent[i] == i:
            return i
        self.parent[i] = self.find(self.parent[i])
        return self.parent[i]

    def union(self, i, j):
        root_i = self.find(i)
        root_j = self.find(j)
        if root_i != root_j:
            self.parent[root_i] = root_j
            return True
        return False

def solve():
    input_data = sys.stdin.read().split()
    if not input_data:
        return
    v = int(input_data[0])
    e = int(input_data[1])
    budget = int(input_data[2])

    edges = []
    idx = 3
    for _ in range(e):
        src = int(input_data[idx])
        dest = int(input_data[idx+1])
        weight = int(input_data[idx+2])
        edges.append((src, dest, weight))
        idx += 3

    edges.sort(key=lambda x: x[2])
```

```

ds = DisjointSet(v)
mst_cost = 0
edges_count = 0

for src, dest, weight in edges:
    if ds.union(src, dest):
        mst_cost += weight
        edges_count += 1
        if edges_count == v - 1:
            break

if mst_cost <= budget:
    print(f"Minimum Cost Spanning Tree within the budget: {mst_cost}")
else:
    print("A valid MST within the budget is not possible.")

if __name__ == "__main__":
    solve()

```

Status : Correct

Marks : 10/10

3. Problem Statement

Sita is planning to connect multiple locations in her city with roads. Each possible road between two locations has a construction distance (in kilometres). The city is represented as an undirected weighted graph using an adjacency matrix, where a value of 0 indicates no direct road between two locations (locations are numbered 0 to numLocations - 1).

To minimise the overall construction distance, she decides to use Prim's Algorithm to construct the Minimum Spanning Tree (MST). Help her find the MST edges and the total minimum distance.

Input Format

The first line of input contains an integer numLocations, representing the number of locations (numbered 0 to numLocations - 1).

The next numLocations lines each contain numLocations space-separated integers representing the adjacency matrix, where graph[i][j] is the distance between location i and location j. A value of 0 indicates no direct road between

those two locations.

Output Format

The first line of output prints: 'Location\tDistance'

Each of the next numLocations - 1 lines prints the MST edge in the format:

'parent -> child \t\t\t dist km'

where parent is the parent location number, child is the child location number, and dist is the edge weight (integer) in kilometres.

The last line of output prints: 'Minimum total distance: totalCost km'

where totalCost is an integer.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 2

0 5

5 0

Output: Location Distance

0 -> 1 5 km

Minimum total distance: 5 km

Answer

You are using Python

```
import sys
```

```
def prim_mst():
```

```
    # Read all input data
```

```
    input_data = sys.stdin.read().split()
```

```
    if not input_data:
```

```
        return
```

```

n = int(input_data[0])
graph = []
idx = 1
for i in range(n):
    row = list(map(int, input_data[idx:idx+n]))
    graph.append(row)
    idx += n

# Initialize Prim's variables
key = [float('inf')] * n
parent = [-1] * n
mst_set = [False] * n

# Start at node 0
key[0] = 0

for _ in range(n):
    # Pick the minimum key vertex not yet in MST
    u = -1
    for i in range(n):
        if not mst_set[i] and (u == -1 or key[i] < key[u]):
            u = i

    mst_set[u] = True

    # Update adjacent vertices
    for v in range(n):
        if graph[u][v] > 0 and not mst_set[v] and graph[u][v] < key[v]:
            key[v] = graph[u][v]
            parent[v] = u

# Print output in required format
print("Location\tDistance")
total_distance = 0
# To match sample 2 logic (parent -> child order), we iterate
# based on the order nodes were likely discovered/connected
for v in range(1, n):
    if parent[v] != -1:
        dist = graph[parent[v]][v]
        print(f"{parent[v]} -> {v} \t\t {dist} km")
        total_distance += dist

```



```
print(f"Minimum total distance: {total_distance} km")  
if __name__ == "__main__":  
    prim_mst()
```

Status : Wrong

Marks : 0/10

4. Problem Statement

Riya works at a data tools company that needs a compact representation of frequently used characters for fast transmission. She decides to use Huffman Coding, a lossless compression technique that assigns shorter bit-strings to more frequent symbols and longer bit-strings to less frequent ones.

The Huffman codes should be printed in the order that leaf nodes are encountered during a left-to-right traversal of the constructed Huffman tree.

Write a program that builds a Huffman tree from a list of symbols with their frequencies and prints the Huffman codes for each symbol.

Input Format

The first line contains an integer n , representing the number of distinct symbols.

Each of the next n lines contains a single printable character (the symbol) followed by a space and a positive integer (the frequency), representing the symbol and its occurrence count.

Output Format

The output prints n lines, one per symbol, in the order that leaf nodes are encountered during a left-to-right traversal of the Huffman tree (left child before right child at every internal node).

Each line prints the symbol followed by a colon, a space, and its Huffman code (a binary string of 0s and 1s):

'<symbol>: <binary_code>'

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 2

a 1

b 2

Output: a: 0

b: 1

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
struct Node {  
    char charVal;  
    int freq;  
    struct Node *left, *right;  
};
```

```
struct MinHeap {  
    int size;  
    struct Node** array;  
};
```

```
struct Node* newNode(char c, int f) {  
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));  
    temp->left = temp->right = NULL;  
    temp->charVal = c;  
    temp->freq = f;  
    return temp;  
}
```

```
void swapNodes(struct Node** a, struct Node** b) {  
    struct Node* t = *a;
```

```

    *a = *b;
    *b = t;
}

void minHeapify(struct MinHeap* minHeap, int idx) {
    int smallest = idx;
    int left = 2 * idx + 1;
    int right = 2 * idx + 2;

    if (left < minHeap->size && minHeap->array[left]->freq < minHeap-
>array[smallest]->freq)
        smallest = left;

    if (right < minHeap->size && minHeap->array[right]->freq < minHeap-
>array[smallest]->freq)
        smallest = right;

    if (smallest != idx) {
        swapNodes(&minHeap->array[smallest], &minHeap->array[idx]);
        minHeapify(minHeap, smallest);
    }
}

struct Node* extractMin(struct MinHeap* minHeap) {
    struct Node* temp = minHeap->array[0];
    minHeap->array[0] = minHeap->array[minHeap->size - 1];
    --minHeap->size;
    minHeapify(minHeap, 0);
    return temp;
}

void insertMinHeap(struct MinHeap* minHeap, struct Node* node) {
    ++minHeap->size;
    int i = minHeap->size - 1;
    while (i && node->freq < minHeap->array[(i - 1) / 2]->freq) {
        minHeap->array[i] = minHeap->array[(i - 1) / 2];
        i = (i - 1) / 2;
    }
    minHeap->array[i] = node;
}

void printCodes(struct Node* root, char* code, int top) {

```

```

    if (root->left) {
        code[top] = '0';
        printCodes(root->left, code, top + 1);
    }
    if (root->right) {
        code[top] = '1';
        printCodes(root->right, code, top + 1);
    }
    if (!(root->left) && !(root->right)) {
        code[top] = '\0';
        printf("%c: %s\n", root->charVal, code);
    }
}

```

```

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

```

```

    struct MinHeap* minHeap = (struct MinHeap*)malloc(sizeof(struct MinHeap));
    minHeap->size = 0;
    minHeap->array = (struct Node**)malloc(n * sizeof(struct Node*));

```

```

    for (int i = 0; i < n; i++) {
        char c;
        int f;
        scanf(" %c %d", &c, &f);
        insertMinHeap(minHeap, newNode(c, f));
    }

```

```

    while (minHeap->size != 1) {
        struct Node *left = extractMin(minHeap);
        struct Node *right = extractMin(minHeap);
        struct Node *top = newNode('$', left->freq + right->freq);
        top->left = left;
        top->right = right;
        insertMinHeap(minHeap, top);
    }

```

```

    struct Node* root = extractMin(minHeap);
    char code[256];
    printCodes(root, code, 0);

```

```
    return 0;  
}
```

Status : Correct

Marks : 10/10

5. Problem Statement

In a data science class, Professor Huffman demonstrates how encoded messages can be decoded using a pre-built Huffman tree.

Given a Huffman code table (mapping each character to its binary code) and an encoded binary message, decode the message back to its original text by traversing the Huffman tree.

The code table is guaranteed to be a valid prefix-free Huffman code.

The encoded message consists only of '0' and '1' characters, and every bit maps to exactly one symbol in the Huffman tree

The tree is defined as follows:

Internal nodes are represented by "*". The left edge (0) leads to the symbol "A". The right edge (1) leads to the symbol "B".

This means:

"0" A "1" B

Your task is to decode a given encoded message (a string of 0s and 1s) back into its original form consisting of "A" and "B".

Input Format

The first line contains an integer n , representing the number of symbols in the code table.

Each of the next n lines contains a character and its Huffman code (binary string), separated by a space.

The last line contains the encoded binary message (a string of 0s and 1s) to be decoded

Output Format

The output prints the decoded message as a sequence of characters ('A' or 'B'), followed by a newline.

Refer to the sample output for formatting specifications

Sample Test Case

Input: 11100011100

Output: BBBAABBBA

Answer

You are using Python
import sys

```
def solve():
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    # The last line contains the encoded binary message
    encoded_message = input_data[-1].strip()

    # Decoding logic based on the problem statement:
    # 0 -> A
    # 1 -> B
    decoded_message = []
    for bit in encoded_message:
        if bit == '0':
            decoded_message.append('A')
        elif bit == '1':
            decoded_message.append('B')

    print(''.join(decoded_message))

if __name__ == "__main__":
    solve()
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: SUSHANT KUMAR

Email: vtu29971@veltech.edu.in

Roll no: vtu29971

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L10

VTU_DAA_Week 8_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 26.5

Section 1 : Coding

1. Problem Statement

You are tasked with finding the shortest communication paths from a source router to all other routers in a network using Dijkstra's algorithm.

The network is represented as a graph, where routers are nodes and communication links between them are edges with weights representing the time or cost of communication. A value of 0 in the adjacency matrix indicates no direct link between two routers.

Example

Explanation:

The minimum distance from 0 to 1 = 4. Path: 0 1

The minimum distance from 0 to 2 = 12. Path: 0 1 2

The minimum distance from 0 to 3 = 19. Path: 0 1 2 3

The minimum distance from 0 to 4 = 21. Path: 0 7 6 5 4

The minimum distance from 0 to 5 = 11. Path: 0 7 6 5

The minimum distance from 0 to 6 = 9. Path: 0 7 6

The minimum distance from 0 to 7 = 8. Path: 0 7

The minimum distance from 0 to 8 = 14. Path: 0 1 2 8"

Input Format

The first line contains an integer V, representing the number of vertices in the graph.

The next V lines each contain V space-separated integers, representing the adjacency matrix of the graph. A value of 0 indicates no direct edge between the vertices.

The last line contains an integer src, representing the source vertex.

Output Format

The first line of output prints: Vertex followed by a tab character followed by Distance from Source

The next V lines each print the vertex number followed by three tab characters, a space, and the shortest distance from the source. Vertices are printed in ascending order from 0 to V-1.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

0 1 3 1

1 0 4 2

3 4 0 5

1 2 5 0

1

Output: Vertex Distance from Source

0	1
1	0
2	4
3	2

Answer

You are using Python

```
import sys
```

```
def solve():
```

```
    # Read number of vertices
```

```
    line = sys.stdin.readline().strip()
```

```
    if not line:
```

```
        return
```

```
    v = int(line)
```

```
    # Read adjacency matrix
```

```
    graph = []
```

```
    for _ in range(v):
```

```
        graph.append(list(map(int, sys.stdin.readline().split())))
```

```
    # Read source vertex
```

```
    src = int(sys.stdin.readline().strip())
```

```
    # Initialize distances and visited set
```

```
    distances = [float('inf')] * v
```

```
    visited = [False] * v
```

```
    distances[src] = 0
```

```
    for _ in range(v):
```

```
        # Find the unvisited vertex with the minimum distance
```

```
        min_dist = float('inf')
```

```
        u = -1
```

```
        for i in range(v):
```

```
            if not visited[i] and distances[i] < min_dist:
```

```
                min_dist = distances[i]
```

```
                u = i
```

```
        if u == -1:
```

```
            break
```

```

visited[u] = True

# Update neighbors
for neighbor in range(v):
    weight = graph[u][neighbor]
    # weight > 0 indicates an existing edge
    if weight > 0 and not visited[neighbor]:
        if distances[u] + weight < distances[neighbor]:
            distances[neighbor] = distances[u] + weight

# Final Output Formatting
print("Vertex\tDistance from Source")
for i in range(v):
    # Format requires vertex number, three tabs, a space, and the distance
    print(f"{i}\t\t\t {distances[i]}")

if __name__ == "__main__":
    solve()

```

Status : Correct

Marks : 10/10

2. Problem Statement

Given a network of cities represented as an unweighted graph, find the shortest path between a source city and a destination city in terms of the minimum number of roads to be traversed. Implement a program that utilizes the Dijkstra algorithm to solve this problem.

The program should take the city network, source city, and destination city as inputs, and provide the shortest path length and the sequence of cities to be visited along the path as outputs.

If no path exists between the source and destination, print 'No path found'.
Note: Since the graph is unweighted, all edges have an implicit weight of 1.

Input Format

The first line contains an integer n , representing the number of cities in the network.

The second line contains an integer m , representing the number of roads in the city network.

The following m lines contain pairs of integers u and v , representing a road between cities u and v .

The next line contains an integer s , representing the source city.

The last line contains an integer d , representing the destination city.

Output Format

The first line prints Shortest path length: followed by the number of edges in the shortest path.

The second line prints Path: followed by the city indices along the shortest path, each followed by a space.

If no path exists, print No path found.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 8

10

0 1

0 3

1 2

3 4

3 7

4 5

4 6

4 7

5 6

6 7

0

7

Output: Shortest path length: 2

Path: 0 3 7

Answer

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        if (!sc.hasNextInt()) return;
        int n = sc.nextInt();
        int m = sc.nextInt();

        List<List<Integer>> adj = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            adj.add(new ArrayList<>());
        }

        for (int i = 0; i < m; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();
            adj.get(u).add(v);
            adj.get(v).add(u);
        }

        int start = sc.nextInt();
        int end = sc.nextInt();

        int[] dist = new int[n];
        int[] parent = new int[n];
        Arrays.fill(dist, Integer.MAX_VALUE);
        Arrays.fill(parent, -1);

        dist[start] = 0;
        PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a -
> a[0]));
        pq.add(new int[]{0, start});

        while (!pq.isEmpty()) {
            int[] current = pq.poll();
            int d = current[0];
            int u = current[1];

            if (d > dist[u]) continue;
            if (u == end) break;
```

```
List<Integer> neighbors = adj.get(u);
Collections.sort(neighbors); // Tie-breaking for path consistency
```

```
    for (int v : neighbors) {
        if (dist[u] + 1 < dist[v]) {
            dist[v] = dist[u] + 1;
            parent[v] = u;
            pq.add(new int[]{dist[v], v});
        }
    }
}
```

```
if (dist[end] == Integer.MAX_VALUE) {
    System.out.println("No path found");
} else {
    System.out.println("Shortest path length: " + dist[end]);
    List<Integer> path = new ArrayList<>();
    for (int at = end; at != -1; at = parent[at]) {
        path.add(at);
    }
    Collections.reverse(path);
    System.out.print("Path: ");
    for (int i = 0; i < path.size(); i++) {
        System.out.print(path.get(i) + (i == path.size() - 1 ? "" : " "));
    }
    System.out.println();
}
}
```

Status : Partially correct

Marks : 5.5/10

3. Problem Statement

Liam, a compiler designer, is optimizing task scheduling in a dependency-based system. He is working with a Directed Acyclic Graph (DAG), where each node represents a task, and a directed edge from one node to another signifies a dependency.

To schedule the tasks optimally, Liam must perform a topological sort using Depth-First Search (DFS), sorting the tasks based on their departure times in descending order.

Your task is to help Liam implement this topological sorting algorithm.

Input Format

The first line contains an integer N, representing the number of vertices (tasks) in the DAG.

The second line contains an integer E, representing the number of directed edges (dependencies) in the graph.

The next E lines each contain two space-separated integers:

- src The starting vertex of a directed edge.
- dest The ending vertex of a directed edge.

Output Format

The output prints N space-separated integers in a single line, representing the 0-indexed vertices in descending order of their DFS departure times, each followed by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

6

5 2

5 0

4 0

4 1

2 3

3 1

Output: 5 4 2 3 1 0

Answer

You are using Python

```
import sys
```

```
# Increase recursion depth for DFS if needed
sys.setrecursionlimit(2000)
```

```
def dfs(u, adj, visited, stack):
    visited[u] = True
```

```
    # Visit all neighbors of the current task
    # Sort neighbors to maintain consistent ordering if needed,
    # though any valid DFS order works for topological sort.
    for v in adj[u]:
        if not visited[v]:
            dfs(v, adj, visited, stack)
```

```
    # Departure point: Task is finished, add to stack
    stack.append(u)
```

```
def solve():
    input_data = sys.stdin.read().split()
    if not input_data:
        return
```

```
    it = iter(input_data)
    n = int(next(it))
    e = int(next(it))
```

```
    adj = [[] for _ in range(n)]
    for _ in range(e):
        u = int(next(it))
        v = int(next(it))
        adj[u].append(v)
```

```
    visited = [False] * n
    stack = []
```

```
    # Process all vertices (to handle disconnected components)
    # We iterate 0 to N-1 to ensure a consistent starting order
    for i in range(n):
        if not visited[i]:
            dfs(i, adj, visited, stack)
```

```

# The stack contains tasks in ascending order of departure times.
# To get descending order, we reverse the stack.
result = stack[::-1]

# Print the tasks separated by spaces
print(*(result), end=" ")

if __name__ == "__main__":
    solve()

```

Status : Partially correct

Marks : 1/10

4. Problem Statement:

You are given a directed acyclic graph (DAG) G with N vertices and M edges. From G , a new graph G^* is constructed using the same vertex set and the following rules:

If there exists an edge from vertex u to vertex v in G , then there is no edge from v to u in G^* . If there exists an edge from u to v in G , and from v to w in G , then there exists an edge from u to w in G^* . In G^* , find the maximum possible size of a set S of vertices such that for every pair of vertices (u, v) in S , there exists a directed edge between them (either $u \rightarrow v$ or $v \rightarrow u$).

Example:

Input: $N=3$, $M=2$, edges: 1 2, 2 3

G^* contains edges: 1 2, 2 3, and 1 3 (by rule 2).

Set $S = \{1, 2, 3\}$: edge exists between every pair (1 2, 2 3, 1 3). This is valid with size 3.

Output: 3

Input Format

The first line contains two space-separated integers N and M , representing the number of vertices and edges in graph G .

The next M lines each contain two space-separated integers u and v ($1 \leq u, v \leq N$), representing a directed edge from vertex u to vertex v .

Output Format

Print a single integer representing the maximum possible size of set S.

Refer to the sample output for formatting specifications

Sample Test Case

Input: 3 2

1 2

2 3

Output: 3

Answer

```
// You are using GCC
#include <stdio.h>
```

```
#define MAXN 101
```

```
int adj[MAXN][MAXN];
int dp[MAXN];
int N, M;
```

```
// Function to find the longest path ending at vertex u
```

```
int findLongestPath(int u) {
```

```
    // If already calculated, return the stored value
```

```
    if (dp[u] != -1) {
```

```
        return dp[u];
```

```
    }
```

```
    int max_prev = 0;
```

```
    for (int v = 1; v <= N; v++) {
```

```
        // If there is an edge from v to u in the original DAG
```

```
        if (adj[v][u]) {
```

```
            int path_len = findLongestPath(v);
```

```
            if (path_len > max_prev) {
```

```
                max_prev = path_len;
```

```
            }
```

```
        }
```

```
    }
```

```

    // Length including current vertex
    return dp[u] = 1 + max_prev;
}

int main() {
    if (scanf("%d %d", &N, &M) != 2) return 0;

    // Initialize adjacency matrix and DP table
    for (int i = 0; i <= N; i++) {
        dp[i] = -1;
        for (int j = 0; j <= N; j++) {
            adj[i][j] = 0;
        }
    }

    for (int i = 0; i < M; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
    }

    int max_size = 0;
    for (int i = 1; i <= N; i++) {
        int current_path = findLongestPath(i);
        if (current_path > max_size) {
            max_size = current_path;
        }
    }

    printf("%d\n", max_size);

    return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

You are given a directed acyclic graph (DAG) represented as an adjacency list.

Your task is to perform a topological sort on the graph and print the vertices in topological order. The input graph is guaranteed to be a directed acyclic graph.

Input Format

The first line of input consists of an integer N, representing the number of vertices in the directed acyclic graph.

The second line consists of an integer E, representing the number of directed edges in the graph.

The following E lines consist of two space-separated integers, src and dest, representing a directed edge from vertex src to vertex dest.

Output Format

The output consists of the vertices in topological order separated by spaces.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6

6

5 2

5 0

4 0

4 1

2 3

3 1

Output: 4 5 0 2 3 1

Answer

// You are using Java

import java.util.*;

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);
```

```

        if (!sc.hasNextInt()) return;
        int n = sc.nextInt();
        if (!sc.hasNextInt()) return;
        int e = sc.nextInt();

        List<List<Integer>> adj = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            adj.add(new ArrayList<>());
        }

        for (int i = 0; i < e; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();
            adj.get(u).add(v);
        }

        boolean[] visited = new boolean[n];
        Stack<Integer> stack = new Stack<>();

        for (int i = n - 1; i >= 0; i--) {
            if (!visited[i]) {
                dfs(i, adj, visited, stack);
            }
        }

        while (!stack.isEmpty()) {
            System.out.print(stack.pop() + (stack.isEmpty() ? "" : " "));
        }
    }

    private static void dfs(int u, List<List<Integer>> adj, boolean[] visited,
        Stack<Integer> stack) {
        visited[u] = true;
        List<Integer> neighbors = adj.get(u);
        Collections.sort(neighbors, Collections.reverseOrder());

        for (int v : neighbors) {
            if (!visited[v]) {
                dfs(v, adj, visited, stack);
            }
        }
        stack.push(u);
    }

```

}
}

Status : Wrong

Marks : 0/10

Vel Tech Group of Institutions

Name: SUSHANT KUMAR

Email: vtu29971@veltech.edu.in

Roll no: vtu29971

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L10

VTU_DAA_Week 10_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 42

Section 1 : Coding

1. Problem Statement

Meera, the director of a national disaster response team, must deploy three specialized rescue units to three critical disaster sites. Each rescue unit has a different response time (in minutes) to reach each site due to distance, terrain, and accessibility constraints.

However, Meera has an additional operational rule:

Rescue Unit 1 cannot be assigned to Site 3 due to equipment limitations. Rescue Unit 2 must not be assigned to Site 1 because of route restrictions.

Meera decides to use the Branch and Bound technique to explore only valid and efficient assignments while respecting all constraints.

Your task is to determine the total response time of the best valid

deployment plan that satisfies all given restrictions.

Input Format

The input consists of three lines with three space-separated integers each, representing a 3×3 response time matrix.

Each cell (i, j) represents the response time (in minutes) if the ith rescue unit is deployed to the jth disaster site.

Output Format

The output should display the total response time in the following exact format: "Total Response Time: X", where X is the total response time of the best valid deployment plan.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 12 25 40
30 14 28
16 18 11

Output: Total Response Time: 37

Answer

```
// You are using GCC
#include <stdio.h>
```

```
int main() {
    int a[3][3];
    for(int i = 0; i < 3; i++) {
        for(int j = 0; j < 3; j++) {
            scanf("%d", &a[i][j]);
        }
    }
}
```

```
int min = 1000000;
```

```
for(int j1 = 0; j1 < 3; j1++) {
    if(j1 == 2) continue;
```

```

for(int j2 = 0; j2 < 3; j2++) {
    if(j2 == 0 || j2 == j1) continue;
    for(int j3 = 0; j3 < 3; j3++) {
        if(j3 == j1 || j3 == j2) continue;

        int sum = a[0][j1] + a[1][j2] + a[2][j3];
        if(sum < min) min = sum;
    }
}

printf("Total Response Time: %d", min);
return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Captain Aidan, a daring pirate, has discovered an ancient treasure cave filled with valuable relics. However, the cave is protected by an ancient spell that limits the weight of the treasure he can carry. Aidan has a magic knapsack with a maximum weight capacity of W . Inside the cave, he finds n different treasures, each with a specific weight and value.

Since he can only take each treasure once, Aidan must carefully choose which items to take to maximize the total value without exceeding the weight limit of his knapsack.

Help Captain Aidan pick the most valuable treasures while staying within the weight limit using branch and bound technique

Input Format

The first line contains two integers n (the number of items) and W (the capacity of the knapsack), separated by a space.

The second line contains n integers, where the i -th integer represents the weight of the i -th item.

The third line contains n integers, where the i-th integer represents the value of the i-th item

Output Format

The output prints the single integer representing the maximum value obtained by filling the knapsack with the given set of items.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5 60
10 20 30 40 50
60 100 120 140 180
Output: 280

Answer

```
// You are using GCC
#include <stdio.h>
```

```
int n, W;
int wt[105], val[105];
int maxVal = 0;
```

```
void solve(int i, int currW, int currV) {
    if (currW <= W && currV > maxVal)
        maxVal = currV;

    if (i >= n) return;

    if (currW + wt[i] <= W)
        solve(i + 1, currW + wt[i], currV + val[i]);

    solve(i + 1, currW, currV);
}
```

```
int main() {
    scanf("%d %d", &n, &W);

    for (int i = 0; i < n; i++)
```

```
scanf("%d", &wt[i]);  
for (int i = 0; i < n; i++)  
    scanf("%d", &val[i]);  
  
solve(0, 0, 0);  
  
printf("%d", maxVal);  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Commander Vikram is preparing a spacecraft for an interplanetary research mission. The spacecraft has a limited cargo capacity, so he must carefully decide which scientific equipment modules to load.

Each module has a scientific importance value and a specific weight. Commander Vikram wants to load the spacecraft in such a way that the total scientific value of the selected modules is maximized without exceeding the cargo weight limit.

To solve this efficiently, he uses the Branch and Bound technique applied to the 0/1 Knapsack problem, which explores possible combinations of equipment and eliminates options that cannot lead to better solutions.

Your task is to help Commander Vikram determine the maximum achievable value and the selected module weights that should be loaded into the spacecraft.

Note: The weights must be printed in reverse order of the input, meaning the algorithm starts evaluating items from the last item and prints the selected weights first.

Input Format

The first line contains an integer n , representing the number of equipment

modules available.

The second line contains n space-separated integers $val[i]$, representing the scientific value of each module.

The third line contains n space-separated integers $wt[i]$, representing the weight of each module.

The fourth line contains an integer W , representing the maximum cargo capacity of the spacecraft.

Output Format

The first line output displays "Maximum value: " followed by an integer, representing the maximum value that can be achieved.

The second line output displays "Selected weights: " followed by space-separated integers, printed in reverse order of the input.

Refer to the sample outputs for the exact format.

Sample Test Case

Input: 3

60 100 120

10 20 30

50

Output: Maximum value: 220

Selected weights: 30 20

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
int n, W;
```

```
int val[20], wt[20];
```

```
int best = 0;
```

```
int include[20], bestSet[20];
```

```
void knapsack(int i, int currWeight, int currValue) {
```

```
    if (currWeight <= W && currValue > best) {
```

```

        best = currValue;
        for (int j = 0; j < n; j++)
            bestSet[j] = include[j];
    }

    if (i >= n) return;

    include[i] = 1;
    knapsack(i + 1, currWeight + wt[i], currValue + val[i]);

    include[i] = 0;
    knapsack(i + 1, currWeight, currValue);
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        scanf("%d", &val[i]);

    for (int i = 0; i < n; i++)
        scanf("%d", &wt[i]);

    scanf("%d", &W);

    knapsack(0, 0, 0);

    printf("Maximum value: %d\n", best);
    printf("Selected weights: ");

    for (int i = n - 1; i >= 0; i--) {
        if (bestSet[i])
            printf("%d ", wt[i]);
    }

    return 0;
}

```

Status : Partially correct

Marks : 2/10

4. Problem Statement

You are tasked with solving the Travelling Salesman Problem (TSP) for a given set of cities.

Given the distances between pairs of cities, your goal is to find the shortest route that visits each city exactly once and returns to the starting city.

Note: Solve it using Branch and Bound approach.

Input Format

The first line of input contains an integer n , representing the number of cities.

The following n lines each contain n space-separated integers, representing the distances between pairs of cities.

The distance between city i and city j is given at the i th row and j th column of the matrix.

Output Format

The first line displays "The Path is: " followed by the sequence of cities visited in the shortest path,

separated by "--->".

The second line displays "Minimum cost is X ", where X is the minimum cost of the shortest path found.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 4 1 3

4 0 2 1

1 2 0 5

3 1 5 0

Output: The Path is: 1--->3--->2--->4--->1

Minimum cost is 7

Answer

```
// You are using GCC
#include <stdio.h>
#include <limits.h>
```

```
int n;
int cost[11][11];
int visited[11];
int final_path[11];
int temp_path[11];
int min_cost = INT_MAX;
```

```
void tsp(int level, int curr_city, int count, int curr_cost) {
    if (count == n && cost[curr_city][1] > 0) {
        int total_cost = curr_cost + cost[curr_city][1];
        if (total_cost < min_cost) {
            min_cost = total_cost;
            for (int i = 0; i < n; i++)
                final_path[i] = temp_path[i];
            final_path[n] = 1;
        }
        return;
    }
}
```

```
for (int i = 1; i <= n; i++) {
    if (!visited[i] && cost[curr_city][i] > 0) {
        visited[i] = 1;
        temp_path[level] = i;
        tsp(level + 1, i, count + 1, curr_cost + cost[curr_city][i]);
        visited[i] = 0;
    }
}
}
```

```
int main() {
    scanf("%d", &n);
```

```
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            scanf("%d", &cost[i][j]);
```

```
    for (int i = 1; i <= n; i++)
```

```

visited[i] = 0;

visited[1] = 1;
temp_path[0] = 1;

tsp(1, 1, 1, 0);

printf("The Path is: ");
for (int i = 0; i < n; i++)
    printf("%d-->", final_path[i]);
printf("%d\n", final_path[n]);

printf("Minimum cost is %d", min_cost);

return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

You are a logistics manager responsible for planning delivery routes for a fleet of vehicles. Each vehicle needs to visit a set of locations to deliver goods, and the cost of traveling between locations is known.

Your goal is to find the shortest route for each vehicle, ensuring that each location is visited exactly once before returning to the starting point.

Note: Solve it using Branch and Bound approach.

Input Format

The first line contains an integer n , the number of locations.

The next n lines contain n integers each, representing the cost matrix of traveling between locations.

The i th integer on the j th line represents the cost of traveling from location i to location j .

Output Format

The first line of output displays "Shortest Path: " followed by the shortest path that visits all cities.

The second line of output displays "Minimum cost is " followed by the minimum cost.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
0 10 8 9 7
10 0 10 5 6
8 10 0 8 9
9 5 8 0 6
7 6 9 6 0

Output: Shortest Path: 1 3 4 2 5 1
Minimum cost is 34

Answer

// You are using Java
import java.util.*;

```
public class Main {
    static int n;
    static int[][] cost;
    static boolean[] visited;
    static int[] bestPath, currPath;
    static int minCost = Integer.MAX_VALUE;

    static void tsp(int level, int currCity, int count, int currCost) {
        if (count == n && cost[currCity][0] > 0) {
            int total = currCost + cost[currCity][0];
            if (total < minCost) {
                minCost = total;
                for (int i = 0; i < n; i++)
                    bestPath[i] = currPath[i];
                bestPath[n] = 0;
            }
        }
        return;
    }
}
```



```

    }
    for (int i = 0; i < n; i++) {
        if (!visited[i] && cost[currCity][i] > 0) {
            visited[i] = true;
            currPath[level] = i;
            tsp(level + 1, i, count + 1, currCost + cost[currCity][i]);
            visited[i] = false;
        }
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    n = sc.nextInt();

    cost = new int[n][n];
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            cost[i][j] = sc.nextInt();

    visited = new boolean[n];
    bestPath = new int[n + 1];
    currPath = new int[n];

    visited[0] = true;
    currPath[0] = 0;

    tsp(1, 0, 1, 0);

    System.out.print("Shortest Path: ");
    for (int i = 0; i < n; i++)
        System.out.print((bestPath[i] + 1) + " ");
    System.out.println((bestPath[n] + 1));

    System.out.println("Minimum cost is " + minCost);
}
}

```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: SUSHANT KUMAR
Email: vtu29971@veltech.edu.in
Roll no: vtu29971
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L10

VTU_DAA_Week 12_CY

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

You are given a graph, and your task is to find the minimum vertex cover for the given graph. A vertex cover is a subset of vertices in the graph such that every edge in the graph is incident to at least one vertex in the cover.

Input Format

The first line contains an integer 'vertices' representing the number of vertices in the graph.

The second line contains an integer 'edges' representing the number of edges in the graph.

The following 'edges' lines each contain two integers 'u' and 'v' representing an edge between vertices 'u' and 'v'.

Output Format

The output prints the minimum vertex cover for the given graph, where each vertex in the cover is separated by a space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 7

6

0 1

0 2

1 3

3 4

4 5

5 6

Output: 0 1 3 4 5 6

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
int main() {
```

```
    int v, e;
```

```
    if (scanf("%d %d", &v, &e) != 2) return 0;
```

```
    int edges[500][2];
```

```
    bool visited[100] = {false};
```

```
    bool edge_covered[500] = {false};
```

```
    for (int i = 0; i < e; i++) {
```

```
        scanf("%d %d", &edges[i][0], &edges[i][1]);
```

```
    }
```

```
    bool first = true;
```

```
    for (int i = 0; i < e; i++) {
```

```
        if (!edge_covered[i]) {
```

```
            int u = edges[i][0];
```

```

    int w = edges[i][1];

    if (!first) printf(" ");
    printf("%d %d", u, w);
    first = false;

    for (int j = 0; j < e; j++) {
        if (edges[j][0] == u || edges[j][1] == u || edges[j][0] == w || edges[j][1] == w)
        {
            edge_covered[j] = true;
        }
    }
}
printf("\n");

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

A financial analyst is studying the daily profit and loss values of a company over a period of time. Each day's profit or loss is recorded as an integer in an array, where positive values indicate profit and negative values indicate loss.

The analyst wants to identify a continuous period of days during which the company achieved the maximum total profit. This period must consist of consecutive days only, and at least one day must be selected.

Given an array of integers representing daily profit and loss values, determine the maximum possible sum of any contiguous subarray.

To solve this efficiently, you must use a linear-time algorithm.

Input Format

The first line contains an integer n , representing the number of days.

The second line contains n space-separated integers representing daily profit or loss values.

Output Format

Print a single integer representing the maximum sum of a contiguous subarray.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 9

-2 1 -3 4 -1 2 1 -5 4

Output: 6

Answer

// You are using GCC

#include <stdio.h>

```
int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    long long current_max, global_max;
    long long val;

    if (scanf("%lld", &val) == 1) {
        current_max = val;
        global_max = val;
    } else {
        return 0;
    }

    for (int i = 1; i < n; i++) {
        if (scanf("%lld", &val) != 1) break;
        if (val > current_max + val) {
            current_max = val;
        } else {
            current_max = current_max + val;
        }
    }
}
```

```

    if (current_max > global_max) {
        global_max = current_max;
    }
}

printf("%lld\n", global_max);

return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Given an array of integers, determine whether there exists a majority element in the array.

A majority element is defined as an element that appears more than $n/2$ times in the array.

Your task is to implement a function `majorityElement()` that takes the input array and returns the majority element if it exists. If no such element exists, return -1.

Formula

An element is considered a majority element if:

$$\text{frequency}(\text{element}) > n/2$$

where n is the total number of elements in the array.

Input Format

The first line of input contains an integer n , representing the number of elements in the array.

The second line contains n space-separated integers $a_1 a_2 a_3 \dots a_n$ representing

the array elements.

Output Format

Print the majority element if it exists. Otherwise, print -1.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 9

3 3 4 2 4 4 2 4 4

Output: 4

Answer

// You are using GCC

#include <stdio.h>

```
int main() {  
    int n;  
    if (scanf("%d", &n) != 1) return 0;
```

```
    int a[n];  
    for (int i = 0; i < n; i++) {  
        scanf("%d", &a[i]);  
    }
```

```
    int candidate = -1, count = 0;  
    for (int i = 0; i < n; i++) {  
        if (count == 0) {  
            candidate = a[i];  
            count = 1;  
        } else if (a[i] == candidate) {  
            count++;  
        } else {  
            count--;  
        }  
    }
```

```
    int freq = 0;  
    for (int i = 0; i < n; i++) {
```

```

    if (a[i] == candidate) {
        freq++;
    }

    if (freq > n / 2) {
        printf("%d\n", candidate);
    } else {
        printf("-1\n");
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Aarav is analyzing temperature fluctuations recorded over several days. The temperature changes are stored in an array of integers. He wants to identify a contiguous period of days such that the product of the temperature changes during that period is maximized.

Given an array of integers, determine the maximum product that can be obtained from any contiguous subarray containing at least one element.

Your task is to compute this value efficiently.

Formula

To correctly handle both positive and negative values, track both the maximum and minimum product ending at each index.

If the current element is negative, swap the previous maximum and minimum values before updating.

At each index i :

$\text{currentMax} = \max(\text{arr}[i], \text{arr}[i] \times \text{currentMax})$

$\text{currentMin} = \min(\text{arr}[i], \text{arr}[i] \times \text{currentMin})$

The final answer is the maximum value obtained among all currentMax values.

Input Format

The first line contains an integer n, representing the number of days.

The second line contains n space-separated integers representing the temperature change factors.

Output Format

The first line contains an integer n, representing the number of days.

The second line contains n space-separated integers representing the temperature change factors.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

2 3 -2 4 -1 5

Output: 240

Answer

```
// You are using GCC
#include <stdio.h>
```

```
int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    long long arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%lld", &arr[i]);
    }

    long long max_so_far = arr[0];
    long long min_so_far = arr[0];
```

```

long long result = arr[0];
for (int i = 1; i < n; i++) {
    if (arr[i] < 0) {
        long long temp = max_so_far;
        max_so_far = min_so_far;
        min_so_far = temp;
    }

    long long prod1 = arr[i] * max_so_far;
    if (arr[i] > prod1) max_so_far = arr[i];
    else max_so_far = prod1;

    long long prod2 = arr[i] * min_so_far;
    if (arr[i] < prod2) min_so_far = arr[i];
    else min_so_far = prod2;

    if (max_so_far > result) result = max_so_far;
}

printf("%lld\n", result);

return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

Sidhant is a logistics manager responsible for planning delivery routes for a fleet of trucks. He has a list of cities, and the distances between every pair of cities are given in the form of a matrix.

Each truck must start from a central warehouse (city 0), visit every other city exactly once, and return back to the warehouse. The goal is to minimize the total travel distance.

Sidhant needs to determine the minimum distance required to complete the route and also print the optimal path taken.

Input Format

The first line contains an integer n , representing the number of cities.

The next n lines contain n space-separated integers, representing the distance matrix.

Output Format

The first line of output prints the minimum travel distance.

In the next line, print the optimal path starting and ending at city 0.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0
Output: 80
0 1 3 2 0

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>
```

```
#define MAX 15
#define INF 1e9
```

```
int n;
int dist[MAX][MAX];
int memo[1 << MAX][MAX];
int parent[1 << MAX][MAX];
```

```
int tsp(int mask, int pos) {
    if (mask == (1 << n) - 1) {
        return dist[pos][0];
```

```

    }
    if (memo[mask][pos] != -1) {
        return memo[mask][pos];
    }

    int ans = INF;
    for (int next = 0; next < n; next++) {
        if (!(mask & (1 << next))) {
            int newAns = dist[pos][next] + tsp(mask | (1 << next), next);
            if (newAns < ans) {
                ans = newAns;
                parent[mask][pos] = next;
            }
        }
    }
    return memo[mask][pos] = ans;
}

```

```

void printPath(int mask, int pos) {
    printf("%d ", pos);
    if (mask == (1 << n) - 1) {
        printf("0\n");
        return;
    }
    printPath(mask | (1 << parent[mask][pos]), parent[mask][pos]);
}

```

```

int main() {
    if (scanf("%d", &n) != 1) return 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &dist[i][j]);
        }
    }
}

```

```

memset(memo, -1, sizeof(memo));
int min_dist = tsp(1, 0);

```

```

printf("%d\n", min_dist);
printPath(1, 0);

```

```

return 0;

```

}

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: SUSHANT KUMAR
Email: vtu29971@veltech.edu.in
Roll no: vtu29971
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L10

VTU_DAA_Week 3_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 13

Section 1 : MCQ

1. What is the primary characteristic of the Brute Force approach to solving the Closest-Pair problem?

Answer

Computing the distance between every pair of points and selecting the minimum

Status : Correct

Marks : 1/1

2. In the brute force convex hull algorithm (Jarvis March/Gift Wrapping), what is the next point selected after identifying a point on the hull?

Answer

The point that creates the smallest counterclockwise turn relative to the current point

Status : Correct

Marks : 1/1

3. In C, if you have a structure `Point { int x; int y; }`, what is the correct formula to calculate the squared distance between two points `p1` and `p2` to avoid unnecessary floating-point operations?

Answer

`(p1.x - p2.x)*(p1.x - p2.x) + (p1.y - p2.y)*(p1.y - p2.y)`

Status : Correct

Marks : 1/1

4. Given an array of `n` points in a 2D plane, what is the exact time complexity of the Brute Force Closest-Pair algorithm?

Answer

$O(n^2)$

Status : Correct

Marks : 1/1

5. Which geometric test is essential for determining if a point lies to the left or right of a directed line in the Convex Hull brute force algorithm?

Answer

The Orientation test (Cross Product)

Status : Correct

Marks : 1/1

6. If two points are exactly the same in the input set for the Closest Pair problem, what will the brute force algorithm correctly return?

Answer

A distance of 0

Status : Correct

Marks : 1/1

7. In the brute force algorithm to check if a set of points is in a convex

position, the sign of the cross product indicates:

Answer

The direction of the turn (Left or Right)

Status : Correct

Marks : 1/1

8. Given 5 points in C where all x-coordinates are identical, what is the result of the brute force closest pair algorithm?

Answer

The distance between the two points with the closest y-coordinates

Status : Correct

Marks : 1/1

9. What is the worst-case time complexity of the Brute Force algorithm for checking if given points form a convex polygon?

Answer

$O(n^2)$

Status : Correct

Marks : 1/1

10. In a C program solving the Closest Pair problem via brute force, which nested loop structure correctly avoids redundant calculations (i.e., doesn't calculate distance between A,B and B,A)?

Answer

`for(i=0; i`

Status : Wrong

Marks : 0/1

11. When implementing the Brute Force Closest Pair algorithm in C, what is the minimal number of distance calculations required if there are exactly 3 points?

Answer

3

Status : Correct

Marks : 1/1

12. In the brute force method for finding the convex hull (checking all edges), an edge (i, j) is part of the convex hull if:

Answer

All other points lie on one side of the line through i and j

Status : Correct

Marks : 1/1

13. Which of the following best describes "collinearity" in the context of the Convex Hull problem when using brute force?

Answer

Points lying exactly on the hull edge

Status : Correct

Marks : 1/1

14. What is a significant disadvantage of the Brute Force Convex Hull algorithm compared to Divide and Conquer algorithms?

Answer

It is $O(n^2)$ in the worst case and $O(n)$ in the best case (for Jarvis March)

Status : Correct

Marks : 1/1

15. Which of the following correctly represents the Euclidean distance between two points (x1,y1) and (x2,y2) in C?

Answer

``sqrt(pow(x2 - x1, 2) + pow(y2 - y1, 2))``

Status : Wrong

Marks : 0/1

Vel Tech Group of Institutions

Name: SUSHANT KUMAR
Email: vtu29971@veltech.edu.in
Roll no: vtu29971
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L10

VTU_DAA_Week 4_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 15

Section 1 : MCQ

1. Which of the following methods can be used to solve the Knapsack problem?

Answer

All the mentioned options

Status : Correct

Marks : 1/1

2. 0/1 knapsack is based on _____method.

Answer

Dynamic programming

Status : Correct

Marks : 1/1

3. What is the primary advantage of using the Branch and Bound approach for the Knapsack problem?

Answer

It systematically explores possible solutions while pruning non-promising ones.

Status : Correct

Marks : 1/1

4. Which data structure is commonly used to implement the Branch and Bound approach for solving the Knapsack problem?

Answer

Priority Queue

Status : Correct

Marks : 1/1

5. In the Branch and Bound approach, what does the term “bounding function” refer to?

Answer

A function that estimates the upper bound on the maximum value that can be achieved from a node.

Status : Correct

Marks : 1/1

6. Which strategy is commonly used in the Branch and Bound method to decide which node to explore next?

Answer

Best-first search

Status : Correct

Marks : 1/1

7. Which of the following methods is used to prune the search space in the Branch and Bound method for the Knapsack problem?

Answer

Ignoring nodes with a weight exceeding the capacity.

Status : Correct

Marks : 1/1

8. The travelling salesman problem can be solved using _____.

Answer

a minimum spanning tree

Status : Correct

Marks : 1/1

9. Travelling salesman problem is an example of _____.

Answer

None of the mentioned options

Status : Correct

Marks : 1/1

10. How does the practical Travelling salesman problem differ from the classical Travelling salesman problem?

Answer

In the practical travelling salesman problem, each vertex can be visited more than once.

Status : Correct

Marks : 1/1

11. The traveling salesman problem involves n cities with paths connecting the cities. The time taken for traversing through all the cities without knowing in advance the length of a minimum tour is

Answer

$O(n!)$

Status : Correct

Marks : 1/1

12. In which search problem, to find the shortest path, each city must be

visited only once?

Answer

Travelling Salesman problem

Status : Correct

Marks : 1/1

13. What is the main goal of using Branch and Bound in TSP?

Answer

To minimize search space by pruning unpromising paths

Status : Correct

Marks : 1/1

14. Which value helps in deciding whether a node should be pruned in Branch and Bound?

Answer

Bound (lower bound of cost)

Status : Correct

Marks : 1/1

15. Given items with weights [10, 20, 30] and values [60, 100, 120], what is the value-to-weight ratio for the second item?

Answer

5

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: SUSHANT KUMAR
Email: vtu29971@veltech.edu.in
Roll no: vtu29971
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L10

VTU_DAA_Week 12_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 15

Section 1 : MCQ

1. How many conditions have to be met if an NP-complete problem is polynomially reducible?

Answer

2

Status : Correct

Marks : 1/1

2. _____ is the class of decision problems that can be solved by non-deterministic polynomial algorithms.

Answer

NP

Status : Correct

Marks : 1/1

3. To which of the following classes does a CNF-satisfiability problem belong?

Answer

NP complete

Status : Correct

Marks : 1/1

4. For problems X and Y, Y is NP-complete and X reduces to Y in polynomial time. Which of the following is TRUE?

Answer

X is in NP, but not necessarily NP-complete

Status : Correct

Marks : 1/1

5. A problem in NP is NP-complete if

Answer

The 3-SAT problem can be reduced to it in polynomial time

Status : Correct

Marks : 1/1

6. Which class contains problems that can be solved in polynomial time using a deterministic machine?

Answer

P

Status : Correct

Marks : 1/1

7. What is the key property of problems in NP?

Answer

Their solutions can be verified in polynomial time

Status : Correct

Marks : 1/1

8. What does it mean if a problem is in both NP and NP-Complete?

Answer

It is the hardest among NP problems

Status : Correct

Marks : 1/1

9. If $P = NP$ is ever proven, what would it imply?

Answer

All problems in NP can be solved in polynomial time

Status : Correct

Marks : 1/1

10. What is the class NP named after?

Answer

Nondeterministic Polynomial time

Status : Correct

Marks : 1/1

11. Which complexity class includes problems that are at least as hard as the hardest problems in NP?

Answer

NP-hard

Status : Correct

Marks : 1/1

12. What is the defining property of NP-hard problems?

Answer

They are at least as hard as the hardest problems in NP.

Status : Correct

Marks : 1/1

13. In computational complexity theory, what does "NP" stand for?

Answer

Non-Deterministic Polynomial Time

Status : Correct

Marks : 1/1

14. What is the main characteristic that distinguishes NP-complete problems from NP-hard problems?

Answer

NP-complete problems are the hardest problems in NP.

Status : Correct

Marks : 1/1

15. The concept of "polynomial-time reduction" is commonly used to establish which of the following relationships between problems?

Answer

NP-hardness

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: SUSHANT KUMAR
Email: vtu29971@veltech.edu.in
Roll no: vtu29971
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L10

VTU_DAA_Week 11_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 15

Section 1 : Coding

1. _____ is a matching with the largest number of edges.

Answer

Maximum bipartite matching

Status : Correct

Marks : 1/1

2. How many colors are used in a bipartite graph?

Answer

2

Status : Correct

Marks : 1/1

3. What is the simplest method to prove that a graph is bipartite?

Answer

It does not have a cycle of an odd length

Status : Correct

Marks : 1/1

4. A matching that matches all the vertices of a graph is called?

Answer

Perfect matching

Status : Correct

Marks : 1/1

5. In a bipartite graph $G=(V,U,E)$, the matching of a free vertex in V to a free vertex in U called?

Answer

Augmenting

Status : Correct

Marks : 1/1

6. There are four students in a class namely A, B, C and D. A tells that a triangle is a bipartite graph. B tells pentagon is a bipartite graph. C tells square is a bipartite graph. D tells heptagon is a bipartite graph. Who among the following is correct?

Answer

C

Status : Correct

Marks : 1/1

7. Which of the following is not a property of the bipartite graph?

Answer

Asymmetric spectrum

Status : Correct

Marks : 1/1

8. The maximum number of edges in a bipartite graph with 14 vertices is _____.

Answer

49

Status : Correct

Marks : 1/1

9. What does the Halting Problem attempt to determine?

Answer

Whether a program will eventually stop running or continue forever

Status : Correct

Marks : 1/1

10. The Halting Problem was introduced by which computer scientist?

Answer

Alan Turing

Status : Correct

Marks : 1/1

11. Why is the Halting Problem considered unsolvable?

Answer

Because no algorithm can determine for every program whether it halts or runs forever

Status : Correct

Marks : 1/1

12. In computation theory, what does an undecidable problem mean?

Answer

A problem that cannot be solved by any algorithm for all inputs

Status : Correct

Marks : 1/1

13. In the context of the Halting Problem, what does the term halting refer to?

Answer

A program stopping its execution after completing its task

Status : Correct

Marks : 1/1

14. What is the main implication of the Halting Problem for computer science?

Answer

Some computational problems cannot be solved by any algorithm

Status : Correct

Marks : 1/1

15. The Halting Problem generally takes which of the following as input?

Answer

A program and its input data

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: SUSHANT KUMAR
Email: vtu29971@veltech.edu.in
Roll no: vtu29971
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L10

VTU_DAA_Week 5_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 12

Section 1 : MCQ

1. In heap sort, when does the actual sorting take place?

Answer

During the swap and heapify phase

Status : Correct

Marks : 1/1

2. In heap sort, which of the following heap operations is performed to build a valid heap from an unsorted array?

Answer

Heapify

Status : Correct

Marks : 1/1

3. In heap sort, the element that is swapped with the final element of the heap is _____, in the case of a max-heap, sorted in descending order.

Answer

Smallest element

Status : Wrong

Marks : 0/1

4. What is the worst-case time complexity of the heap sort algorithm?

Answer

$O(n^2)$

Status : Wrong

Marks : 0/1

5. In a quick sort algorithm, where are the smaller elements placed to the pivot during the partition process, assuming we are sorting in increasing order?

Answer

To the left of the pivot

Status : Correct

Marks : 1/1

6. Which of the following is true about Quicksort?

Answer

It is an in-place sorting algorithm

Status : Correct

Marks : 1/1

7. What is the worst-case time complexity of the Quicksort algorithm?

Answer

$O(n^2)$

Status : Correct

Marks : 1/1

8. Given a sorted array of 1000 unique integers, how many maximum comparisons does Binary Search make in the worst case?

Answer

10

Status : Correct

Marks : 1/1

9. Suppose you are using Binary Search on a sorted array of even length (e.g., 8 elements). At each step, you calculate the middle as:

$\text{mid} = (\text{low} + \text{high}) // 2$

Which of the following statements is always true?

Answer

The mid index may not divide the array into equal halves in all steps.

Status : Correct

Marks : 1/1

10. You apply Binary Search on the sorted array [3, 7, 10, 15, 21, 30] to find the element 13. Which sequence of middle elements will Binary Search compare with before declaring that the element is not present?

Answer

10 15 21

Status : Wrong

Marks : 0/1

11. You are using Binary Search to find the first occurrence of a value x in a sorted array that may contain duplicates. Which of the following modifications is necessary?

Answer

When $x == \text{arr}[\text{mid}]$, move high to mid - 1.

Status : Correct

Marks : 1/1

12. A student implements Binary Search and calculates the middle index as:

$\text{mid} = (\text{low} + \text{high}) // 2$

But when the input values of low and high are both large (e.g., low = 2_000_000_000, high = 2_000_000_010), the program crashes due to integer overflow. What is the best fix?

Answer

Use $\text{mid} = \text{low} + (\text{high} - \text{low}) // 2$

Status : Correct

Marks : 1/1

13. Is Merge Sort a stable sorting algorithm?

Answer

Yes, always stable.

Status : Correct

Marks : 1/1

14. Merge sort is _____.

Answer

Comparison-based sorting algorithm

Status : Correct

Marks : 1/1

15. Which of the following methods is used for sorting in merge sort?

Answer

merging

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: SUSHANT KUMAR
Email: vtu29971@veltech.edu.in
Roll no: vtu29971
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L10

VTU_DAA_Week 8_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 11

Section 1 : MCQ

1. Which type of graphs can be topologically sorted?

Answer

Directed graphs

Status : Correct

Marks : 1/1

2. Topological sort can be used to detect which of the following?

Answer

Cycles in a directed graph

Status : Correct

Marks : 1/1

3. In topological sort, vertices are visited in which order?

Answer

In topological order

Status : Correct

Marks : 1/1

4. Which of the following data structures can be used to implement topological sort?

Answer

All of the listed options

Status : Wrong

Marks : 0/1

5. Which of the following algorithms can be used to perform topological sort?

Answer

Breadth First Search (BFS)

Depth First Search (DFS)

Status : Correct

Marks : 1/1

6. Topological sort is equivalent to which of the traversals in trees?

Answer

Post-order traversal

Status : Wrong

Marks : 0/1

7. When is the topological sort of a graph unique?

Answer

In the presence of single node with indegree 0

Status : Wrong

Marks : 0/1

8. In Dijkstra's algorithm, if two vertices u and v have the same tentative distance from the source, which one is chosen first?

Answer

It can be arbitrary

Status : Correct

Marks : 1/1

9. In a graph with non-negative weights, why is Dijkstra's algorithm guaranteed to find the shortest path?

Answer

Because once a vertex's distance is finalized, it cannot be improved

Status : Correct

Marks : 1/1

10. How can Dijkstra's algorithm be adapted to return not only distances but also the actual paths?

Answer

Maintain a parent array for each vertex

Status : Correct

Marks : 1/1

11. In Dijkstra's algorithm, what is the purpose of the relaxation step when processing an edge (u, v) ?

Answer

To update the distance to vertex v if a shorter path through u is found

Status : Correct

Marks : 1/1

12. In a graph with multiple edges between the same pair of vertices, how should Dijkstra's algorithm handle them?

Answer

Consider only the edge with the minimum weight

Status : Correct

Marks : 1/1

13. If a graph has an edge weight of zero, will Dijkstra's algorithm still work correctly?

Answer

Yes, always

Status : Correct

Marks : 1/1

14. Which property of Dijkstra's algorithm guarantees that the shortest path to a vertex is correct when the vertex is added to the finalized set?

Answer

Greedy choice property

Status : Wrong

Marks : 0/1

15. Why can Dijkstra's algorithm fail on a graph with a negative weight edge even if there are no negative cycles?

Answer

Because a previously finalized vertex may get a shorter path later

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: SUSHANT KUMAR
Email: vtu29971@veltech.edu.in
Roll no: vtu29971
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L10

VTU_DAA_Week 7_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 13

Section 1 : MCQ

1. The frequencies of letters in a file are:

$s = 17, k = 24, y = 37$

A Huffman tree is constructed, and we assign 0 to the left branch and 1 to the right branch at each node.

Which of the following is the Huffman code for the letter 's'?

Answer

10

Status : Correct

Marks : 1/1

2. In Huffman coding, does data in a tree always occur?

Answer

Leaves

Status : Correct

Marks : 1/1

3. Which of the following algorithms is the best approach for solving Huffman codes?

Answer

Greedy algorithm

Status : Correct

Marks : 1/1

4. From the following given tree, what is the computed codeword for 'c'?

Answer

110

Status : Correct

Marks : 1/1

5. From the following given tree, what is the code word for the character 'a'?

Answer

011

Status : Correct

Marks : 1/1

6. Kruskal's algorithm is used to _____.

Answer

Find minimum spanning tree

Status : Correct

Marks : 1/1

7. Consider the following graph:

Which edges would be included in the minimum spanning tree using Prim's algorithm starting from vertex A?

Answer

AB, BD, DE, EC, CF

Status : Wrong

Marks : 0/1

8. Which of the following is true?

Answer

Prim's algorithm initializes with a vertex

Status : Correct

Marks : 1/1

9. Prim's algorithm can be efficiently implemented using _____ for graphs with greater density.

Answer

fibonacci heap

Status : Wrong

Marks : 0/1

10. Which of the following is false about Prim's algorithm?

Answer

It constructs MST by selecting edges in increasing order of their weights

Status : Correct

Marks : 1/1

11. What is the worst-case time complexity of Prim's algorithm if the adjacency matrix is used?

Answer

$O(V^2)$

Status : Correct

Marks : 1/1

12. Consider a graph $G = (V, E)$, where $V = \{v_1, v_2, \dots, v_{100}\}$, $E = \{(v_i, v_j) \mid 1 \leq i < j \leq 100\}$ and the weight of the edge (v_i, v_j) is $i - j$. The weight of the minimum spanning tree of G is _____.

Answer

99

Status : Correct

Marks : 1/1

13. An undirected graph $G(V, E)$ contains n ($n > 2$) nodes named $v_1, v_2, \dots, v_n$. Two nodes v_i, v_j are connected if and only if $0 < |i - j| \leq 2$. Each edge (v_i, v_j) is assigned a weight $i + j$. A sample graph with $n = 4$ is shown below. What will be the cost of the minimum spanning tree (MST) of such a graph with n nodes?

Answer

$n^2 - n + 1$

Status : Correct

Marks : 1/1

14. An undirected graph G has n nodes. Its adjacency matrix is given by an $n \times n$ square matrix whose (i) diagonal elements are 0's and (ii) non-diagonal elements are 1's. which one of the following is TRUE?

Answer

Graph G has multiple distinct MSTs, each of cost $n-1$

Status : Correct

Marks : 1/1

15. Consider a weighted complete graph G on the vertex set $\{v_1, v_2, \dots, v_n\}$ such that the weight of the edge (v_i, v_j) is $2|i-j|$. The weight of a minimum spanning tree of G is:

Answer

$2n-2$

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: SUSHANT KUMAR
Email: vtu29971@veltech.edu.in
Roll no: vtu29971
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L10

VTU_DAA_Week 9_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 13

Section 1 : MCQ

1. The problem of finding a path in a graph that visits every vertex exactly once is called?

Answer

Hamiltonian path problem

Status : Correct

Marks : 1/1

2. In the Hamiltonian Cycle problem, if a graph contains a Hamiltonian cycle, it is called a

Answer

Hamiltonian graph

Status : Correct

Marks : 1/1

3. What is the primary difficulty in finding a Hamiltonian Path in a general graph?

Answer

It is an NP-complete problem.

Status : Correct

Marks : 1/1

4. Which of the following statements is true regarding the backtracking algorithm for the Hamiltonian Cycle problem?

Answer

It explores different paths in the graph to find a Hamiltonian cycle.

Status : Correct

Marks : 1/1

5. Which algorithmic paradigm is commonly used to solve the Hamiltonian Cycle problem?

Answer

Backtracking

Status : Correct

Marks : 1/1

6. The following paradigm can be used to find the solution to the problem in minimum time: Given a set of non-negative integers, and a value K, determine if there is a subset of the given set with a sum equal to K:

Answer

Dynamic Programming

Status : Correct

Marks : 1/1

7. What is a subset sum problem?

Answer

Finding a subset of a set that has the sum of elements equal to a given number

Status : Wrong

Marks : 0/1

8. Which of the following is not true about the subset sum problem?

Answer

the dynamic programming solution has a time complexity of $O(n \log n)$

Status : Correct

Marks : 1/1

9. What is the worst case time complexity of dynamic programming solution of the subset sum problem(sum=given subset sum)?

Answer

$O(\text{sum} * n)$

Status : Correct

Marks : 1/1

10. Placing N-queens so that no two queens attack each other is called _____.

Answer

N-queen's problem

Status : Correct

Marks : 1/1

11. Where is the n-queens problem implemented?

Answer

Chess

Status : Correct

Marks : 1/1

12. Which of the following methods can be used to solve N-Queen's problem?

Answer

Backtracking

Status : Correct

Marks : 1/1

13. Which one of the following is an application of the backtracking algorithm?

Answer

Crossword

Status : Correct

Marks : 1/1

14. For N Queens problem, the time complexity is

Answer

$O(N!)$

Status : Correct

Marks : 1/1

15. How many unique solutions are there to the 4 Queens problem, where solutions that are reflections or rotations of each other are considered the same?

Answer

1

Status : Wrong

Marks : 0/1

Vel Tech Group of Institutions

Name: SUSHANT KUMAR
Email: vtu29971@veltech.edu.in
Roll no: vtu29971
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L10

VTU_DAA_Week 10_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 15

Section 1 : MCQ

1. Which of the following methods is commonly used to solve assignment problems?

Answer

Hungarian method

Status : Correct

Marks : 1/1

2. The application of assignment problems is to obtain _____

Answer

Minimum cost or maximum profit

Status : Correct

Marks : 1/1

3. The assignment problem is said to be balanced if _____.

Answer

the number of rows is equal to the number of columns

Status : Correct

Marks : 1/1

4. The minimum number of lines covering all zeros in a reduced cost matrix of order n can be _____.

Answer

at the most n

Status : Correct

Marks : 1/1

5. The assignment problem _____.

Answer

All of the mentioned options

Status : Correct

Marks : 1/1

6. You are given a knapsack that can carry a maximum weight of 60. There are 4 items with weights of {20, 30, 40, and 70} and values {70, 80, 90, and 200}.

What is the maximum value of the items you can carry in the knapsack?

Answer

160

Status : Correct

Marks : 1/1

7. What is the objective of the knapsack problem?

Answer

To Get Maximum Total Value In The Knapsack

Status : Correct

Marks : 1/1

8. Which of the following methods is used to prune the search space in the Branch and Bound method for the Knapsack problem?

Answer

Ignoring nodes with a weight exceeding the capacity.

Status : Correct

Marks : 1/1

9. What is the key difference between Backtracking and Branch and Bound in solving the Knapsack problem?

Answer

Backtracking does not use any bounding function, while Branch and Bound does.

Status : Correct

Marks : 1/1

10. What is the primary advantage of using the Branch and Bound approach for the Knapsack problem?

Answer

It systematically explores possible solutions while pruning non-promising ones.

Status : Correct

Marks : 1/1

11. The travelling salesman problem can be solved using _____.

Answer

a minimum spanning tree

Status : Correct

Marks : 1/1

12. In a traveling salesman problem, the elements of diagonal from left-top to right-bottom are _____.

Answer

all infinity

Status : Correct

Marks : 1/1

13. Which type of graph is used to model the TSP problem?

Answer

Undirected Weighted Graph

Status : Correct

Marks : 1/1

14. What is the main goal of using Branch and Bound in TSP?

Answer

To minimize search space by pruning unpromising paths

Status : Correct

Marks : 1/1

15. Which value helps in deciding whether a node should be pruned in Branch and Bound?

Answer

Bound (lower bound of cost)

Status : Correct

Marks : 1/1